

HÖHERE TECHNISCHE BUNDESLEHRANSTALT
KLAGENFURT, MÖSSINGERSTRASSE

ABTEILUNG ELEKTRONIK UND TECHNISCHE
INFORMATIK

DIPLOMARBEIT

EVENT INSIGHT



JAHRGANG 5AHEL

eingereicht von

Lukas Peter Sumann
Maximilian Wolf Leopold Lindner

Projektbetreuer

Dipl.-Ing. Alexander Rodiga

Diese Diplomarbeit entspricht den Standards gemäß dem Leitfaden zur Umsetzung der Reife- und Diplomprüfung des BMBWF in der letztgültigen Fassung.

Klagenfurt, am 04.04.2025

EIDESSTATTLICHE ERKLÄRUNG

Ich versichere an Eides statt, dass ich diese Diplomarbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe. Alle Gedanken, die im Wortlaut oder in grundlegenden Inhalten aus unveröffentlichten Texten oder aus veröffentlichter Literatur übernommen, oder mit künstlicher Intelligenz generiert wurden, sind ordnungsgemäß gekennzeichnet, zitiert und mit genauer Quellenangabe versehen.

Verfasser / Verfasserin



Lukas Peter Sumann



Maximilian Wolf Leopold
Lindner

Klagenfurt, am 04.04.2025

1.1. Kurzfassung

Aufgabenstellung

Ziel dieses Projekts ist die Entwicklung einer Webapplikation, die es ermöglicht, Renndaten von Sportveranstaltungen in Echtzeit zu visualisieren und die Positionen der Teilnehmenden live darzustellen. Über die SimpleAPI der race result AG werden die Rennzeiten und relevanten Daten abgerufen und in einer zentralen Webansicht präsentiert. Diese Lösung soll es sowohl den Veranstaltenden als auch den Zusehenden ermöglichen, die Renndaten effizient zu verwalten und das Rennen in Echtzeit zu verfolgen. Ein besonderer Fokus liegt auf der Benutzerfreundlichkeit und der kostengünstigen Umsetzung.

Echtzeit-Datenvisualisierungen und die Interaktivität haben in der Eventorganisation und für die Zusehenden zunehmend an Bedeutung gewonnen. Bestehende Systeme bieten Lösungen für kleinere Veranstaltungen, jedoch fehlt eine skalierbare und benutzerfreundliche Lösung für Events mit vielen Teilnehmenden. Das Projekt zielt darauf ab, eine Lösung zu schaffen, die es ermöglicht, Renndaten aus mehreren Quellen in Echtzeit zu verarbeiten und auf eine übersichtliche Weise darzustellen.

Das übergeordnete Ziel dieser Diplomarbeit ist es, ein System zu entwickeln, das sowohl den Veranstaltenden eine einfache Verwaltung der Renndaten ermöglicht als auch den Zusehenden eine interaktive, dynamische Ansicht der Ergebnisse bietet. Die Lösung soll benutzerfreundlich und einfach zu bedienen sein, sodass sie auch von Personen ohne technische Vorkenntnisse schnell eingesetzt werden kann. [1]

Realisierung

Die Webapplikation wird entwickelt, um die Renndaten in Echtzeit von der SimpleAPI der Race Result AG abzurufen und diese auf einer benutzerfreundlichen Oberfläche darzustellen. Die Applikation besteht aus zwei Hauptkomponenten: einer Verwaltungsoberfläche für die Veranstaltenden und einer Anzeigefläche für die Zusehenden. Die Daten werden kontinuierlich über eine API abgerufen und auf einer interaktiven Karte angezeigt, die den Zusehenden ermöglicht, das Rennen live zu verfolgen.

Die Entwicklung erfolgt mit React, wodurch die Applikation auf allen gängigen Plattformen (Desktop und Mobilgeräte) zugänglich ist. Der Server, der die Daten verarbeitet und an die Webapplikation weitergibt, basiert auf Node.js und Express. Sicherheitsprotokolle wie HTTPS sorgen für eine sichere Datenübertragung. Durch den Einsatz von Docker wird gewährleistet, dass die Applikation auf verschiedenen Systemen stabil und problemlos funktioniert.

Ergebnisse

Alle festgelegten Ziele wurden erfolgreich erreicht. Es ist nun möglich, beliebig viele Renndaten zu integrieren und deren Ergebnisse in Echtzeit darzustellen. Die Webapplikation ermöglicht eine benutzerfreundliche Anzeige der Rennergebnisse und eine interaktive Darstellung der aktuellen Positionen der Teilnehmenden. Die Anwendung funktioniert stabil und ist

einfach zu bedienen, sodass sowohl Veranstaltende als auch Zusehende die Plattform ohne technische Vorkenntnisse nutzen können. Die Echtzeitverfolgung und die präzise Darstellung der Renndaten auf der interaktiven Karte gewährleisten ein optimales Nutzererlebnis.

1.2. Abstract

Assignment of Tasks

The goal of this project is to develop a web application that allows for the real-time visualization of race data from sporting events and the live tracking of participants' positions. The race times and relevant data are retrieved via the SimpleAPI from race result AG and presented in a central web interface. This solution is designed to enable both event organizers and spectators to efficiently manage the race data and track the race in real-time.

A particular focus is placed on user-friendliness and cost-effective implementation. Real-time data visualizations and the interactivity of the spectator experience have increasingly become crucial in event organization and spectator engagement. While existing systems cater to smaller events, there is a gap in the market for scalable and user-friendly solutions for large-scale events with many participants. The project aims to provide a solution that processes race data from multiple sources in real-time and presents it in an easy-to-understand manner. The overarching objective is to create a system that simplifies race data management for event organizers while providing spectators with an interactive, dynamic view of the results. The solution is intended to be user-friendly and easily deployable, even for individuals without technical expertise.

Implementation

The web application is developed to retrieve real-time race data from the SimpleAPI of race result AG and display it on a user-friendly interface. The application consists of two main components: an admin interface for event organizers and a display interface for spectators. The data is continuously retrieved via an API and displayed on an interactive map, allowing spectators to follow the race live.

The development is carried out using React, ensuring the application is accessible on all major platforms (desktop and mobile devices). The server, which processes the data and sends it to the web application, is based on Node.js and Express. Security protocols such as HTTPS are used to ensure secure data transmission. Docker is utilized to ensure that the application functions smoothly across different systems.

Kurztitel: Event Insight

Schlüsselwörter: Renndaten, Echtzeitvisualisierung, Webapplikation, Live-Tracking, Benutzerfreundlichkeit

Results

All defined goals were successfully achieved. It is now possible to integrate any number of race data and display the results in real-time. The web application provides a user-friendly display of race results and an interactive representation of the participants' current positions. The application is stable and easy to use, ensuring that both event organizers and spectators can navigate the platform without technical knowledge. The real-time tracking and precise representation of race data on the interactive map guarantee an optimal user experience.

Short title: Event Insight

Keywords: race data, real-time visualization, web application, live tracking, user-friendliness

1.1. Kurzfassung	3
1.2. Abstract	4
2. Einleitung.....	8
2.1. Ausgangslage und Problemstellung	8
2.2. Aufgabenstellung und Zielsetzung	8
2.3. Fachliches und wirtschaftliches Umfeld.....	8
3. Grundlagen und Methoden.....	10
3.1. Ist-Situation / Stand der Technik.....	10
3.2. Lösungsmethoden / Lösungsansatz	10
3.3. Grundlagen für das Gesamtprojekt.....	11
3.3.1. React	11
3.3.2. Leaflet	11
3.3.3. Chart.js	11
3.3.4. Node.js und Express.js	11
3.3.5. RACE RESULT 12 API.....	12
3.3.6. MySQL-Datenbank	12
3.4. Produktstrukturplan (PdSP)	12
3.5. Scrum-Projektplan (Epics und Stories)	13
4. Softwareentwicklung des Enduserinterfaces (Lukas Peter Sumann).....	14
4.1. Systemarchitektur und Design	15
4.1.1. Überblick über die Architektur der Oberfläche für Endnutzende	15
4.1.2. Verwendete Komponenten	20
4.1.3. Zustandshandling und Datenmanagement	21
4.2. Technologische Grundlage	22
4.2.1. Race Result AG	22
4.2.2. SimpleAPI	22
4.2.3. Live-Map mit Leaflet	23
4.3. Implementierung der Live-Map	24
4.3.1. Erstellen einer GPX-Karte mit Leaflet	24
4.3.2. Berechnung der Geschwindigkeit und Position der Teilnehmenden	30

4.3.3. ElevationChart.....	30
4.4. Implementierung der Ergebnistabelle	34
4.4.1. Interaktive Features der Ergebnistabelle.....	35
4.4.2. Berechnung der Platzierungen	38
5. Softwareentwicklung des Administratorinterfaces und Backend (Maximilian Wolf Leopold Lindner)	41
5.1. Systemarchitektur und Design	42
5.1.1. AdminHome.jsx.....	42
5.1.2. AdminLiveMap.jsx.....	43
5.1.3. AdminSponsors.jsx.....	44
5.1.4. AdminResults.jsx	45
5.1.5. EditText.jsx	46
5.1.6. AdminEdit.....	47
5.1.7. Fehlerbehandlung in der Anwendung	48
5.2. Komponenten.....	49
5.2.1. Frontend.....	50
5.2.2. Backend.....	50
5.3. Implementierung/Realisierung	52
5.3.1. Frontend.....	52
5.3.2. Backend.....	63
6. Anhang.....	76
6.1. Projektmanagement.....	76
6.1.1. Aufgabenstellung des Gesamtprojekts.....	76
6.1.2. Scrum-Projektplan	77
6.2. Inbetriebnahme.....	78
6.3. Besprechungsprotokolle	79
6.4. Arbeitszeitnachweis	83
6.5. Wettbewerbe	89
Literaturverzeichnis.....	90
Abbildungsverzeichnis.....	93
Tabellenverzeichnis	93
Listings.....	94

Die vorliegende Arbeit wurde hinsichtlich Grammatik, Rechtschreibung und Formulierung mittels KI-Modellen, allen Voran ChatGPT 4.0, überarbeitet. Die inhaltliche Erstellung dieser Arbeit ist jedoch eigenständig entstanden.

2. Einleitung

Die Wahl des Themas „Event Insight“ basiert auf der Notwendigkeit, eine Webapplikation zu entwickeln, die Renndaten von Sportveranstaltungen in Echtzeit visualisiert und die Positionen der Teilnehmenden live darstellt. Besonders bei großen Veranstaltungen mit vielen Teilnehmenden fehlt es oft an effizienten und skalierbaren Lösungen, um diese Daten für Zusehende und Veranstaltende übersichtlich und in Echtzeit darzustellen. Dieses Projekt zielt darauf ab, eine Lösung zu schaffen, die diese Lücke schließt und eine präzise Visualisierung der Renndaten ermöglicht.

2.1. Ausgangslage und Problemstellung

Die race result AG stellt eine API zur Verfügung, die Renndaten in Echtzeit erfasst und überträgt. Derzeit existieren jedoch nur wenige kostengünstige und skalierbare Lösungen, die diese Daten für Veranstaltungen mit vielen Teilnehmenden effektiv aufbereiten und anzeigen. Bisherige Lösungen sind häufig teuer und erfordern spezialisiertes Fachwissen. Ziel des Projekts ist es, eine kostengünstige, einfach anwendbare Lösung zu entwickeln, die es Veranstaltenden und Zusehenden ermöglicht, Renndaten in Echtzeit zu verwalten und darzustellen, ohne auf komplexe Systeme angewiesen zu sein. [1]

2.2. Aufgabenstellung und Zielsetzung

Die Aufgabenstellung dieses Projekts besteht darin, eine Webapplikation zu entwickeln, die Renndaten in Echtzeit darstellt. Über die SimpleAPI der race result AG werden die Rennzeiten und Positionen der Teilnehmenden abgerufen und in einer Webapplikation angezeigt. Die Anwendung soll es den Zusehenden ermöglichen, das Rennen live zu verfolgen, und den Veranstaltenden eine einfache Verwaltung der Renndaten bieten. Ein wichtiger Bestandteil der Zielsetzung ist die Entwicklung einer flexiblen Lösung, die auf verschiedenen Geräten zugänglich ist und es den Nutzern ermöglicht, das System problemlos zu verwenden. [1]

2.3. Fachliches und wirtschaftliches Umfeld

Das Projekt befindet sich im Kontext der Digitalisierung von Sportevents, wo Echtzeit-Datenübertragung und -visualisierung immer wichtiger werden. In wirtschaftlicher Hinsicht besteht eine hohe Nachfrage nach kostengünstigen, skalierbaren Lösungen für die Darstellung von Renndaten bei Veranstaltungen. Die Implementierung von Technologien wie React, Node.js und Docker ermöglicht es, die Applikation auf unterschiedlichen Geräten und Systemen bereitzustellen, was sowohl die Skalierbarkeit als auch die einfache Wartung und Bereitstellung des Systems sicherstellt.

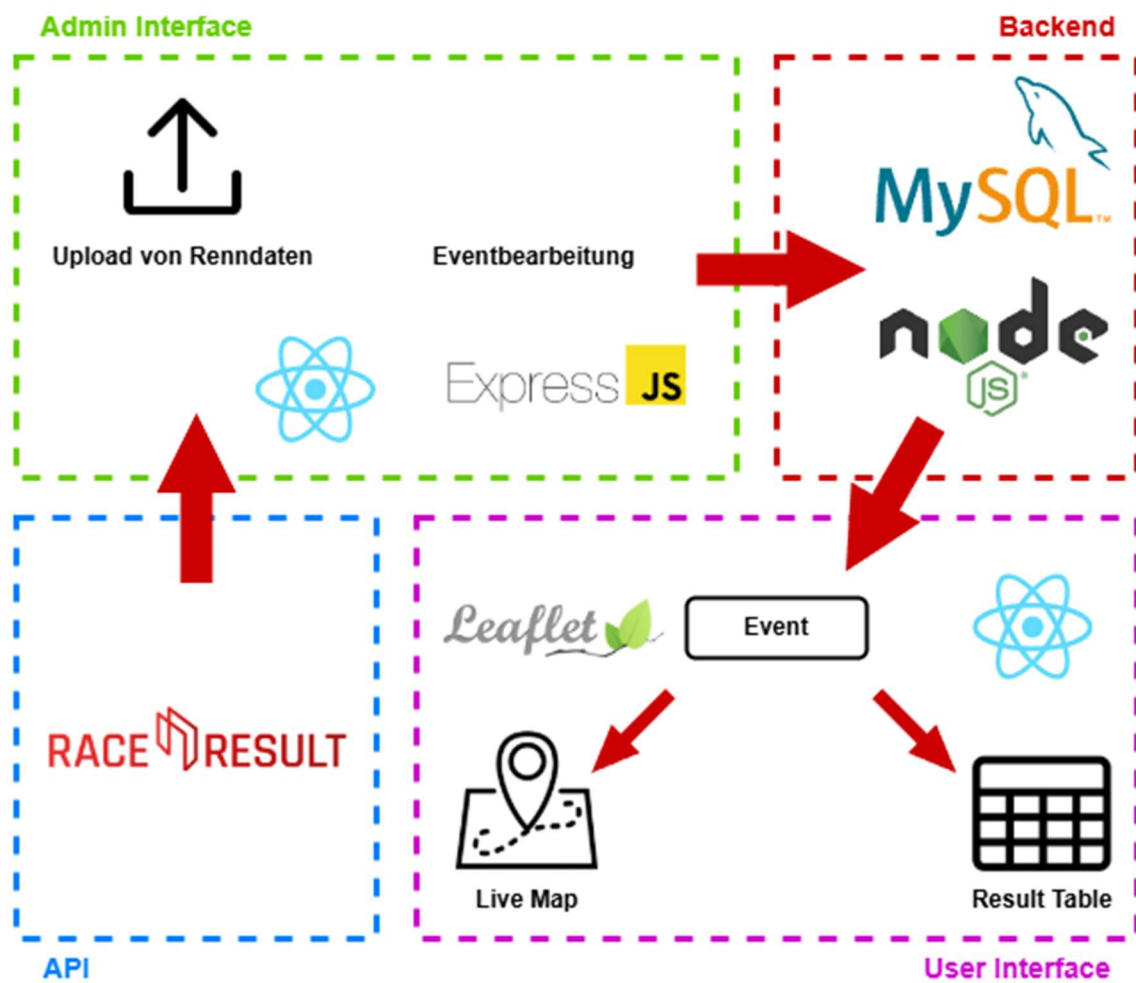


Abbildung 1.1: Systemstruktur

3. Grundlagen und Methoden

Dieses Kapitel behandelt die grundlegenden Technologien, Methoden und Prinzipien, die für das Projekt „Event Insight“ von Bedeutung sind. Es wird der aktuelle Stand der Technik erklärt und die gewählten Lösungsansätze werden begründet. Darüber hinaus werden die wesentlichen Technologien und Schnittstellen beschrieben, die im gesamten Projekt verwendet werden.

3.1. Ist-Situation / Stand der Technik

Im Bereich der Echtzeit-Datenvisualisierung von Sportveranstaltungen gibt es verschiedene Systeme, die Renndaten und die Positionen der Teilnehmenden in Echtzeit anzeigen. Jedoch sind diese Lösungen oft kostspielig, komplex und nicht immer skalierbar. Die meisten bestehenden Systeme bieten nur eingeschränkte Anpassungsmöglichkeiten und sind auf kleinere Veranstaltungen ausgelegt.

Ein gängiger Ansatz zur Bereitstellung von Echtzeit-Renndaten ist die Nutzung von APIs wie der RACE RESULT 12 API, die kontinuierlich Rennzeiten und Positionen liefert. Die Herausforderung liegt darin, diese Daten effizient zu verarbeiten und übersichtlich darzustellen, insbesondere bei größeren Veranstaltungen mit vielen Teilnehmenden.

Event Insight stellt eine skalierbare Lösung dar, die es ermöglicht, diese Renndaten in Echtzeit zu visualisieren und gleichzeitig eine benutzerfreundliche Oberfläche anzubieten. Ziel ist es, eine effiziente, kostengünstige und einfach zu bedienende Lösung zu entwickeln, die die Anzeige von Echtzeit-Renndaten für eine große Anzahl an Teilnehmenden ermöglicht. [1]

3.2. Lösungsmethoden / Lösungsansatz

Das Projekt „Event Insight“ verfolgt den Ansatz, eine Webapplikation zu entwickeln, die Renndaten in Echtzeit visualisiert und es Zusehenden ermöglicht, die Positionen der Teilnehmenden live zu verfolgen. Die Lösung basiert auf der SimpleAPI der race result AG, die die Rennzeiten und Positionen der Teilnehmenden in Echtzeit bereitstellt. Diese Daten werden über den Backend-Server verarbeitet, gegebenenfalls in der MySQL-Datenbank gespeichert und anschließend an das Frontend zur Darstellung übermittelt.

Für das Frontend wurde React gewählt, da es eine effiziente und skalierbare Lösung für die Erstellung interaktiver Benutzeroberflächen bietet. Mit React wird es ermöglicht, die Renndaten dynamisch darzustellen und eine interaktive Benutzererfahrung zu gewährleisten. Zudem kommt Leaflet zum Einsatz, um eine interaktive Karte zu integrieren, auf der die Zusehenden die Renndaten und Positionen der Teilnehmenden in Echtzeit verfolgen können.

Das Backend wird mit Node.js und Express.js entwickelt, was eine effiziente Verwaltung und Weiterverarbeitung der Renndaten ermöglicht. Der Server ist für die Kommunikation mit der RACE RESULT 12 API verantwortlich, um die aktuellen Renndaten abzurufen und an das Frontend weiterzugeben. Hierbei werden die Daten, wie die Positionen der Teilnehmenden,

Zeitstempel und Zwischenzeiten, in einer MySQL-Datenbank gespeichert. Diese Datenbank dient der langfristigen Speicherung und der schnellen Abrufbarkeit von Renndaten, die während und nach der Veranstaltung zur Analyse und Visualisierung benötigt werden. Die Datenbankstruktur umfasst Tabellen für Veranstaltungen, Teilnehmer, Renndaten sowie Zugangsdaten für administrierende Personen. [1] [2] [3] [4]

3.3. Grundlagen für das Gesamtprojekt

Die grundlegenden Technologien, die im Projekt verwendet werden, sind:

3.3.1. React

React ist ein JavaScript-Framework, das speziell für die Erstellung von interaktiven Benutzeroberflächen entwickelt wurde. Für das Projekt Event Insight wurde React aufgrund seiner Flexibilität, Effizienz und der Unterstützung für interaktive Benutzeroberflächen gewählt. React ermöglicht eine schnelle und dynamische Aktualisierung der UI, was für die Echtzeit-Datenanzeige auf der Live-Karte und die Verfolgung von Teilnehmenden erforderlich ist. Die komponentenbasierte Architektur erleichtert zudem die Wartung und Erweiterung der Anwendung, da jede Funktionalität in wiederverwendbare Module unterteilt werden kann. [2] [5]

3.3.2. Leaflet

Leaflet ist eine Open-Source-Bibliothek für interaktive Karten. Sie ermöglicht die Darstellung und Visualisierung geografischer Daten. Für Event Insight wurde *Leaflet* aufgrund seiner Effizienz bei der Kartenanzeige und -interaktion gewählt. Die Bibliothek ermöglicht die Integration von GPS-Daten und das Platzieren benutzerdefinierter Marker, wodurch die Positionen der Teilnehmenden auf der Live-Karte dargestellt werden können. Zudem lässt sich *Leaflet* flexibel anpassen und erweitern. [6] [7]

3.3.3. Chart.js

Chart.js ist eine Open-Source-Bibliothek zur Erstellung interaktiver Diagramme. Sie ermöglicht die Darstellung verschiedener Diagrammtypen und lässt sich einfach integrieren. Für das Höhenprofil-Diagramm in Event Insight wurde Chart.js gewählt, um das Höhenprofil der Strecke als Liniendiagramm darzustellen. Die Bibliothek unterstützt Verarbeitung großer Datenmengen und ermöglicht eine anpassbare Visualisierung der Höhenänderungen entlang der Strecke. [8] [9]

3.3.4. Node.js und Express.js

Node.js und Express.js wurden für Event Insight aufgrund ihrer Effizienz für Echtzeit-Anwendungen gewählt. Node.js ermöglicht die Ausführung von JavaScript auf dem Server und ist besonders für schnelle Datenverarbeitung und Echtzeit-Anwendungen geeignet. Express.js vereinfacht die Erstellung von *RESTful* APIs und die Verarbeitung von HTTP-Anfragen, was die Kommunikation zwischen *Frontend* und *Backend* erleichtert. Gemeinsam bieten die beiden Technologien eine leistungsfähige und skalierbare Lösung für die Anwendung. [3] [10] [4] [11]

3.3.5. RACE RESULT 12 API

RACE RESULT 12 ist eine Software zur Verwaltung und Auswertung von Sportveranstaltungen. Sie bietet Funktionen zur Zeitmessung, Ergebnisberechnung und Teilnehmerverwaltung. Die *SimpleAPI* von der race result AG ermöglicht den Zugriff auf Veranstaltungsdaten wie Teilnehmerinformationen und Zeiten. In Event Insight wird diese API genutzt, um Echtzeit-Teilnehmerdaten in die Webapplikation zu integrieren. Die Software wurde von der Sumann & Menner OG bereitgestellt. [1]

3.3.6. MySQL-Datenbank

MySQL ist ein weit verbreitetes, relationales Datenbankmanagementsystem, das zur Verwaltung strukturierter Daten in Tabellenform dient. Es zeichnet sich durch hohe Performance, Zuverlässigkeit und Skalierbarkeit aus.

Für Event Insight wurde MySQL gewählt, da es eine effiziente Speicherung und Verwaltung von Teilnehmerdaten, Ergebnissen und Veranstaltungsinformationen ermöglicht. Die relationale Struktur von MySQL eignet sich gut für die Verwaltung komplexer Datenbeziehungen, wie etwa die Verknüpfung von Teilnehmern mit deren Zeiten und Fortschritten. Darüber hinaus bietet MySQL eine zuverlässige Leistung bei großen Datenmengen und stellt so eine skalierbare Lösung für das Backend bereit. [12] [13]

3.4. Produktstrukturplan (PdSP)

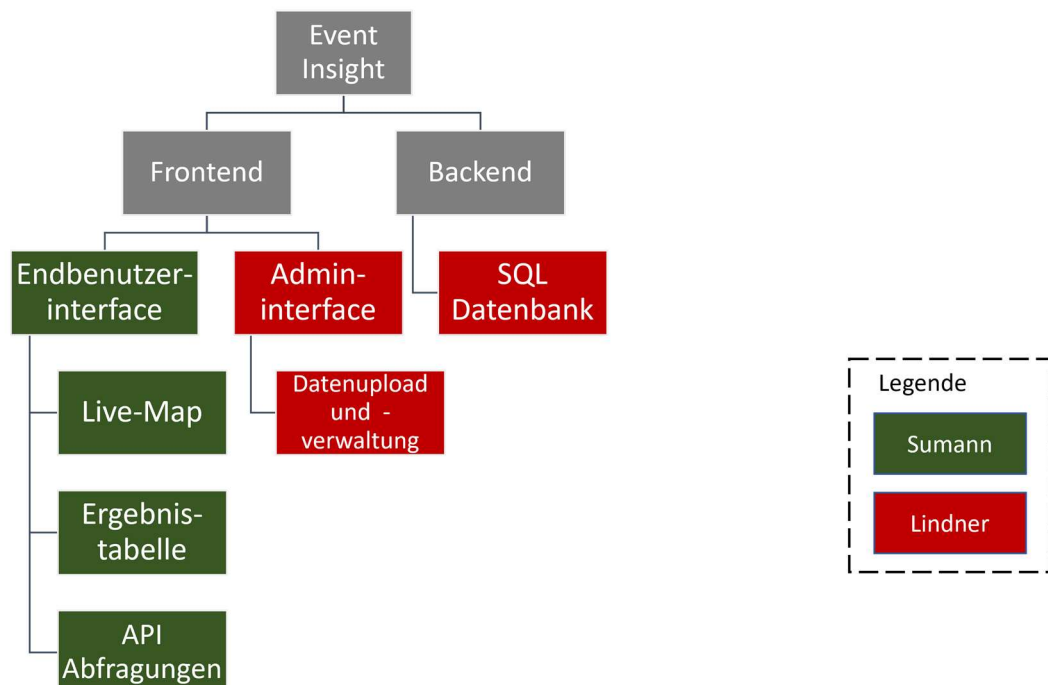


Abbildung 2: Produktstruktur

3.5. Scrum-Projektplan (Epics und Stories)

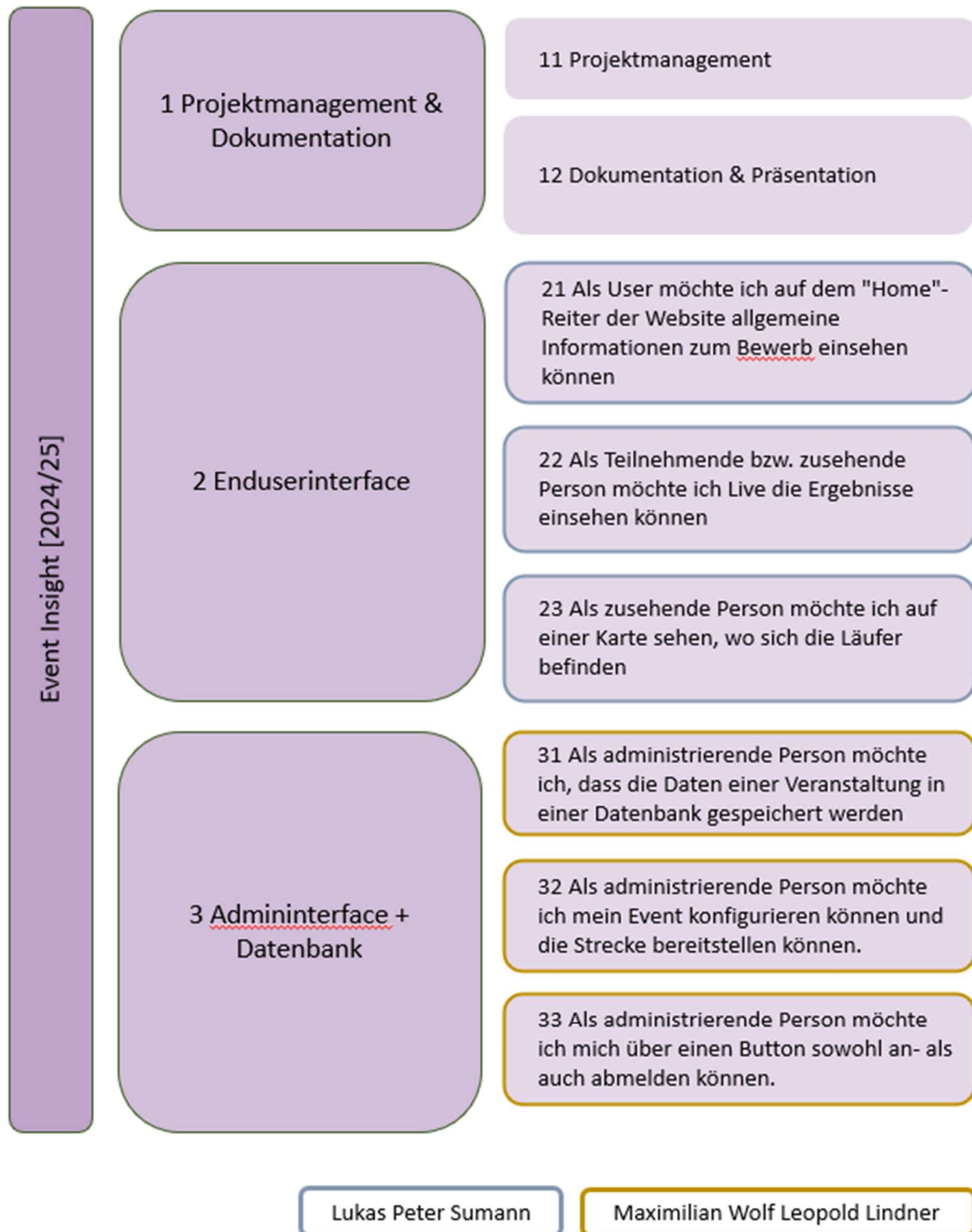


Abbildung 3: Scrum Projektplan

4. Softwareentwicklung des Enduserinterfaces (Lukas Peter Sumann)

Ziel ist es ein praktisches Interface für Endnutzende, wie z.B. teilnehmende oder zusehende Personen, zu erstellen, auf welchen mittels Suchformular nach der jeweiligen Sportveranstaltung gesucht werden kann. In der Benutzeroberfläche der Sportveranstaltung können Informationen über die Veranstaltung, die Strecke sowie Live- bzw. endgültige Ergebnisse eingesehen werden.

In der Oberfläche für endnutzende Personen der Veranstaltung kann sich eine nutzende Person über ein Anmeldeformular in die Oberfläche für administrierende Personen einloggen. Die administrierende Person hat anschließend die Möglichkeit, der Endnutzenden Person über Event Insight Informationen zu den jeweiligen Veranstaltungen bereitzustellen, beispielsweise über einen verspäteten Start.

Durch die Anbindung an die RACE RESULT 12 SimplyAPI können die Daten der Teilnehmenden Personen sowie deren Zwischen- und Endzeiten genutzt werden, um diese zu veröffentlichen und ihre ungefähre Position auf der Karte zu berechnen. Die Position der Teilnehmenden wird aus den zuletzt erfassten Zeitpunkten und der bisherigen Durchschnittsgeschwindigkeit geschätzt. Auf diese Weise lässt sich der ungefähre Aufenthaltsort in Echtzeit auf einer digitalen Karte anzeigen. Zusehende Personen können so den Verlauf des Events nachvollziehbar verfolgen. Das Interface ermöglicht es zudem, den verantwortlichen Personen, wichtige Event-Informationen wie kurzfristige Hinweise zu Verspätungen oder organisatorische Änderungen an die Nutzenden weiterzugeben. Dies verbessert die Kommunikation zwischen den Veranstaltenden und den Teilnehmenden erheblich. [1]

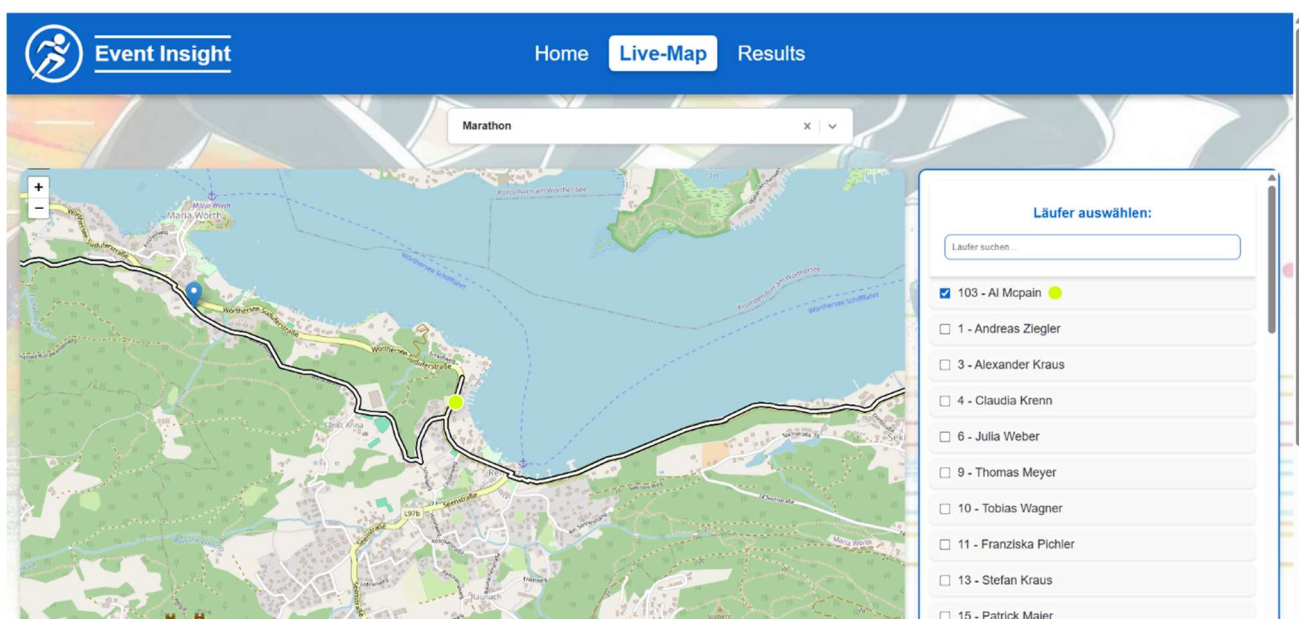


Abbildung 4: Live-Map

4.1. Systemarchitektur und Design

Die Systemarchitektur von Event Insight basiert auf einer modularen, komponentenbasierten Struktur, die durch den Einsatz von React ermöglicht wird. Diese Architektur stellt sicher, dass die Anwendung sowohl skalierbar als auch flexibel ist. Zudem sorgt sie dafür, dass die Oberfläche schnell und benutzerfreundlich bleibt, auch bei der Anzeige von Echtzeitdaten und bei der Interaktion mit komplexen, dynamischen Elementen wie Live-Maps, Teilnehmermarkierungen und Ergebnistabellen.

Der Designansatz verfolgt das Ziel, eine klare Trennung zwischen den verschiedenen Funktionseinheiten der Anwendung zu gewährleisten. Die Architektur wird dabei durch React-Komponenten definiert.

Die grundlegenden Anforderungen der Anwendung umfassen die Echtzeitverarbeitung von Event-Daten, eine klare Visualisierung der Teilnehmenden und Fortschritte, sowie eine benutzerfreundliche Interaktion für die Zusehenden und administrierende Personen. Für die Realisierung dieser Anforderungen wurde eine Architektur entwickelt, die die Aktualisierung und Darstellung von Echtzeitdaten nahtlos integriert, um eine flüssige und informative Benutzererfahrung zu gewährleisten. [14]

4.1.1. Überblick über die Architektur der Oberfläche für Endnutzende

Das Enduserinterface von Event Insight folgt einer modularen Architektur, die eine hohe Wartbarkeit und Erweiterbarkeit gewährleistet. Die Kommunikation zwischen den Komponenten erfolgt primär über *properties*, welche zwischen den Komponenten mitgegeben werden, sowie über den lokalen Zustand (*useState*), der in React für jede Komponente individuell verwaltet wird.

Ein zentraler Bestandteil dieser Struktur ist der gezielte Einsatz von benutzerdefinierten React-Hooks, die für die Datenabfrage und -aktualisierung verantwortlich sind. Ein Beispiel dafür ist der Hook *useAPIData*, der Echtzeitdaten von API-Endpunkten abrufen und in den jeweiligen Komponentenstatus überträgt. Diese Trennung von Logik und Darstellung fördert eine klare Modularisierung und erleichtert sowohl die Wartung als auch zukünftige Erweiterungen der Anwendung.

Zur Navigation innerhalb der Anwendung wird React Router eingesetzt. Durch den Einsatz von *useParams* können spezifische Veranstaltungsdaten geladen und dargestellt werden, wodurch eine dynamische und effiziente Navigation ermöglicht wird. Die Oberfläche für endnutzende Personen umfasst im Wesentlichen vier Webseiten: [2]

4.1.1.1. Landing Page

Die Landing Page stellt die zentrale Anlaufstelle der Anwendung dar und wird standardmäßig beim Aufruf geladen. Auf dieser Seite besteht die Möglichkeit, nach Veranstaltungen zu suchen, um die spezifische Oberfläche einer Veranstaltung aufzurufen. Zusätzlich bietet die

Startseite einen klaren und gut sichtbaren Button, über welchen man zum Anmeldeformular für die Oberfläche für administrierende Personen gelangt.

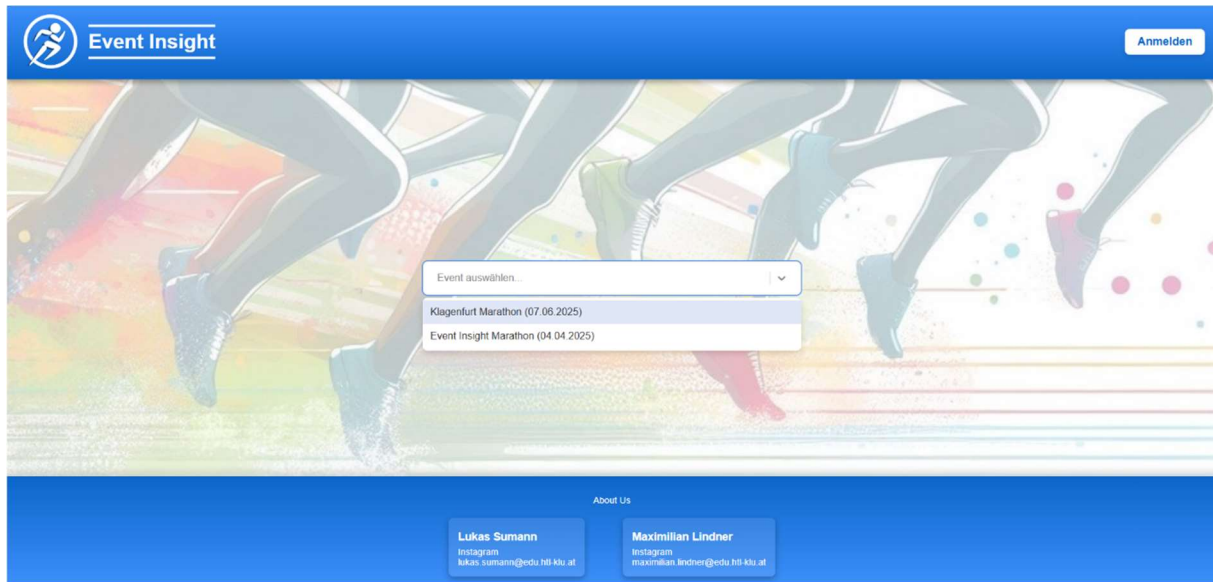


Abbildung 5: Landing Page

4.1.1.2. UserHome

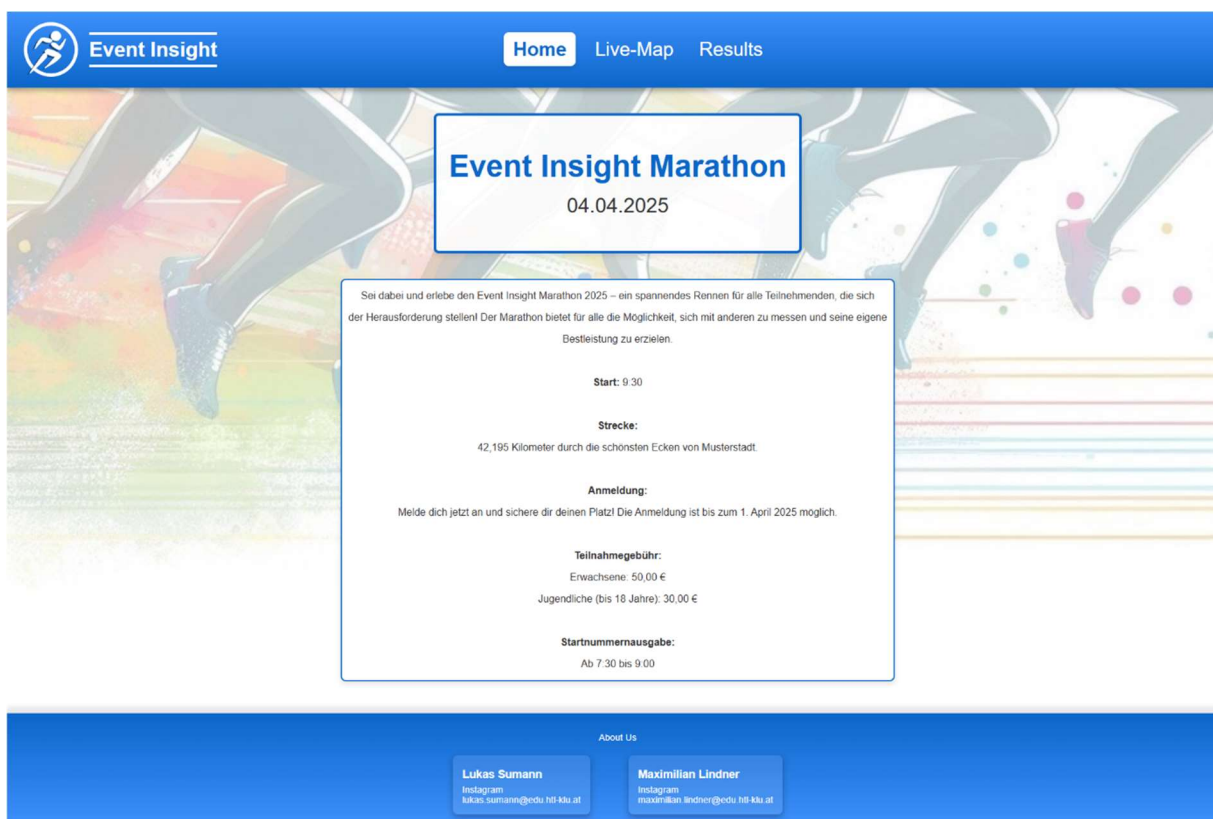


Abbildung 6: UserHome

Diese Seite bietet eine zentrale Übersicht der Veranstaltung und stellt grundlegende Veranstaltungsdaten dar, wie den Namen der Veranstaltung, das Datum sowie die von der administrierenden Person veröffentlichte Sponsorenwerbung. Endnutzende Personen erhalten hier Zugriff auf die wichtigsten Informationen sowie auf die Navigationslinks zu den weiteren Seiten, darunter die Live-Karte und die Ergebnisse.

4.1.1.3. UserLiveMap

Die *UserLiveMap*-Seite bildet das Herzstück der Anwendung, da sie die Live-Positionen der teilnehmenden Personen auf einer interaktiven Karte darstellt. Zu Beginn wählen die Nutzenden den gewünschten Bewerb aus, da Veranstaltungen oft mehrere Bewerbe beinhalten, wie beispielsweise einen Halbmarathon zusätzlich zum Marathon.

Für die Kartendarstellung wird die Open-Source-Library *Leaflet* verwendet, die eine effiziente Visualisierung der Positionsdaten in Echtzeit ermöglicht. Die Bewegungen der teilnehmenden Personen können in Echtzeit verfolgt werden, während Start-, Ziel- und Zwischenzeitmarkierungen auf der Karte für eine bessere Orientierung sorgen.

Zusätzlich steht eine Such- und Filterfunktion zur Verfügung: Mithilfe einer Checkbox können spezifische teilnehmende Personen ausgewählt werden, sodass nur deren Positionen auf der Karte sichtbar sind. Um eine klare Unterscheidung zu gewährleisten, erhält jede markierte Person eine einzigartige Farbe für ihren Marker.

Unterhalb der Karte befindet sich eine Grafik, die das Höhenprofil der Strecke visualisiert. Auch hier werden die Positionen der teilnehmenden Personen angezeigt, sodass ersichtlich ist, an welcher Stelle der Strecke sich jede ausgewählte Person gerade befindet. [6]

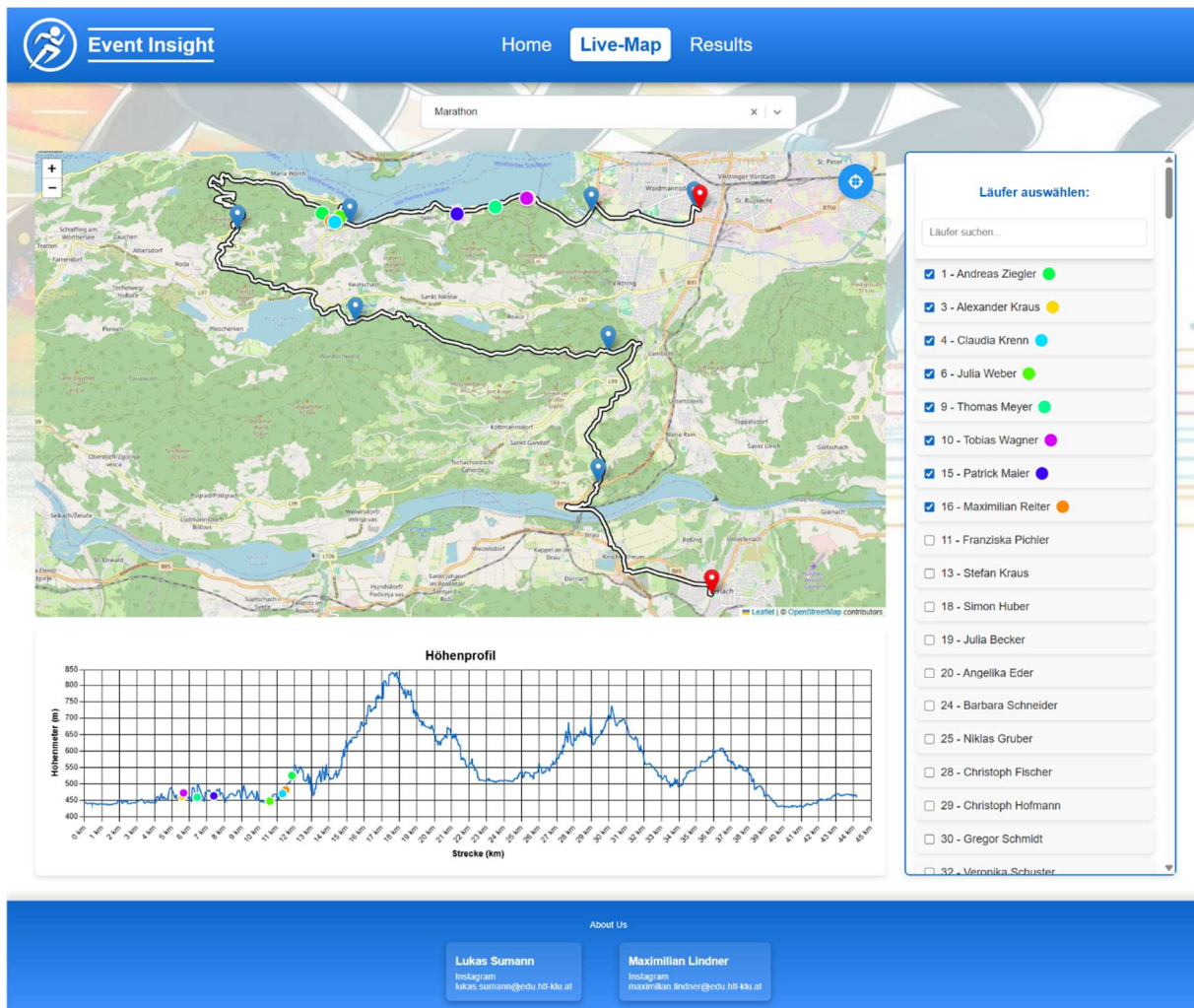


Abbildung 7: UserLiveMap

4.1.1.4. UserResults

Diese Seite stellt eine Tabelle mit den Ergebnissen der teilnehmenden Personen dar. Sie zeigt den Fortschritt der Teilnehmenden an und ermöglicht eine detaillierte Ansicht ihrer Platzierungen und Zeiten. Mithilfe von einer Suchleiste können z.B. Zusehender gezielt nach bestimmten Personen suchen.

Die Tabelle bietet außerdem die Möglichkeit, nach allen Spalten sowohl auf- als auch absteigend zu sortieren. Um die Übersichtlichkeit zu gewährleisten, werden pro Tabelle maximal 30 Teilnehmende angezeigt, wobei mit einem Klick auf „Alle Anzeigen“, alle Teilnehmende in der Tabelle visualisiert werden können.

Auch hier lässt sich der Bewerb auswählen, da es bei einem Event mehrere Bewerbe geben kann. Dabei gibt es zwei Ansichtsmodi: eine nach Geschlecht gruppierte Liste (MW) sowie eine nach Altersklassen sortierte Liste (AK), zwischen denen per Button umgeschaltet werden kann.

MW PL	STARTNR.	VORNAME	NACHNAME	NAT.	JG.	AK.	VEREIN	ZW1	ZW2	ZW3	ZW4	ZW5	ZW6	ZW7	ZEIT
1	1452	Sven	Andersson	SWE	1972	U50	L_f_c_ub Stockholm	01:10	30:49	58:32	1:25:31				
2	1157	Rafael	Go_zález	ESP	1953	U70	_leichtathle_t_Club V_len_t_	01:02	34:02	1:03:19	1:30:26				
3	1088	Charles	Davies	GBR	2000	Ju_ioren	Sport_erein Sheffield	01:05	28:33	1:00:06	1:30:42				
4	1129	Anders	Johan_son	SWE	1956	U60	_hletikve_ein Malmö	01:09	30:00	59:26	1:33:22				
5	13	Stefan	Kraus	AUT	1965	U50	Leichtathletik Club Klagenfurt	01:03	29:51	1:01:25					
6	55	Tobias	Maier	AUT	1983	U40	Road Runners Niederösterreich	01:12	31:47	1:02:37					
7	81	Sebastian	Götz	AUT	1981	U40	Sprint Club Steiermark	01:07	35:48	1:03:27					
8	35	Michael	Dietrich	AUT	1958	U60	Leichtathletik Club Klagenfurt	00:56	34:08	1:04:06					
9	1204	Olof	Nilss_n	SWE	2000	Junioren	_aufclub Helsi_gborg	01:00	30:58	1:05:13					

Abbildung 8: UserResults

In Abbildung 8 ist ersichtlich, dass einige Einträge in der Tabelle mit Unterstrichen versehen sind. Dies liegt daran, dass für die Darstellung der Daten die SimpleAPI eines Testevents der RACE RESULT 12 Software genutzt wurde. Die Verwendung eines Testevents war notwendig, um Gebühren zu vermeiden, die ansonsten für jede teilnehmende Person anfallen würden. Bei Testevents werden die Daten der Teilnehmenden automatisch durch willkürlich gesetzte Unterstriche unkenntlich gemacht. [1]

ABRECHNUNGSMODUS

i Neue Teilnehmer werden nachts von Ihrem Guthaben abgebucht, es sei denn Sie definieren die Veranstaltung als Test-Veranstaltung.
Während Sie offline arbeiten, ist keine Freischaltung notwendig.

☐ **Echte Veranstaltung**
Teilnehmer nachts zwischen 2 und 6 Uhr abrechnen. Wenn Ihr Teilnehmer-Guthaben aufgebraucht ist, werden die Teilnehmer auf Listen durch Unterstriche unkenntlich gemacht.

☒ **Test-Veranstaltung**
Dies ist eine Test-Veranstaltung, Teilnehmer nicht abrechnen. Die Teilnehmer werden auf Listen durch Unterstriche unkenntlich gemacht.

Abbildung 9: RACE RESULT 12 Abrechnungsmodus

4.1.2. Verwendete Komponenten

Im vorherigen Kapitel wurden die zentralen Seiten der Event Insight-Anwendung beschrieben, die das Enduserinterface für Zusehende und administrierende Personen bilden. Diese Seiten setzen sich aus verschiedenen React-Komponenten zusammen, die jeweils spezifische Funktionen und Interaktionen bereitstellen. In diesem Kapitel werden die wichtigsten Komponenten erläutert, die zur Umsetzung der zuvor beschriebenen Seiten beitragen.

4.1.2.1. Komponenten der Live-Map

Die Live-Map umfasst mehrere Funktionen, die jeweils als eigene spezifische Komponenten umgesetzt sind und miteinander interagieren.

- **ParticipantCheckboxes**

Mit dieser Filterfunktion können Zusehende auswählen, welche Teilnehmer auf der Karte angezeigt werden sollen. Besonders bei großen Events mit vielen Teilnehmern hilft das, den Überblick zu behalten und sich auf bestimmte Personen zu konzentrieren. Mit der Tastenkombination [STRG] + [a] werden alle Teilnehmenden ausgewählt und mit einem erneuten Drücken der Tastenkombination werden wieder alle abgewählt. Teilnehmende welche in der Checkbox ausgewählt werden, werden sowohl auf der Live-Map als auch im Diagramm des Höhenprofils mit einem farbigen Punkt visualisiert. Dieser farbige Punkt befindet sich ebenfalls im Eintrag der jeweiligen Teilnehmenden Person in der Checkbox, sollte man diese ausgewählt haben.

- **GPXMap**

Diese Komponente bildet die Karte, indem sie die dazugehörige GPX-Datei aus dem Backend bekommt. Die GPX-Datei besteht aus einer Liste von Koordinaten, welche aus Längen- und Breitengraden, sowie meist noch der Seehöhe bestehen. Die Strecke wird erstellt, indem alle Koordinaten verbunden werden. In der *GPXMap*-Komponente werden noch andere Komponenten verwendet:

- **StartGoalMarkers**

Zur Visualisierung von Start und Ziel werden am ersten und am letzten Punkt der GPX-Datei ein roter Marker gesetzt. Sollte Start und Ziel am selben Punkt sein, dann wird nur ein Marker für Start/Ziel gesetzt.

- **IntermediateTimesMarker**

Die IntermediateTimesMarker sind Marker für die Zwischenmesspunkte, welche im Interface für administrierende Personen gesetzt werden können. Diese Zwischenmesspunkte sind im weiteren Verlauf wichtig für die Berechnung der Positionen der Teilnehmenden.

- **ParticipantMarker**

Die Teilnehmer, welche per API eingelesen werden, werden auf der Karte durch einen Marker dargestellt. Diese Marker bieten eine visuelle Darstellung der Positionen der Teilnehmenden. Bei einem Klick auf den Marker öffnet sich ein Popup, das detaillierte Informationen zu jener Person enthält, wie Name, Verein,

Jahrgang, Durchschnittsgeschwindigkeit, erwartete Zielzeit und einen Link zur *UserResult*-Seite, wobei nur der Eintrag jenes Teilnehmenden, von welchem man das Popup geöffnet hat, in der Tabelle angezeigt wird. Jeder Marker hat eine einzigartige Farbe, sodass man die einzelnen Teilnehmer voneinander unterscheiden kann.

- **ElevationChart**

Unterhalb der Karte wird das Höhenprofil der Strecke dargestellt. Der Graph entsteht, indem die Informationen über die Seehöhe aus der GPX-Datei extrahiert werden und damit ein Diagramm erstellt wird.

4.1.2.2. Komponenten der Ergebnistabelle

Die *UserResults*-Seite basiert hauptsächlich auf der *ResultTable*-Komponente, die eine kompakte und übersichtliche Darstellung aller relevanten Rennergebnisse bietet. Die Komponente errechnet die Platzierungen und visualisiert dann diese und andere Daten wie z.B. Namen, Nation und Zeiten der Teilnehmenden in einer Tabelle. Die Tabelle ermöglicht es, die Ergebnisse nach verschiedenen Kriterien wie Altersklassen oder Geschlecht zu sortieren.

Zusehende haben die Möglichkeit, innerhalb der Tabelle gezielt nach Teilnehmenden zu suchen und die Spalten nach Belieben auf- oder absteigend zu sortieren. [15]

4.1.3. Zustandshandling und Datenmanagement

Das Zustandshandling und die Verwaltung dynamischer Daten spielen eine zentrale Rolle im Design von Event Insight. Der Zustand wird über den *useState*-Hook gesteuert, wodurch jede Komponente ihren eigenen Zustand speichern und aktualisieren kann. Eine der größten Herausforderungen bestand darin, die Echtzeitdaten der Teilnehmenden sowohl auf der Karte als auch in der Ergebnisübersicht ohne Verzögerung und stets synchron darzustellen.

4.1.3.1. Verwendung von *useState* und *useEffect*

Die Zustandsverwaltung der Anwendung basiert auf dem *useState*-Hook, der essenzielle Daten wie Teilnehmerinformationen, GPX-Daten und Fortschrittsberechnungen speichert. Um sicherzustellen, dass diese Daten stets aktuell sind, kommt *useEffect* zum Einsatz. Dadurch werden Änderungen automatisch erkannt und der Zustand entsprechend aktualisiert.

Besonders wichtig ist das Echtzeit-Zustandshandling für die Live-Map. Die Positionen der Teilnehmenden werden regelmäßig über API-Aufrufe aktualisiert, um den aktuellen Fortschritt anzuzeigen. Damit dies ohne spürbare Verzögerungen geschieht und gleichzeitig die Performance erhalten bleibt, wird der Zustand der Anwendung alle fünf Sekunden aktualisiert. Dies geschieht mithilfe eines *setInterval*-Mechanismus, der kontinuierlich die API abfragt und sowohl die Karte als auch die Ergebnistabelle mit den neuesten Daten versorgt.

4.1.3.2. Dynamische Daten und Performance

Eine der größten Herausforderungen beim Zustandshandling war die Verwaltung von dynamischen Daten, die kontinuierlich aktualisiert werden müssen. Besonders anspruchsvoll

gestaltete sich die Handhabung der Teilnehmermarkierungen auf der Live-Map sowie die Berechnung und Anzeige der Zeiten in der Ergebnisübersicht. Um eine flüssige Darstellung zu gewährleisten, wurde das System so optimiert, dass es auch bei einer großen Anzahl von Teilnehmenden oder bei langen Rennen zuverlässig und performant bleibt.

4.2. Technologische Grundlage

Um Zugriff zu Renndaten, wie Vor- und Nachname, Zeiten, etc. zu erhalten und diese anschließend weiterzuverarbeiten, unter anderem zur Anzeige von Livepositionen auf einer Karte, werden verschiedene Technologien eingesetzt. Zum einen wird eine Software-Schnittstelle zur Zeitmessungstechnologie der jeweiligen Sportveranstaltung benötigt. Uns wird sämtliches Zeitmessungsequipment sowie der Zugriff zur Race Result 12 Software der race result AG von race result Austria, insbesondere der Sumann & Menner OG zur Verfügung gestellt. [1]

4.2.1. Race Result AG

Die Race Result AG bietet umfassende Lösungen für die Zeitmessung und Datenverarbeitung bei Sportveranstaltungen. Die Software RACE RESULT 12 ermöglicht eine effiziente Verwaltung von Online-Anmeldungen, Teilnehmermanagement und Ergebnisverarbeitung. Für den Offline-Betrieb während Veranstaltungen kann die Software heruntergeladen werden.

Im Hardware-Bereich stellt die race result AG verschiedene Systeme bereit, darunter das *Ubidium*-System, Transponder, Decoder und Track Boxes, die eine präzise und zuverlässige Zeitmessung gewährleisten. Mit ihren Software- und Hardwarelösungen im Bereich der Zeitmessung entwickelte sich die race result AG in den letzten Jahren bzw. Jahrzehnten zu einem der weltweit führenden Unternehmen. [1]

4.2.2. SimpleAPI

Zudem stellt die race result AG in ihrer RACE RESULT 12 Software eine Programmierschnittstelle, welche SimpleAPI genannt wird, bereit, die es erlaubt spezifische Daten der Veranstaltung weiterzuverarbeiten. In der RACE RESULT 12 Software wird unter folgenden API-Typen unterschieden, welche es ermöglichen:

- eine Veröffentlichung von Listen der Teilnehmenden, Ergebnislisten oder Live-Ergebnislisten in den Formaten TXT, JSON, JSON ext, HTML, PDF, XML, XLSX oder CSV.
- dem Nutzenden Ergebnis- oder Teilnehmende Statistiken im PDF oder XLSX-Format zu veröffentlichen.
- die von der benutzenden Person konfigurierten Einstellungen des Bewerbs zu veröffentlichen.
- einen Counter bzw. Zähler der Teilnehmenden Personen zu veröffentlichen.

- Benutzerdefinierte Daten der Veranstaltung in einem beliebigen Format weiterzuverarbeiten. [1]

Aktiv	Typ	Details	Bezeichnung	Link
☒	Liste	Ergebnisliste	JSON	
☒				

Abbildung 10: SimpleAPI Eingabefenster in RACE RESULT 12

Für die Implementierung der Echtzeit-Ergebnisse in Event Insight wird von der SimpleAPI der RACE RESULT 12 Software ein API-Link des Typs „Liste“ im JSON Datenformat benötigt. Hierbei sollte per API eine Liste übertragen werden, welche sowohl Zwischenzeiten und endgültige Zeiten der Teilnehmenden beinhaltet. Die API ermöglicht es, diese Liste regelmäßig abzurufen und die darin enthaltenen Daten kontinuierlich zu aktualisieren. Durch die regelmäßige Synchronisation mit der SimpleAPI wird die Aktualität der Live-Ergebnisse gewährleistet, da die angezeigten Daten stets den aktuellen Stand der Veranstaltung widerspiegeln. Die Schnittstelle zur API bietet dabei eine effiziente Möglichkeit, die Ergebnisdaten fortlaufend zu erhalten. [1]

4.2.3. Live-Map mit Leaflet

Um die berechneten Positionen der teilnehmenden Personen auf einer digitalen Karte übersichtlich darzustellen, wird das Open-Source JavaScript-Framework *Leaflet* eingesetzt. *Leaflet* zeichnet sich insbesondere dadurch aus, dass es flexibel und zugleich sehr ressourcenschonend arbeitet und somit ideal geeignet ist, um Echtzeit-Positionierungen auch auf mobilen Geräten flüssig darzustellen. Aufgrund der geringen Dateigröße und schnellen Ladezeiten ist *Leaflet* bestens für die Nutzung auf unterschiedlichen Endgeräten optimiert, was speziell bei Sportveranstaltungen von großem Vorteil ist.

Neben der Tatsache, dass *Leaflet* eine Open-Source-Library ist, war ein entscheidendes Argument für die Wahl von *Leaflet* als technologische Grundlage der Live-Map zudem die einfache Integration und Anpassbarkeit innerhalb der Event Insight-Oberfläche. Durch die modular aufgebaute Struktur lassen sich individuelle Funktionen wie das Hervorheben bestimmter teilnehmender Personen oder Marker auf der Karte für Start, Ziel oder Zwischenzeiten unkompliziert integrieren. Zusätzlich unterstützt *Leaflet* zahlreiche Kartenanbieter wie OpenStreetMap oder *Mapbox*, wodurch veranstaltende Personen eine freie Auswahl und einfache Anpassung des Kartenmaterials ermöglicht wird.

Im Kontext von Event Insight fügt sich *Leaflet* daher optimal ein, da es sich problemlos in das bestehende System integrieren lässt, gut skalierbar ist und gleichzeitig eine zuverlässige, schnelle und übersichtliche Darstellung der Renndaten auf einer interaktiven Karte

gewährleistet. Im direkten Vergleich zu schwergewichtigeren Alternativen bietet Leaflet damit eine klare technologische Überlegenheit, insbesondere in Bezug auf Ladezeiten, Benutzerfreundlichkeit und Kompatibilität mit verschiedenen Browsern und Betriebssystemen. [6] [16]

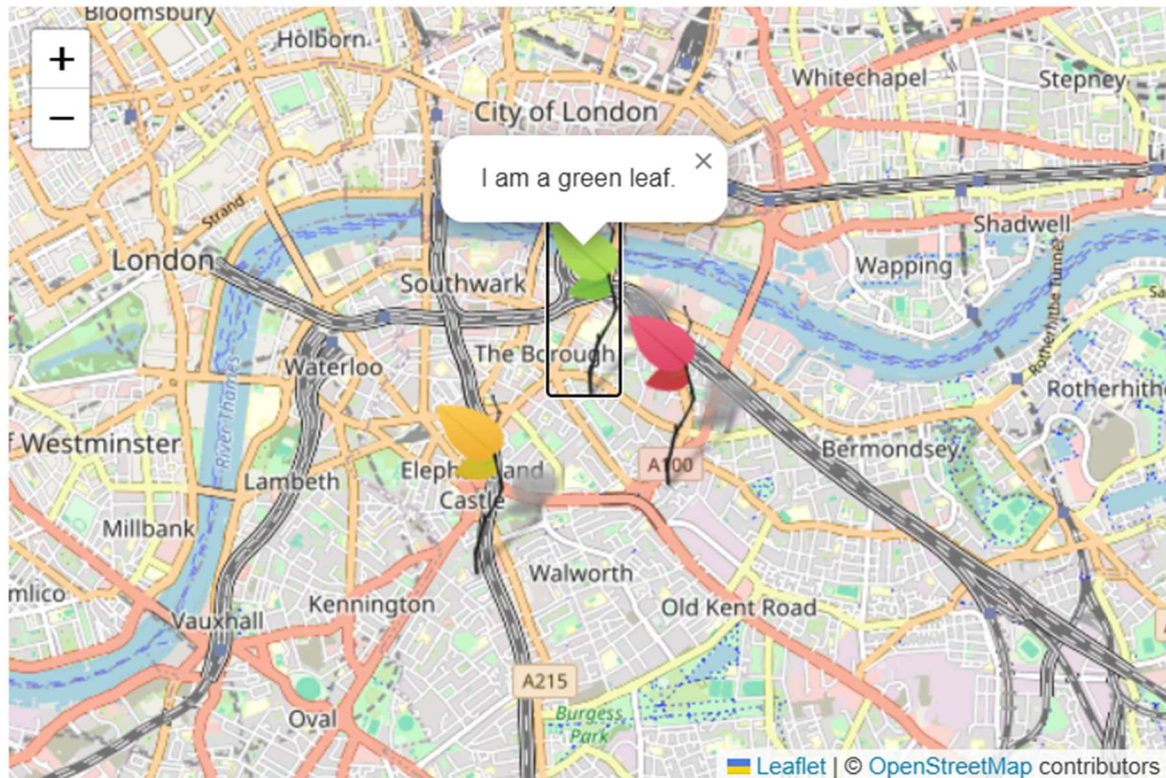


Abbildung 11: Beispiel für Leaflet Marker (Leafletjs.com)

4.3. Implementierung der Live-Map

Die Implementierung der Live-Map stellt das zentrale Element der Webapplikation dar, da sie den Zusehenden und Administrierenden eine dynamische, interaktive Möglichkeit bietet, den Verlauf der Veranstaltung in Echtzeit zu verfolgen. Die Live-Map stellt die Position der Teilnehmenden, die Streckenverläufe und wichtige Event-Daten dar. In diesem Abschnitt wird detailliert erläutert, wie die Live-Map in der Anwendung implementiert wurde, welche Technologien verwendet wurden und wie die einzelnen Funktionalitäten realisiert sind.

4.3.1. Erstellen einer GPX-Karte mit Leaflet

Die Live-Map wird mit Hilfe des Open-Source-JavaScript-Frameworks *Leaflet* realisiert, das eine effiziente und ressourcenschonende Darstellung von interaktiven Karten ermöglicht. *Leaflet* ist besonders für Web-Anwendungen geeignet, da es eine geringe Dateigröße hat und auf mobilen Geräten gut funktioniert, was es ideal für die Darstellung von Echtzeit-Positionsdaten macht. Die Karteninhalte werden durch GPX-Daten ergänzt, die als Grundlage für die Darstellung des Streckenverlaufs dienen. [6]

4.3.1.1. Laden und Parsen von GPX-Dateien

GPX (GPS Exchange Format) ist ein XML-basiertes Dateiformat, das GPS-Daten wie Wegpunkte, Routen und Tracks speichert. Um diese Daten in die Live-Map zu integrieren, wird die GPX-Datei zunächst geladen und geparkt, um die relevanten Informationen zu extrahieren. [17]

```
<trkpt lat="46.61014920301949" lon="14.304441097871255">
  <ele>444</ele>
</trkpt>
<trkpt lat="46.610149" lon="14.304439">
  <ele>444</ele>
</trkpt>
<trkpt lat="46.610727" lon="14.304433">
  <ele>442</ele>
</trkpt>
<trkpt lat="46.610727" lon="14.304433">
  <ele>442</ele>
</trkpt>
<trkpt lat="46.610775" lon="14.30498">
  <ele>441</ele>
</trkpt>
<trkpt lat="46.610861" lon="14.30591">
  <ele>439</ele>
</trkpt>
```

Listing 1: Beispiel für eine GPX-Datei

Wie bereits erwähnt, besteht eine GPX-Datei aus einzelnen sogenannten „Trackpunkten“. Diese Punkte setzen sich aus Längen- und Breitengraden (lat, lon) zusammen, wodurch geographische Koordinaten gebildet werden. Zusätzlich kann zu jedem *Trackpunkt* die Seehöhe (ele) gespeichert sein, welche später zur Erstellung des Höhenprofilendiagramms genutzt wird. [17]

Zu Beginn wird die GPX-Datei asynchron über den angegebenen URL-Pfad abgerufen. Hierzu wird die *fetch*-Methode verwendet, um die Datei entweder von einem Server oder lokalem Speicher zu laden. Die *fetch*-Methode liefert die Datei als Text zurück, der in der Funktion *loadGPXFile* in die Variable *gpxText* gespeichert wird. Falls beim Laden der Datei ein Fehler auftritt, wird dieser abgefangen und eine Fehlermeldung ausgegeben.

```
async function loadGPXFile(url) {
  try {
    const response = await fetch(url);
    const gpxText = await response.text();
    return gpxText;
  } catch (error) {
    console.error("Fehler beim Laden der GPX-Datei:", error);
    return null;
  }
}
```

Listing 2: GPX-Data fetching

Nachdem die Datei erfolgreich geladen wurde, wird sie in einen DOM-Parser überführt, um die XML-Daten zu extrahieren. Hierbei werden die *trkpt*-Tags (Track Points) ausgelesen, die die GPS-Daten für jeden Punkt der Strecke enthalten (Latitude, Longitude). Die Route wird dann als Array von Koordinaten gespeichert, das in Leaflet für die Darstellung verwendet wird.

```
function parseGPXtoTrackData(gpxText) {
  const parser = new DOMParser();
  const xmlDoc = parser.parseFromString(gpxText, "text/xml");
  const trackPoints = xmlDoc.getElementsByTagName("trkpt");

  const trackData = [];
  for (let i = 0; i < trackPoints.length; i++) {
    const lat = parseFloat(trackPoints[i].getAttribute("lat"));
    const lon = parseFloat(trackPoints[i].getAttribute("lon"));
    if (!isNaN(lat) && !isNaN(lon)) {
      trackData.push([lat, lon]);
    }
  }
  return trackData;
}
```

Listing 3: Parsing der GPXDatei

Dieser Codeabschnitt parst die GPX-Daten und speichert die GPS-Koordinaten als Array von Arrays, das dann für die Darstellung der Route auf der Karte verwendet wird. [18]

4.3.1.2. Berechnung von Kartenmitte

Nachdem die GPX-Daten *gparsed* wurden, wird die Route analysiert, um die geografische Mitte (Zentrum) der Strecke zu berechnen. Hierbei wird die minimale und maximale Breiten- und Längengradkoordinate der Strecke ermittelt. Diese Koordinaten werden dann genutzt, um den Mittelpunkt der Karte zu berechnen. Diese Werte werden gebraucht, um die Start-Position der Live-Map zu ermitteln. Eine separate Funktion *calculateCenter* berechnet diese Werte basierend auf den Routenpunkten.

```
function calculateCenter(route) {
  let minLat = 90, maxLat = -90, minLon = 180, maxLon = -180;
  route.forEach(([lat, lon]) => {
    if (lat < minLat) minLat = lat;
    if (lat > maxLat) maxLat = lat;
    if (lon < minLon) minLon = lon;
    if (lon > maxLon) maxLon = lon;
  });
  const center = [(minLat + maxLat) / 2, (minLon + maxLon) / 2];
  return {center};
}
```

Listing 4: calculateCenter-Methode

Die Berechnung des Zentrums erfolgt, indem die minimalen und maximalen Breiten- und Längengradwerte der Route mittels einer Schleife ermittelt werden. Dann werden jeweils die Längen und Breitengrade ermittelt, welche sich in der Mitte der minimalen und maximalen Werte befinden und daraus ergeben sich die Koordinaten des Zentrums (centerLat, centerLon).

$$centerLat = \frac{minLat + maxLat}{2}, centerLon = \frac{minLon + maxLon}{2}$$

4.3.1.3. Karte mit Leaflet erstellen

Die Karte wird dann mit den *MapContainer* und *TileLayer* von Leaflet erstellt. Der *TileLayer* zeigt die OpenStreetMap-Kartenkacheln an. Die Route wird mit *Polyline* dargestellt, wobei zwei Linien gezeichnet werden: eine schwarze Linie für den Hauptpfad und eine weiße Linie für den Hintergrund, um die Sichtbarkeit zu verbessern. [6]

```
<MapContainer
  center={center}
  zoom={17}
  style={{ height: "80vh", width: "100%" }}
>
  <TileLayer
    url="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png"
    attribution="&copy; OpenStreetMap contributors"
  />
  {route.length > 1 && (
    <Polyline
      positions={route}
      pathOptions={{ color: "black", weight: 8 }}
    />
  )}
  {route.length > 1 && (
    <Polyline
      positions={route}
      pathOptions={{ color: "white", weight: 4 }}
    />
  )}
</MapContainer>
```

Listing 5: Mapcontainer

4.3.1.4. Platzierung von Start-, Ziel- und Zwischenzeit-Markern

Die Platzierung von Start-, Ziel- und Zwischenzeit-Markern erfolgt basierend auf den GPX-Daten. Dazu wird der erste Punkt der Route als Startpunkt und der letzte Punkt als Zielpunkt festgelegt. In der *StartGoalMarkers*-Komponente werden Marker an diesen Punkten positioniert und die entsprechenden Popups angezeigt.

Die folgenden Marker-Komponenten sind verantwortlich für die Darstellung der Start- und Ziemarkierungen sowie für die Zwischenzeitmarken auf der Karte. Der Marker für den Startpunkt wird an der ersten Position der Route platziert und der für den Zielpunkt an der letzten Position, sollte Start und Ziel sich an derselben Position befinden, dann wird nur ein Marker erstellt (Start/Ziel).

```
const StartGoalMarkers = ({ route }) => {  
  const startPoint = route[0];  
  const goalPoint = route[route.length - 1];  
  
  return (  
    <>  
      <Marker position={startPoint} icon={StartGoalIcon}>  
        <Popup>Startpunkt</Popup>  
      </Marker>  
      <Marker position={goalPoint} icon={StartGoalIcon}>  
        <Popup>Zielpunkt</Popup>  
      </Marker>  
    </>  
  );  
};
```

Listing 6: StartGoalMarker

Zwischenzeit-Markierungen, die für bestimmte Abschnitte der Strecke relevant sind, werden ebenfalls in der gleichen Weise hinzugefügt. Die Positionen für die Zwischenzeitmarker werden vom Backend entnommen, da diese die administrierende Person in der Datenbank speichert. [6]

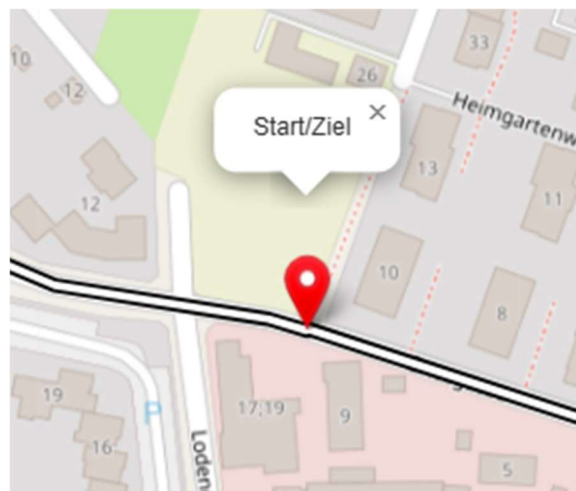


Abbildung 12: StartGoalMarker

4.3.1.5. Marker für Teilnehmende

Die Darstellung der Teilnehmenden erfolgt durch die *ParticipantMarker*-Komponente, die an den errechneten Positionen der Teilnehmenden auf der Karte angezeigt werden. Diese werden mit Echtzeit-Daten aktualisiert, um die ungefähren Positionen der Teilnehmenden anzuzeigen.

Für die visuelle Darstellung wird jedem Teilnehmenden ein spezielles benutzerdefiniertes Icon zugewiesen, das mit einer dynamischen Farbe versehen wird. Diese Farbe wird auf Grundlage der Startnummer des Teilnehmenden berechnet. Die Farbe wird durch die folgende Formel erzeugt.

$$\text{Farbe} = (\text{startnummer} * 137) \bmod 360$$

Der Faktor 137 ist willkürlich gewählt, sorgt aber für eine abwechslungsreiche Verteilung der Farben. Durch Modulo 360 wird sichergestellt, dass sich der Berechnete Farbwert im gültigen Bereich von 0-360 befindet.

Zusätzlich zum Marker wird ein Popup für jeden Teilnehmenden angezeigt, das beim Klicken auf den Marker geöffnet wird. Der Popup zeigt wichtige Informationen über den Teilnehmenden an, darunter den Vornamen und Nachnamen, den Jahrgang, den Verein sowie die aktuelle Geschwindigkeit des Teilnehmenden. Ist die erwartete Zielzeit des Teilnehmenden verfügbar und hat dieser das Ziel noch nicht erreicht, wird auch die voraussichtliche Zielzeit angezeigt. Ein Link zu den Ergebnissen des Teilnehmenden wird ebenfalls im Popup angezeigt, damit Benutzende direkt zum Eintrag in der Ergebnisliste gelangen können. [19]

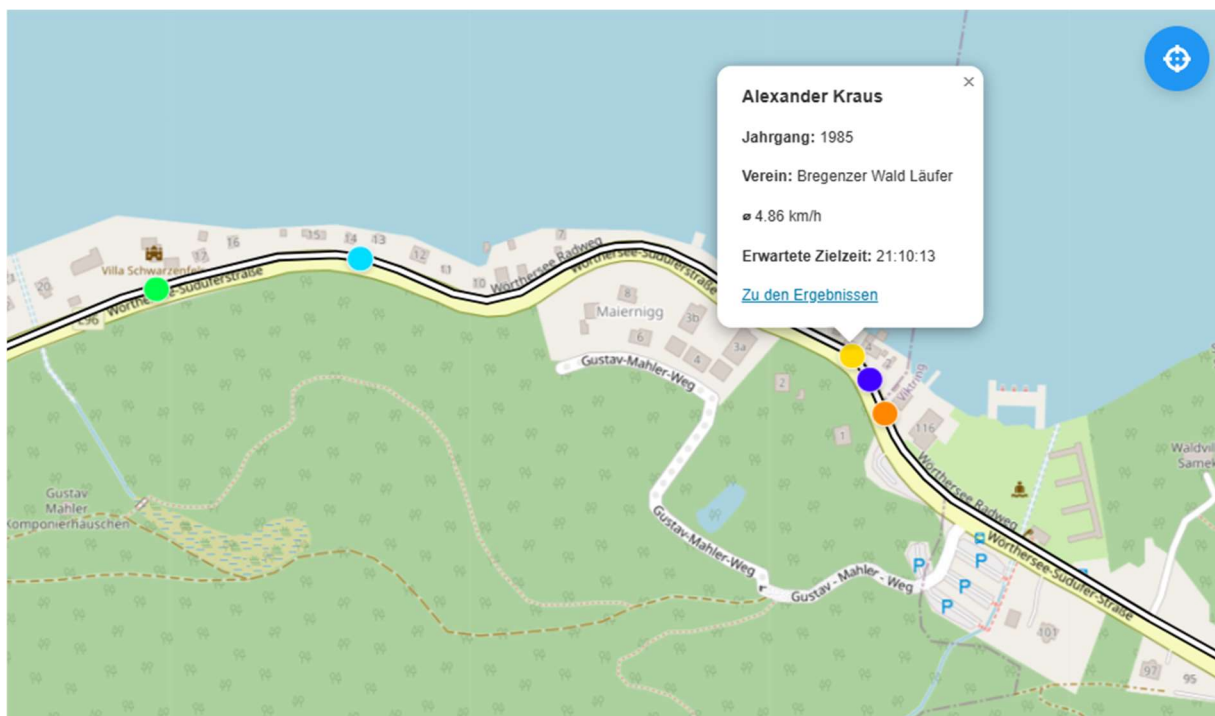


Abbildung 13: Marker + Popup der Teilnehmenden

4.3.2. Berechnung der Geschwindigkeit und Position der Teilnehmenden

Die Berechnung der Geschwindigkeit und Position der teilnehmenden Personen erfolgt in zwei wesentlichen Schritten: Zunächst wird die durchschnittliche Geschwindigkeit jeder teilnehmenden Person auf der Strecke berechnet. Anschließend wird die zurückgelegte Entfernung ermittelt, um den Fortschritt der Person auf der Karte darzustellen.

4.3.2.1. Berechnung der Geschwindigkeit

Die durchschnittliche Geschwindigkeit einer teilnehmenden Person wird basierend auf den Zwischenergebnissen, welche per API kommen errechnet. Da die Position des jeweiligen Zwischenmesspunkt bekannt ist und die Zeit, die ein Teilnehmer bis zu diesem Punkt brauchte, ebenfalls bekannt ist, kann damit sehr simpel die durchschnittliche Geschwindigkeit berechnet werden, indem die Distanz von Start bis zum Zwischenmesspunkt durch die Zeit der teilnehmenden Person dividiert wird.

Berechnung der Position und des Fortschritts

Nachdem die Geschwindigkeit berechnet wurde, geht es darum, den Fortschritt der Teilnehmenden auf der Strecke zu ermitteln. Der Fortschritt wird in Prozent angegeben, da beim Erstellen der *ParticipantMarker*-Komponente der Fortschritt in Prozent gebraucht wird.

Die gelaufene Distanz (*currentDistance*) des Teilnehmenden wird anhand der berechneten Geschwindigkeit und der verstrichenen Zeit ermittelt

$$\text{Distanz} = \text{Geschwindigkeit} * \text{vergangender Zeit}$$

Der *progress* wird dann auf Basis der aktuellen Distanz berechnet:

```
const progress = Math.min(100, (currentDistance / totalDistance) * 100);
```

Listing 7: Berechnung des Fortschritts

Die Methode „*Math.min()*“ stellt sicher, dass der Fortschritt niemals mehr als 100% beträgt.

4.3.3. ElevationChart

Die *ElevationChart*-Komponente ist ein zentrales Element der *UserLiveMap*, das eine visuelle Darstellung des Höhenprofils der Strecke bietet. Diese Visualisierung ermöglicht es den Zusehenden und Teilnehmenden, die topografischen Merkmale der Strecke nachzuvollziehen

4.3.3.1. Wahl von *chart.js* für die Implementierung

Für die Umsetzung des *ElevationCharts* wurde *chart.js* gewählt, eine weit verbreitete JavaScript-Bibliothek zur Erstellung von interaktiven Diagrammen. Die Entscheidung für *chart.js* fiel aufgrund seiner Flexibilität, Benutzerfreundlichkeit und Leistungsfähigkeit. Insbesondere bietet die Bibliothek eine einfache API, die es ermöglicht, Daten schnell in ansprechende

Diagramme zu transformieren. Diese Eigenschaften machen *chart.js* besonders geeignet für Web-Anwendungen, in denen sowohl die Benutzererfahrung als auch die technische Effizienz von Bedeutung sind. [8]

4.3.3.2. Funktionsweise des ElevationCharts

Das *ElevationChart* visualisiert das Höhenprofil einer Strecke, das aus GPX-Daten extrahiert wird. Die GPX-Datei enthält Informationen zu den GPS-Koordinaten und Höhenpunkten der Strecke, die dann in einem interaktiven Liniendiagramm dargestellt werden. Die x-Achse des Diagramms repräsentiert die Distanz entlang der Strecke, während die y-Achse die Höhe (in Metern) darstellt. Diese Darstellung gibt den Benutzenden eine klare Vorstellung von den Steigungen und Gefällen der Strecke.

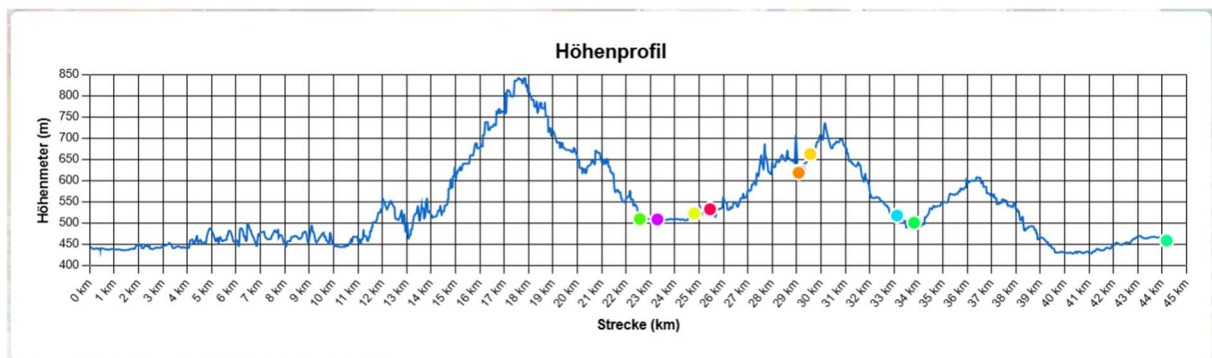


Abbildung 14: ElevationChart

4.3.3.3. Extrahierung der Daten aus der GPX-Datei

Die erste Phase der ElevationChart-Erstellung besteht in der Extraktion der erforderlichen Daten aus einer GPX-Datei. Die GPX-Datei enthält die geografischen Koordinaten der Strecke, die als *Trackpoints* gespeichert sind, sowie die Höhenangaben. Der *DOMParser* wird verwendet, um diese Daten aus der GPX-Datei zu extrahieren und in ein brauchbares Format zu bringen. Dabei wird jeder *Trackpoint* verarbeitet, und es wird die zurückgelegte Strecke sowie die Höhenangabe gespeichert.

Die Funktion *parseGPXtoElevationData* übernimmt diese Aufgabe. Sie liest die GPX-Daten ein und berechnet die kumulierte Distanz für jedes *Trackpoint*. Dabei wird der Haversine-Algorithmus verwendet, um die Distanz zwischen zwei GPS-Koordinaten zu berechnen, da dieser Algorithmus besonders gut für relativ kleine Distanzen geeignet ist, wie sie in GPS-basierten Anwendungen häufig vorkommen. [20]

```
const parseGPXtoElevationData = (gpxText) => {
  const parser = new DOMParser();
  const xmlDoc = parser.parseFromString(gpxText, 'text/xml');
  const trackPoints = xmlDoc.getElementsByTagName('trkpt');

  let cumulativeDistance = 0;
  const R = 6371000; // Erdradius in Metern
  const toRad = (deg) => (deg * Math.PI) / 180;

  let prevLat = null, prevLon = null;
  const data = [];

  for (let i = 0; i < trackPoints.length; i++) {
    const lat = parseFloat(trackPoints[i].getAttribute('lat'));
    const lon = parseFloat(trackPoints[i].getAttribute('lon'));
    const elevation = parseFloat(trackPoints[i].getEle-
mentsByTagName('ele')[0]?.textContent || 0);

    if (prevLat !== null && prevLon !== null) {
      const dLat = toRad(lat - prevLat);
      const dLon = toRad(lon - prevLon);
      const a = Math.sin(dLat / 2) ** 2 + Math.cos(toRad(prevLat)) *
Math.cos(toRad(lat)) * Math.sin(dLon / 2) ** 2;
      const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
      cumulativeDistance += R * c;
    }
    prevLat = lat;
    prevLon = lon;

    data.push({ distance: cumulativeDistance, elevation });
  }
  return data;
};
```

Listing 8: parseGPXtoElevationData

Die Funktion *parseGPXtoElevationData* extrahiert die GPS-Koordinaten und Höhenangaben aus einer GPX-Datei und berechnet die kumulierte horizontale Distanz zwischen den *Track-points*. Dabei wird der Haversine-Algorithmus verwendet, um die Distanz zwischen den GPS-Koordinaten zu berechnen, was für kleine Distanzen eine präzise Methode darstellt. Die kumulierte Distanz und die Höhenangaben werden in einem Array gespeichert, das später zur Visualisierung des Höhenprofils im ElevationChart genutzt wird. Diese Methode stellt sicher, dass die Höhenangaben entlang der Strecke korrekt mit der zurückgelegten Distanz kombiniert werden. [20] [21]

4.3.3.4. Visualisierung der Daten im Chart

Nach der Verarbeitung der Höhen- und Distanzdaten werden diese in einem *chart.js*-Diagramm dargestellt. Die x-Achse des Diagramms zeigt die zurückgelegte Distanz in Kilometern, während die y-Achse die Höhe in Metern darstellt. Zusätzlich werden die Teilnehmenden als Marker angezeigt, wobei ihre Position basierend auf dem Fortschritt und der Höhe berechnet wird. Jeder Marker erhält eine individuelle Farbe, die von der Startnummer des Teilnehmenden abhängt, um eine visuelle Unterscheidung zu ermöglichen. Hierbei wird dieselbe Berechnung wie bei den *ParticipantMarker* verwendet.

```
const chartData = {
  datasets: [
    // Höhenprofil-Daten
    {
      label: 'Elevation',
      data: dataPoints.map(dp => ({ x: dp.distance / 1000, y:
dp.elevation })),
      borderColor: '#0c67c9',
      borderWidth: 2,
      fill: false,
      tension: 0.3,
      pointRadius: 0,
    },
    // Teilnehmer-Daten
    {
      label: 'Participants',
      data: participants.map(participant => {
        const progressIndex = Math.floor((participant.progress /
100) * dataPoints.length);
        const point = dataPoints[progressIndex];
        return {
          x: point.distance / 1000,
          y: point.elevation,
          name: `${participant.Vorname} ${participant.Nachname}`,
          startnr: participant.startnr,
        };
      }),
      pointRadius: 7,
      pointBackgroundColor: participants.map(participant =>
getColorFromParticipant(participant.startnr)),
      pointBorderColor: '#ffffff',
      pointBorderWidth: 2,
      showLine: false,
    },
  ],
};
```

Listing 9: chartData

Zunächst werden die Höhenprofil-Daten verarbeitet: Die Distanz wird von Metern in Kilometer umgerechnet, während die Höhe unverändert bleibt. Diese Daten werden als geschwungene blaue Linie im Diagramm dargestellt, wobei die Krümmung durch den *tension*-Wert gesteuert wird.

Teilnehmende werden als Marker angezeigt, deren Position anhand ihres Fortschritts in Prozent berechnet wird. Dadurch wird genau bestimmt, wo sich eine Person auf dem Höhenprofil befindet. Die Marker erhalten eine Farbe, die aus der Startnummer abgeleitet wird (*getColorFromParticipant*), und sind rund (Radius 7) mit einer weißen Umrandung, um eine klare visuelle Abgrenzung zu gewährleisten.

Zusätzlich verfügt das Diagramm über interaktive Funktionen wie *Tooltips* oder *Hover*-Effekte. Beim Überfahren eines Markers werden weitere Details wie Name, Startnummer oder die zurückgelegte Strecke angezeigt, was eine intuitive Nutzererfahrung ermöglicht.

[22]

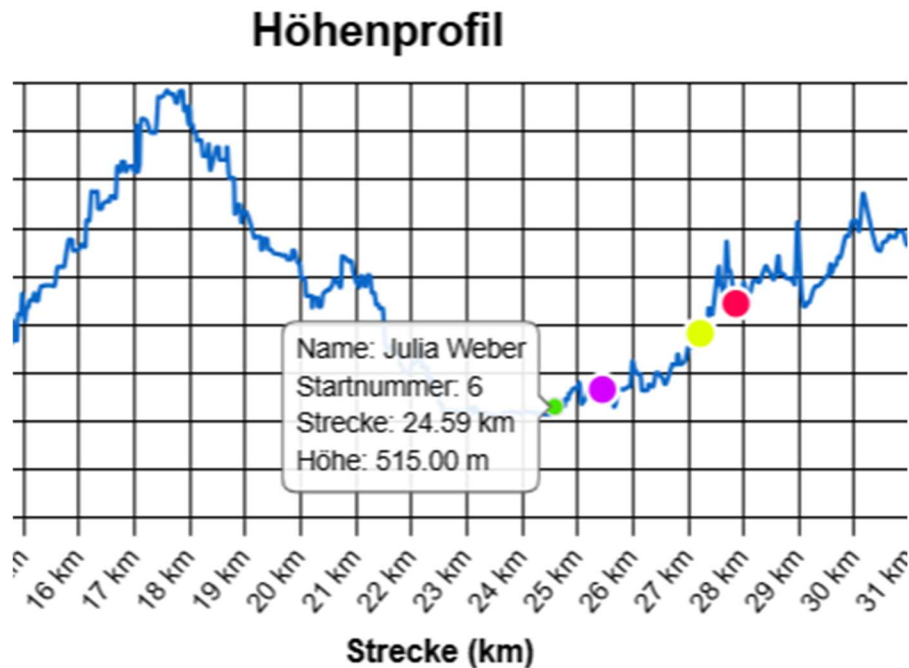


Abbildung 15: ElevationChart-Popup

4.4. Implementierung der Ergebnistabelle

Die Ergebnistabelle ist ein zentraler Bestandteil der *UserResults*-Seite und bildet zusammen mit der Live-Map einen wichtigen Bereich der Webapplikation. Sie ermöglicht eine schnelle und übersichtliche Einsicht in die Veranstaltungsdaten, indem sie relevante Informationen wie Namen, Platzierung, Zeit und Verein der Teilnehmenden auflistet.

Die Tabelle ist dynamisch und bietet Funktionen zur Sortierung und Filterung. Nutzer können nach Platzierung, Zeit oder Verein sortieren und die Ansicht nach Altersklassen (AK) oder Geschlecht (MW) gruppieren, um gezielte Vergleiche zu ermöglichen.

Die Daten werden kontinuierlich über die RACE RESULT 12 SimpleAPI aktualisiert, sodass Platzierungen und Zeiten stets in Echtzeit synchronisiert werden. Dadurch bleibt die Ergebnistabelle jederzeit auf dem aktuellen Stand und unterstützt eine präzise Echtzeit-Verfolgung der Veranstaltung. [1]

Ein weiteres zentrales Feature ist die Verknüpfung mit den Teilnehmer-Markern auf der Live-Map. Jeder Marker-Popup enthält einen Link zur entsprechenden Zeile in der Ergebnistabelle. So können Nutzende schnell zwischen Karte und Tabelle navigieren und sich gezielt über einzelne Teilnehmende informieren.

4.4.1. Interaktive Features der Ergebnistabelle

Die Ergebnistabelle ist nicht nur eine einfache Übersicht der Ergebnisse, sondern bietet eine Vielzahl interaktiver Funktionen, die es den Benutzenden ermöglichen, die angezeigten Daten nach ihren individuellen Bedürfnissen zu filtern und anzupassen.

4.4.1.1. Wechsel zwischen Altersklassen (AK) und Geschlecht (MW)

Ein zentrales Feature der *ResultTable* ist die Möglichkeit, zwischen den Kategorien Altersklassen (AK) und Geschlecht (MW) zu wechseln. Dies ermöglicht eine benutzerfreundliche Filterung der angezeigten Daten, sodass Benutzende gezielt die Ergebnisse für eine bestimmte Altersklasse oder für Männer und Frauen einsehen können. Diese Funktion wird durch Schaltflächen realisiert, die es den Nutzenden ermöglichen, zwischen den Kategorien zu wechseln.

Die Tabelle wird nach Auswahl der Kategorie dynamisch neu geladen und gruppiert die Ergebnisse nach der gewählten Kategorie. Dieser Wechsel wird mithilfe von React-Komponenten realisiert, wobei die Anzeige der Daten durch den `useState`-Hook angepasst wird, um die Ergebnisse basierend auf der Auswahl des Benutzenden zu aktualisieren.

```
const [selectedCategory, setSelectedCategory] = useState("AK");

const handleCategoryChange = (category) => {
  setSelectedCategory(category);
};

const filteredResults = results.filter(result => result.category ===
selectedCategory);
```

Listing 10: ResultTable Kategorienwechsel

Der Zustand `selectedCategory` wird mit `useState("AK")` initialisiert, sodass zu Beginn die Kategorie Altersklassen (AK) vorausgewählt ist. Die Funktion `handleCategoryChange` ermöglicht es, diesen Wert zu ändern. Wird sie durch eine Benutzerinteraktion aufgerufen, setzt sie den Zustand auf die jeweils gewählte Kategorie – entweder AK oder MW (Männer/Frauen).

Die Ergebnistabelle wird anschließend mithilfe der `filter`-Methode angepasst: Es werden nur jene Einträge angezeigt, deren `category`-Wert mit `selectedCategory` übereinstimmt. Dadurch aktualisiert sich die Tabelle dynamisch und zeigt jeweils nur die relevanten Ergebnisse für die gewählte Kategorie.

Durch diese Funktion kann die Tabelle für spezifische Gruppen wie Altersklassen oder Männer/Frauen gefiltert werden, was vor allem bei großen Events mit verschiedenen Kategorien von Bedeutung ist. So bleibt die Seite übersichtlich, und die Daten sind leicht zugänglich. [23]

4.4.1.2. Sortierung der Tabelle

Ein weiteres interaktives Feature der *ResultTable* ist die Möglichkeit, die Ergebnisse nach verschiedenen Kriterien zu sortieren. Durch einen Klick auf die Spaltenüberschrift können Benutzende die Tabelle nach Platzierung, Zeit, Verein oder anderen relevanten Kriterien auf- oder absteigend anordnen. Dies erleichtert die individuelle Anpassung der Anzeige und verbessert die Navigation durch die Daten.

Die Sortierung erfolgt direkt im Frontend und wird mithilfe des `useState`-Hooks verwaltet. Dieser speichert das aktuell gewählte Sortierkriterium und passt die Anordnung der Einträge dynamisch an. Dadurch wird eine schnelle und flexible Darstellung der Ergebnisse gewährleistet.

```
const [sortOrder, setSortOrder] = useState("asc"); // standardmäßig
aufsteigend sortieren

const handleSort = (column) => {
  const sortedResults = [...filteredResults].sort((a, b) => {
    // Vergleiche je nach Spalte
    if (column === "time") {
      return sortOrder === "asc" ? a.time - b.time : b.time - a.time;
    } // Sortiere nach Zeit
    if (column === "placement") {
      return sortOrder === "asc" ? a.placement - b.placement :
b.placement - a.placement; // Sortiere nach Platzierung
    }
    return 0;
  });

  // Wechsel der Sortierreihenfolge (aufsteigend/absteigend)
  setSortOrder(sortOrder === "asc" ? "desc" : "asc");

  setSortedResults(sortedResults); // Setze die sortierten Ergebnisse
};
```

Listing 11: Sortierfunktion

Die Funktion *handleSort* ermöglicht die Sortierung der *ResultTable*-Daten nach einer ausgewählten Spalte, z. B. Zeit oder Platzierung. Dabei wird der Zustand *sortOrder* verwaltet, der zwischen aufsteigender („*asc*“) und absteigender („*desc*“) Sortierreihenfolge wechselt.

Beim Aufruf der Funktion wird die Liste der Ergebnisse mit der *sort*-Methode entsprechend den gewählten Spalten sortiert.

- Ist *sortOrder* auf „*asc*“ gesetzt, erfolgt die Sortierung in aufsteigender Reihenfolge.
- Bei „*desc*“ wird die Liste in absteigender Reihenfolge geordnet.

Nach jeder Sortierung wird *sortOrder* umgekehrt, sodass ein erneuter Klick auf dieselbe Spalte die Reihenfolge wechselt. Abschließend werden die sortierten Ergebnisse im Zustand *sortedResults* gespeichert und in der Tabelle aktualisiert. [24]

MW PL	STARTNR	VORNAME	NACHNAME	NAT.	JG	AK	VEREIN	ZW1	ZW2	ZW3	ZW4	ZW5	ZW6	ZW7	ZEIT
11	1168	__oma_	Roberts	GBR	1950	U70	A_hletikverein t_	01:03	34:55	1:02:54					
37	1120	Jonas	Meyer	GER	1950	U70	L_ufldu_Mu_ch_n	01:22	39:24						
96	96	Sebastian	Müller	AUT	1950	U70	Laufteam Tirol	01:46	52:02						
117	70	Johann	Maier	AUT	1950	U70	LAC Salzburg	01:57	56:27						
131	1188	Al_in	Dubo_s	FRA	1950	U70	Sportverei_Lil_e	01:58	59:22						
142	1398	Leon	Müller	GER	1950	U70	Leichtath_etik_üb_Fr_n_furt	01:56							
156	1436	Luca	Ferrari	ITA	1950	U70	Lei_tath_etik_Club_Rome	02:13							

Abbildung 16: Nach Jahrgang sortierte Tabelle

4.4.1.3. Suchfunktion

Zusätzlich zur Sortierung und dem Wechsel zwischen Kategorien bietet die *ResultTable* eine Suchleiste, mit der Benutzende nach spezifischen Teilnehmenden suchen können. Die Suchfunktion ermöglicht es den Nutzenden, direkt nach Namen, Startnummern oder anderen Kriterien zu suchen. Dies ist besonders nützlich bei großen Veranstaltungen, bei denen die Anzahl der Teilnehmenden hoch ist und eine gezielte Suche erforderlich ist, um schnell die gewünschte Person zu finden.

Die Such- und Filterfunktion wird durch den *useState*-Hook unterstützt, der den aktuellen Suchbegriff speichert und die Tabelle entsprechend aktualisiert. So können die Ergebnisse sofort gefiltert werden, ohne dass die Seite neu geladen werden muss.

```
const [searchQuery, setSearchQuery] = useState("");

const handleSearch = (event) => {
  setSearchQuery(event.target.value);
};

const filteredBySearch = filteredResults.filter(result =>
  result.name.toLowerCase().includes(searchQuery.toLowerCase())
);
```

Listing 12: Suchfunktion

Der *searchQuery*-Zustand speichert den aktuellen Suchbegriff, der von den Endnutzenden eingegeben wird. Mit der Funktion *handleSearch* wird dieser Zustand aktualisiert, wenn in das Suchfeld getippt wird. Die *filteredBySearch*-Variable filtert dann die *filteredResults*-Liste, indem sie nach Ergebnissen sucht, bei denen der Name des Teilnehmers die Zeichenkette aus *searchQuery* enthält, wobei die Groß- und Kleinschreibung ignoriert wird. [25]

4.4.1.4. Verbindung von ParticipantMarker und der UserResult-Seite

Ein weiteres Feature der *ResultTable* ist die direkte Verknüpfung mit den Teilnehmenden-Markern auf der Live-Map. Jeder *ParticipantMarker* auf der Karte enthält einen Link, der den Benutzenden direkt zur *UserResults*-Seite führt und dort die Ergebnisse des Teilnehmenden anzeigt. Dies ermöglicht eine nahtlose Navigation zwischen der Live-Map und der *ResultTable*.

Die Verlinkung wird durch den Namen der teilnehmenden Person ermöglicht. Jeder *ParticipantMarker* enthält einen Link, der den Namen des Teilnehmenden als URL-Parameter überträgt.

Auf der *UserResults*-Seite wird dieser Link verarbeitet, um die *ResultTable* entsprechend zu filtern, indem jener URL-Parameter in der Suchleiste eingegeben wird. Wenn der Benutzer also auf einen Marker auf der Karte klickt, wird er direkt zur *UserResults*-Seite weitergeleitet, wo per Suchleiste schon nach dem Teilnehmer gesucht wurde

```
const ResultPath =
  `/event/${id}/userResults?list=${encodeURIComponent(listlink)}&search=${encodeURIComponent(`${Vorname} ${Nachname}`)}`;
<p><a href={ResultPath}>Zu den Ergebnissen</a></p>
```

Listing 13: Participant Marker Results-Link

MW PL	STARTNR.	VORNAME	NACHNAME	NAT.	JG	AK	VEREIN	ZW1	ZW2	ZW3	ZW4	ZW5	ZW6	ZW7	ZEIT
72	11	Franziska	Pichler	AUT	1968	Ü50	Steirer Sprinter	01:56	54:38						

Abbildung 17: Gefilterte Ergebnisliste

In der Abbildung 17 lässt sich erkennen, wie die *UserResult* – Seite aussieht, wenn man im *ParticipantMarker* – Popup eines Teilnehmenden auf den Link klickt.

4.4.2. Berechnung der Platzierungen

Die Berechnung der Platzierungen basiert in erster Linie auf den endgültigen Zielzeiten der teilnehmenden Personen. Sollte eine Person noch keine Zielzeit erreicht haben, wird

stattdessen die aktuelle verfügbare Zwischenzeit herangezogen. Diese wird mit den Zwischenzeiten anderer Personen verglichen, die ebenfalls noch keine endgültige Zielzeit haben.

Die Platzierung erfolgt nach dem folgenden Prinzip: Personen mit einer gültigen Zielzeit werden stets vor jenen gereiht, die noch keine Zielzeit erreicht haben. Unter den Teilnehmenden ohne Zielzeit entscheidet die höchste erreichte Zwischenzeit über die Reihenfolge – das bedeutet, dass eine Person, die eine spätere Zwischenzeit erfasst hat, besser platziert wird als jemand, dessen letzte Zwischenzeit weiter zurückliegt.

In der *ResultTable* wird dieser Berechnungsprozess durch die Funktion *calculatePlacement* umgesetzt. Dabei werden die Zeiten der Teilnehmenden in mehreren Schritten analysiert: Zunächst erfolgt die Sortierung anhand der endgültigen Zielzeiten. Fehlt diese, wird die neueste verfügbare Zwischenzeit herangezogen. Falls mehrere Personen denselben Fortschritt aufweisen, entscheidet die erreichte Zwischenzeit über die Reihenfolge.

```
const calculatePlacement = (data) => {
  return data.sort((a, b) => {
    const timeA = TimeStringToSeconds(a.Zeit || "");
    const timeB = TimeStringToSeconds(b.Zeit || "");

    if (timeA !== Infinity && timeB === Infinity) return -1; // a hat
    eine Zeit, b nicht
    if (timeA === Infinity && timeB !== Infinity) return 1; // b hat
    eine Zeit, a nicht

    if (timeA !== Infinity && timeB !== Infinity) {
      if (timeA < timeB) return -1;
      if (timeA > timeB) return 1;
    }

    const zwCountA = Object.keys(a).filter(key => /^ZW\d+$/.test(key)
    && a[key] !== "").length;
    const zwCountB = Object.keys(b).filter(key => /^ZW\d+$/.test(key)
    && b[key] !== "").length;

    if (zwCountA > zwCountB) return -1;
    if (zwCountA < zwCountB) return 1;

    for (let i = Math.max(...Object.keys(a).filter(k =>
    /^ZW\d+$/.test(k))).map(k => parseInt(k.replace('ZW', ''))); i >= 1; i--) {
      const zwKey = `ZW${i}`;
      if (a[zwKey] !== "" && b[zwKey] !== "") {
        const timeA = TimeStringToSeconds(a[zwKey]);
        const timeB = TimeStringToSeconds(b[zwKey]);

        if (timeA < timeB) return -1;
        if (timeA > timeB) return 1;
      }
    }

    return 0;
  });
};
```

Listing 14: Berechnung der Platzierung

In diesem Code werden die Teilnehmenden zunächst anhand ihrer Zeiten verglichen. Falls eine Person bereits eine Zielzeit erreicht hat, während eine andere noch keine hat, wird die Person mit der vorhandenen Zeit höher platziert. Haben beide eine Zielzeit, entscheidet die bessere (niedrigere) Zeit über die Reihenfolge.

Sollten zwei Teilnehmende exakt die gleiche Zeit aufweisen, wird die Anzahl der erfassten Zwischenzeiten als weiteres Kriterium herangezogen. In diesem Fall erhält die Person mit mehr Zwischenzeiten eine höhere Platzierung, da dies darauf hinweist, dass sie weiter im Rennen fortgeschritten ist. [26]

5. Softwareentwicklung des Administratorinterfaces und Backend (Maximilian Wolf Leopold Lindner)

In diesem Teilbereich wird das Administratorinterface für Event Insight entwickelt, das den Veranstaltenden eine zentrale Plattform zur Verwaltung der Renndaten bietet. Es ermöglicht das Erfassen und Verwalten von Echtzeit-Ergebnissen sowie Teilnehmerdaten. Veranstalter*innen können wichtige Informationen wie Startzeiten und Streckenbeschreibungen einsehen und bei Bedarf anpassen.

Das Interface bietet Funktionen zur Erstellung, Bearbeitung und Löschung von Veranstaltungen. Zudem haben die Veranstalter*innen die Möglichkeit, die Ansicht für die Zuschauenden individuell anzupassen. Hierbei können insbesondere die Darstellung von Ergebnissen, Streckenübersichten und andere relevante Informationen für die Zuschauer*innen angepasst werden.

Ein weiteres wichtiges Element ist das Login-System, das sicherstellt, dass nur berechtigte administrierende Personen auf die Verwaltungsfunktionen haben. Über dieses System wird auch geregelt, welche administrierende Personen Zugang zu den verschiedenen Veranstaltungen hat.

Das Backend unterstützt diese Funktionen durch API-Endpunkte, die für die Datenverwaltung, die Benutzerauthentifizierung sowie die Kommunikation mit der Datenbank zuständig sind. Dabei werden Informationen zu Events, Teilnehmer*innen und Ergebnissen in Echtzeit verarbeitet und aktualisiert.

5.1. Systemarchitektur und Design

Jede Seite im Interface ist auf die Aufgaben der administrierenden Personen ausgelegt. Auf der AdminHome-Seite sind die wichtigsten Event-Daten schnell sichtbar, und administrierende Personen können Event-Informationen bearbeiten. Auf der AdminLiveMap-Seite können GPX-Daten hochgeladen und Zwischenzeiten auf der Karte eingezeichnet werden. Auf der AdminSponsors-Seite lassen sich Sponsorenbilder verwalten. Auf der AdminResults Seite können administrierenden Personen Live-Maps mit API-Links verknüpfen.

Im Vergleich zum Interface für die zuschauenden Personen ist das Administratorinterface farblich eher schlicht gehalten. Das Interface ist nicht blau gefärbt, sondern grau, um eine farbliche Trennung zu erschaffen.

5.1.1. AdminHome.jsx

Die AdminHome-Seite ist der zentrale Punkt für administrierende Personen, um auf alle relevanten Funktionen der Eventverwaltung zuzugreifen. Auf der Startseite sind die wichtigsten Informationen zum Event übersichtlich dargestellt, einschließlich des Eventnamens und des Datums. Administrierende Personen haben zudem die Möglichkeit, den Event-Text über einen Button zu bearbeiten, welcher nach Bearbeitung auf der Startseite des Enduserinterfaces angezeigt wird

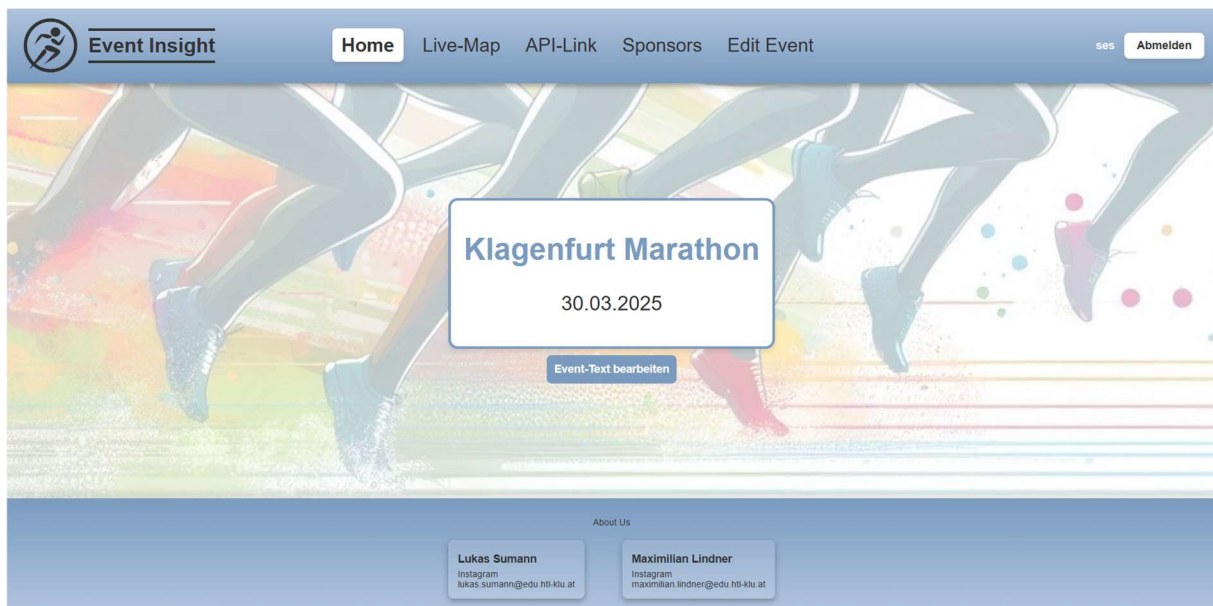


Abbildung 18: Startseite des Administratorinterfaces

5.1.2. AdminLiveMap.jsx

In der AdminLiveMap-Seite können administrierende Personen Live-Maps hochladen und Zwischenzeiten einzeichnen. Das Interface besteht aus zwei Hauptbereichen:

- **Live-Map Hochladen:** Hier können administrierende Personen eine neue Live-Map hochladen, indem sie einen Kartennamen, eine Startzeit und eine Datei auswählen. Die Datei wird dann zusammen mit den anderen Daten über ein Formular an das Backend gesendet.
- **Zwischenzeiten einzeichnen:** In diesem Bereich können administrierende Personen eine Live-Map auswählen und sich anzeigen lassen. Die Auswahl erfolgt über eine "SearchableSelect"-Komponente, die eine Liste der verfügbaren Live-Maps anzeigt. Nachdem eine Karte ausgewählt wurde, können Marker auf der Karte gesetzt und gespeichert werden, um Zwischenzeiten für den Eventverlauf festzulegen.

Werden GPX-Dateien hochgeladen, wird lediglich der Pfad zu der Datei in der Datenbank gespeichert, diese befindet sich tatsächlich im Projektlaufwerk unter *backend/src/uploads/livemaps*.

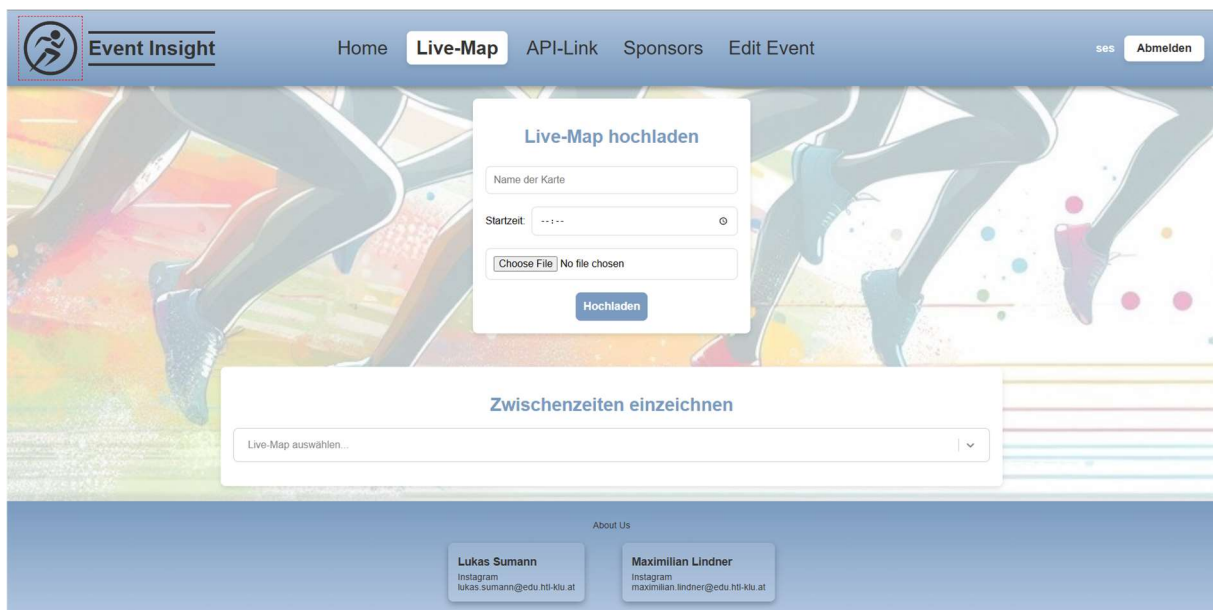


Abbildung 19: Webpage des Administratorinterfaces zum Verwalten von Live-Maps

5.1.3. AdminSponsors.jsx

Die AdminSponsors-Seite ermöglicht es administrierenden Personen, die Sponsoren des Events zu verwalten. Es gibt zwei Funktionen:

- **Sponsorbild hochladen:** Auf dieser Seite können administrierenden Personen ein Sponsorbild hochladen, indem sie den Namen des Sponsors eingeben, ein Bild auswählen und den Upload-Button drücken. Das Bild wird dann in die Liste der Sponsoren aufgenommen, und administrierenden Personen können entscheiden, ob es auf der Event-Seite angezeigt wird.
- **Sponsorenliste verwalten:** Es wird eine Liste der hochgeladenen Sponsoren angezeigt. Jeders Sponsorbild hat ein Kästchen, mit dem administrierenden Personen auswählen können, ob der Sponsor auf der Startseite des Enduserinterfaces sichtbar sein soll.

Ebenso wie bei Live-Maps, wird nur der Dateipfad in der Datenbank gespeichert, die Datei befindet sich in *backend/src/uploads/sponsors*

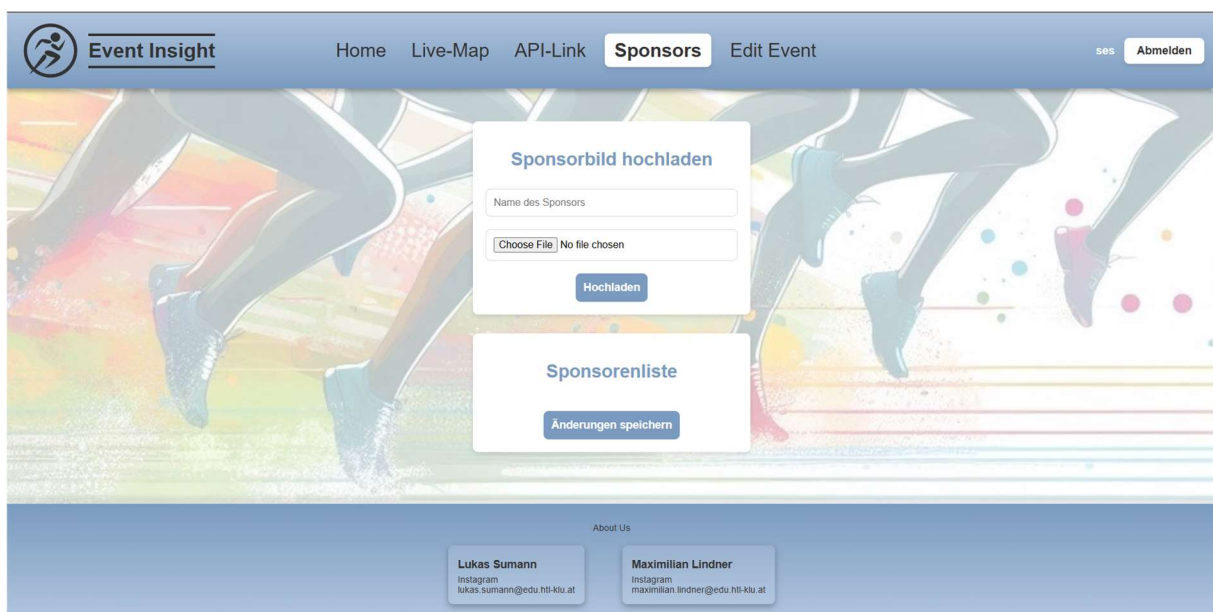


Abbildung 20: Webpage des Administratorinterfaces zum Verwalten von Sponsoren

5.1.4. AdminResults.jsx

Die AdminResults-Seite ermöglicht administrierenden Personen, einen API-Link für das Event hochzuladen, um eine Liste mit Renndaten mit einer zugehörigen Live-Map zu verknüpfen. Die Benutzeroberfläche besteht aus einem einfachen Formular, in dem der Name der Liste, der API-Link sowie die Auswahl einer Live-Map angegeben werden müssen.

- **API-Link hochladen:** Administrierende Personen können den Namen der Liste sowie den API-Link eingeben. Der Link muss dem Format `https://api.raceresult.com/` entsprechen, um gültig zu sein.
- **Live-Map auswählen:** Durch eine Drop-Down-Auswahl können administrierende Personen eine Live-Map auswählen, die mit der Liste verknüpft werden soll.

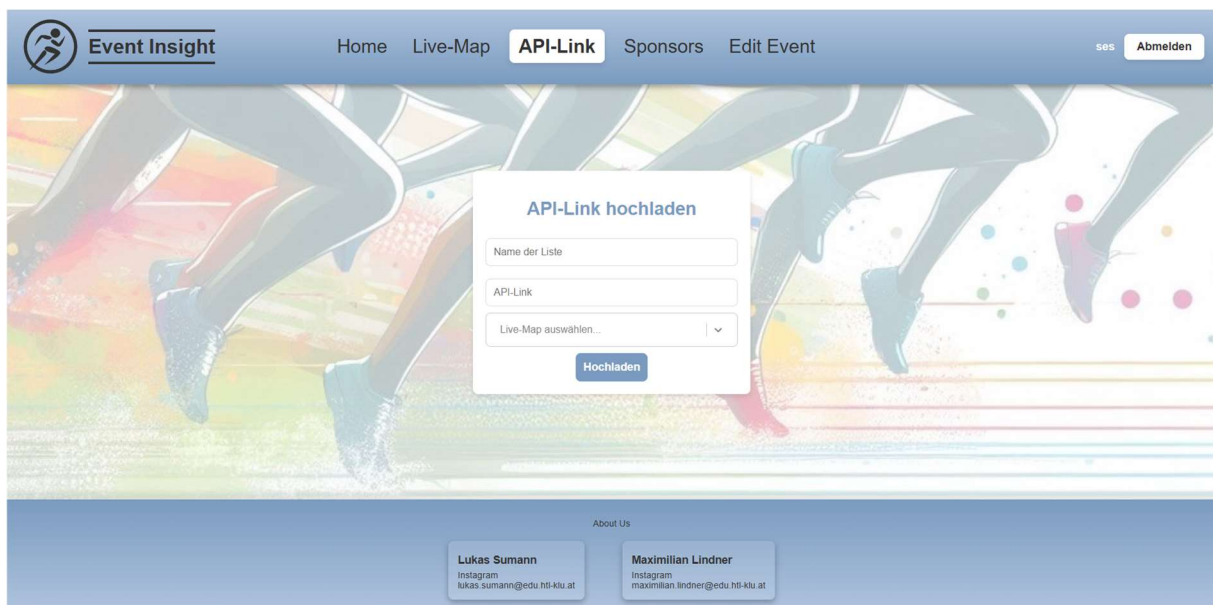


Abbildung 21: Webpage des Administratorinterfaces zum Verknüpfen von API-Links

5.1.5. EditText.jsx

Die EditText-Seite ermöglicht administrierenden Personen, den Event-Text zu bearbeiten. Diese Seite verwendet den ReactQuill-Editor, der es administrierenden Personen erlaubt, Text zu formatieren und Links hinzuzufügen.

- **Textbearbeitung:** Im Editor können administrierende Personen den Event-Text mit einer Vielzahl von Formatierungsoptionen (z.B. fett, kursiv, Listen, Überschriften) bearbeiten.
- **Speichern:** Nach dem Bearbeiten des Texts können administrierende Personen diesen speichern, und die Änderungen werden auf der Startseite des Enduserinterfaces angezeigt.

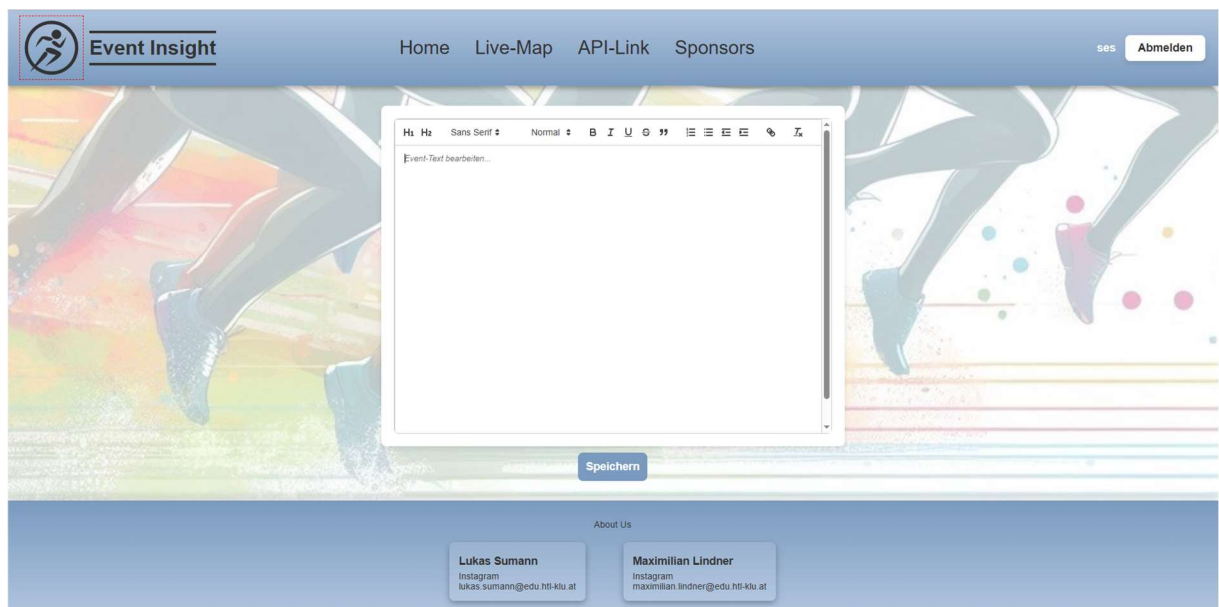


Abbildung 22: Webpage des Administratorinterfaces zum Bearbeiten von Infotext

5.1.6. AdminEdit

Die AdminEdit-Seite ermöglicht administrierenden Personen die Bearbeitung von Event-Daten, einschließlich der grundlegenden Eventinformationen sowie Karten-, Listen- und Sponsorinformationen

- **Textbearbeitung:** Auf der AdminEdit-Seite können administrierende Personen verschiedene Event-Daten bearbeiten. Unter anderem den Eventnamen, das Datum, den Ort und die Adresse. Jeder dieser Werte kann durch einfache Eingabefelder bearbeitet werden. Wenn ein Wert aktualisiert wird, erscheint ein Bearbeitungsfenster, in dem die neuen Informationen eingegeben werden können. Anschließend können die Änderungen mit einem Klick auf "Speichern" übernommen werden.
- **Karten- und Listenbearbeitung:** Administrierende Personen können auch Karten und Listen bearbeiten, die mit dem Event verbunden sind. Über eine Dropdown-Auswahl können Karten und Listen ausgewählt werden, die dann bearbeitet oder gelöscht werden können. Karten beinhalten Informationen wie den Kartennamen und die Zeit, während Listen Links zu den jeweiligen Ressourcen enthalten.
- **Sponsorenverwaltung:** Sponsoren können ebenfalls über die AdminEdit-Seite verwaltet werden. Sponsor-Informationen können bearbeitet oder gelöscht werden.

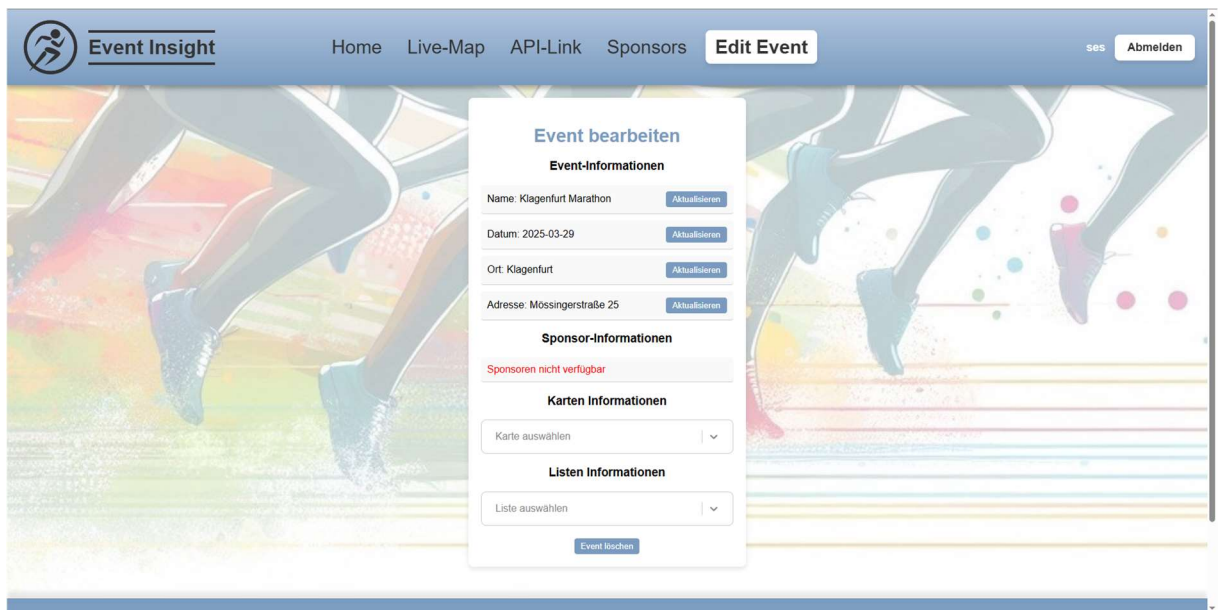


Abbildung 23: Webpage der AdminEdit Seite

5.1.7. Fehlerbehandlung in der Anwendung

Fehlerbehandlung ist ein wichtiger Bestandteil jedes Projekts, insbesondere bei der Verwendung von Systemen wie APIs und der Datenbank. Fehler können in unterschiedlichsten Bereichen auftreten, beim Abrufen von Daten, beim Speichern von Dateien oder beim Interagieren mit API-Endpunkten.

5.1.7.1. Fehlerbehandlung beim Abrufen von Daten

Ein häufiger Fehlerfall tritt beim Abrufen von Daten von einem API-Punkt. Im Administratorinterface wird die Fehlerbehandlung mithilfe von try und catch-Blöcken durchgeführt. Beispielsweise wird in der AdminLiveMap-Komponente eine API-Anfrage gestellt, um GPX-Daten abzurufen

```
const fetchDatabaseFiles = async () => {
  try {
    const response = await axios.get(`http://localhost:3001/files?eventid=${id}`);
    setGpxData(response.data.map(file => ({
      value: `http://localhost:3001/${file.mappath}`,
      label: file.mapname,
      id: file.idlivemap,
    })));
  } catch (err) {
    console.error("Fehler beim Abrufen der Daten:", err);
  }
};
```

Listing 15: Beispiel einer Fehlerbehandlung beim Abrufen von Daten

Falls die API-Anfrage fehlschlägt, wird eine Fehlermeldung in der Konsole ausgegeben.

5.1.7.2. Fehlerbehandlung beim Hochladen von Dateien

Beim Hochladen von Dateien werden ebenfalls try und catch-Blöcke verwendet, um sicherzustellen, dass Fehler während des Upload-Prozesses korrekt abgefangen werden. Ein Beispiel ist das Hochladen einer Live-Map in der AdminLiveMap-Komponente:

```
const handleSubmit = async (e) => {
  e.preventDefault();
  if (!file || !mapName.trim() || !time) {
    alert("Bitte wähle eine Datei, gib einen Kartennamen und eine Zeit ein.");
    return;
  }

  const formData = new FormData();
  formData.append("LiveMap", file);
  formData.append("mapname", mapName);
  formData.append("eventid", id);
  formData.append("maptime", time);

  try {
    const response = await fetch("http://localhost:3001/files/upload-Livemap", {
      method: "POST",
      body: formData,
    });
    const messageText = await response.text();
    alert(messageText);
  } catch (error) {
    console.error("Fehler:", error);
    alert("Fehler beim Hochladen der Live-Map");
  }
};
```

Listing 16: Beispiel einer Fehlerbehandlung beim Hochladen von Daten

Hier wird überprüft, ob alle notwendigen Felder ausgefüllt sind, bevor der Upload gestartet wird. Sollte der Upload fehlschlagen, wird der Fehler im catch-Block abgefangen und der Benutzer benachrichtigt.

5.1.7.3. Allgemeine Fehlerbehandlung

Neben der spezifischen Fehlerbehandlung in den Funktionen erfolgt auch eine allgemeine Fehlerüberprüfung, die dafür sorgt, dass die Anwendung auch bei unerwarteten Problemen stabil bleibt. In allen Funktionen werden console.error-Befehle verwendet, um Fehler in der Konsole zu protokollieren.

5.2. Komponenten

Für das Administratorinterface wurden verschiedene Komponenten ausgewählt, um das System zu gestalten. Das Frontend nutzt React, um eine interaktive und einfach zu bedienende Oberfläche zu schaffen. Auf der Backend-Seite wird Express.js verwendet, um die Serverlogik zu verwalten, und MySQL, um die Daten zu speichern.

5.2.1. Frontend

Das Frontend des Administratorinterfaces von Event Insight wurde mit React entwickelt, um eine interaktive Oberfläche zu schaffen. Axios wird verwendet, um Daten vom Backend abzurufen, während React Router für eine Navigation zwischen Seiten sorgt.

5.2.1.1. React

React ist eine JavaScript-Bibliothek, die speziell für die Entwicklung von Benutzeroberflächen in Webanwendungen gemacht wurde. Diese Technologie wurde gewählt wegen ihrer Fähigkeit einzelne Komponenten zu erstellen, die bei Änderungen automatisch neu gerendert werden. Dies geschieht durch das „virtuelle DOM“, dort werden nur die Teile der Seite, die sich geändert haben, neu berechnet und aktualisiert. [2]

5.2.1.2. Axios

Axios ist eine HTTP-Client-Bibliothek, die es ermöglicht, HTTP-Anfragen zu senden und Daten abzurufen. Sie unterstützt alle gängigen HTTP-Methoden wie GET, POST, PUT und DELETE und funktioniert sowohl im Browser als auch in Node.js. Die Entscheidung für Axios fiel, da es eine einfache Möglichkeit bietet, Anfragen zu stellen und Fehler zu behandeln, was die Datenabfrage und -aktualisierung im Administratorinterface deutlich vereinfacht. [27]

5.2.1.3. React Router

React Router ermöglicht das Routing innerhalb der Anwendung, ohne dass die Seite komplett neu geladen werden muss. Es sorgt für die Navigation zwischen den verschiedenen Seiten im Administratorinterface. Diese Technologie wurde gewählt, weil sie eine einfache Lösung für das Routing bietet. Sie stellt auch sicher, dass die URL der Anwendung immer mit der angezeigten Seite übereinstimmt, ohne dass die Seiten vollständige aktualisiert werden muss. [28]

5.2.2. Backend

Das Backend des Administratorinterfaces von Event Insight nutzt Express.js für die Verwaltung von Routen. JWT (JSON Web Token) wird zur Authentifizierung verwendet, um den Zugang zum Interface zu sichern. Bcrypt.js sorgt für die Verschlüsselung von Passwörtern, um deren Sicherheit zu gewährleisten. Die Daten werden in einer MySQL-Datenbank gespeichert, die für die Sicherung der Daten zuständig ist.

5.2.2.1. Express.js

Express.js ist ein Web-Framework für Node.js, das das Erstellen von APIs vereinfacht. Express wurde gewählt, weil es eine einfache Möglichkeit bietet, Server-Routen zu definieren und HTTP-Anfragen zu bearbeiten. Im Backend wird Express verwendet, um Routen wie /events, /admins und /files zu definieren, die es ermöglichen, auf Renndaten und Admin-Informationen zuzugreifen. [4]

5.2.2.2. JWT

JSON Web Token (JWT) ist ein Standard, der ein sicheres Format für den Austausch von Informationen zwischen zwei Parteien ermöglicht. JWT wird häufig zur Authentifizierung in Webanwendungen eingesetzt. Im Administratorinterface von Event Insight wird JWT verwendet, um die Identität von administrierenden Personen zu validieren.

Ein JWT besteht aus drei Teilen:

- **Header:** Der Header enthält Informationen über den verwendeten Algorithmus zur Signatur (z.B. HMAC SHA256 oder RSA).
- **Payload:** Der Payload enthält die Informationen, die zwischen den Parteien ausgetauscht werden.
- **Signature:** Die Signature ist eine digitale Signatur, die mit dem Secret Key des Servers erzeugt wird, um sicherzustellen, dass der Token während der Übertragung nicht manipuliert wurde.

Das Administratorinterface nutzt JWT zur Authentifizierung, indem nach der Anmeldung ein Token generiert wird, das an den Client gesendet wird. Bei jeder nachfolgenden Anfrage an das Backend sendet der Client dieses Token mit, sodass das Backend den Token überprüfen kann. Ist der Token gültig, wird der Zugriff auf die geschützten Ressourcen gewährt. Dieser Ansatz verhindert, dass Benutzer*innen ihre Zugangsdaten bei jeder Anfrage erneut eingeben müssen, was die Benutzererfahrung verbessert. [29]

5.2.2.3. Bcrypt.js

Bcrypt.js wird verwendet, um Passwörter sicher zu hashen, bevor sie in der Datenbank gespeichert werden. Bcrypt funktioniert so:

- **Salt und Hashing:** Wenn ein Benutzer ein Passwort erstellt oder sich registriert, wird das Passwort zusammen mit einem zufällig generierten Salt gehasht. Das Salt stellt sicher, dass jedes Passwort, selbst wenn es identisch ist, einen einzigartigen Hash hat.
- **Speichern des Hashes:** Der resultierende Hash wird in der Datenbank gespeichert. Da der Hash keine Informationen über das Originalpasswort enthält, sind Passwörter in der Datenbank nicht im Klartext sichtbar.
- **Verifizierung:** Wenn der Benutzer sich anmeldet, wird das eingegebene Passwort zusammen mit dem gespeicherten Salt erneut gehasht. Dieser neue Hash wird mit dem in der Datenbank gespeicherten Hash verglichen. Wenn sie übereinstimmen, ist das Passwort korrekt.

Bcrypt wird im Backend verwendet, um die Passwörter der administrierenden Personen sicher zu speichern. Bei der Benutzeranmeldung wird das eingegebene Passwort gehasht und mit dem gespeicherten Hash verglichen. [30]

5.2.2.4. MySQL

MySQL ist das Datenbanksystem, das für die Speicherung und Verwaltung von den Daten eingesetzt wird. Über die MySQL-Datenbank erfolgt der Zugriff auf die gespeicherten Daten, die für die Anzeige und Verarbeitung im Frontend des Administratorinterfaces erforderlich sind. MySQL wurde gewählt, weil es ein gut unterstütztes System ist, das hohe Abfragegeschwindigkeiten und eine sichere Datenverwaltung bietet. [12] [31]

5.3. Implementierung/Realisierung

In diesem Abschnitt wird die Implementierung des Administratorinterfaces von Event Insight detailliert beschrieben. Die verschiedenen Code-Komponenten und deren Funktionalitäten werden erklärt. Jeder Schritt der Implementierung wird dabei sowohl aus technischer als auch aus praktischer Sicht betrachtet, um die Funktionsweise des gesamten Systems darzustellen.

5.3.1. Frontend

Die Frontend-Implementierung basiert auf React und umfasst mehrere Module: eine Übersicht der Events, ein Upload-Formular für Sponsorenbilder und eine Live-Map-Verwaltung, die es ermöglicht, GPX-Daten hochzuladen und Zwischenzeiten auf der Karte zu markieren.

In allen Komponenten wird der useEffect-Hook verwendet, um beim Laden der Seite automatisch die relevanten Daten von den API-Endpunkten abzurufen. Dies stellt sicher, dass die Benutzeroberfläche mit den neuesten Informationen aktualisiert wird, sobald die Komponente geladen wird. Die API-Anfragen werden mit axios durchgeführt, um die Daten in den Zustand zu laden und anzuzeigen.

Ein Beispielcode für den useEffect-Hook in einer React-Komponente, um Daten abzurufen:

```
const Events = () => {
  const [events, setEvents] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    // Datenabruf bei der Initialisierung der Komponente
    const fetchEvents = async () => {
      try {
        const response = await axios.get("http://localhost:3001/events");
        setEvents(response.data);
        setLoading(false);
      } catch (err) {
        setError("Fehler beim Abrufen der Events");
        setLoading(false);
      }
    };

    fetchEvents(); // Abrufen der Events-Daten der API
  }, []); // Leeres Array bedeutet, dass der Effekt nur einmal beim Laden
  // ausgeführt wird
};
```

Listing 17: Beispielcode für useEffect-Hook

Die AdminHome-Komponente stellt die Startseite für administrierende Personen dar, die Informationen zu einem bestimmten Event verwalten. Sie lädt und zeigt die grundlegenden Event-Daten an, wie den Eventnamen und das Datum, und ermöglicht es der administrierenden Personen, den Event-Text zu bearbeiten.

Im useEffect-Hook wird eine API-Anfrage an den Server gesendet, um die Event-Daten anhand der Event-ID zu laden.

Im JSX-Bereich wird die Benutzeroberfläche gerendert. Sobald die Event-Daten erfolgreich abgerufen wurden, wird der Name des Events (*event.eventname*) und das formatierte Datum (*event.formatted_date*) angezeigt. Diese Daten werden in der *event-info-admin*-Sektion innerhalb des *main*-Teils gerendert.

```
<div className="event-info-admin">
  <h1 className="event-title-admin">{event?.eventname}</h1>
  <p className="event-date">{event?.formatted_date}</p>
</div>
```

Listing 18: AdminHome Anzeige der Event-Daten

Ein Button unterhalb der Event-Informationen ermöglicht es der administrierenden Person, den Event-Text zu bearbeiten. Beim Klicken auf den Button wird die administrierende Person zu *editText.js* weitergeleitet, wo sie den Event-Text ändern kann.

```
<button className="submit-button" onClick={() => navigate(`/event/${id}/editText`)}>
  Event-Text bearbeiten
</button>
```

Listing 19: AdminHome Button zum Bearbeiten des Event-Text

5.3.1.1. AdminLiveMap.jsx

In der AdminLiveMap-Komponente wird eine Benutzeroberfläche zum Hochladen von Live-Maps und zum Einzeichnen von Zwischenzeiten für ein Event erstellt.

Der useEffect-Hook wird verwendet, um beim Laden der Komponente eine API-Anfrage auszuführen, die die GPX-Daten des Events abrufen.

Wenn eine Live-Map aus der Auswahl getroffen wird, sorgt der *handleSelectionChange*-Handler dafür, dass die Zwischenzeiten und Markierungen der ausgewählten Karte abgerufen und angezeigt werden. Eine zusätzliche API-Anfrage wird gestellt, um die Zwischenzeiten und Markierungen basierend auf der ausgewählten Karte zu laden.

```
const handleSelectionChange = async (selectedOption) => {
  if (!selectedOption) {
    setSelectedGpx(null);
    setIntermediateMarkers([]);
    setIntermediateTimes([]);
    return;
  }

  const newGpx = gpxData.find(data => data.value === selectedOption?.value);
  if (newGpx) {
    setSelectedGpx({
      id: newGpx.id,
      value: newGpx.value,
      label: newGpx.label
    });

    try {
      const response = await axios.get(`http://localhost:3001/files/intermediateTimes/getByMap?idlivemap=${newGpx.id}`);
      setIntermediateTimes(response.data.map(marker => ({
        latitude: parseFloat(marker.latitude),
        longitude: parseFloat(marker.longitude),
        position: [parseFloat(marker.latitude), parseFloat(marker.longitude)]
      })));

      setIntermediateMarkers(response.data.map(marker => ({
        latitude: parseFloat(marker.latitude),
        longitude: parseFloat(marker.longitude),
```

Listing 20: Funktion zum Wählen einer LiveMap in AdminLiveMap

Die Funktion *handleSubmit* wird aufgerufen, wenn die administrierende Person das Formular zum Hochladen der Live-Map absendet. Zuerst werden alle notwendigen Felder überprüft, um sicherzustellen, dass die Eingaben vollständig und korrekt sind. Anschließend wird die Live-Map mit einem POST-Request zusammen mit dem Kartennamen, der Startzeit und der Datei an das Backend gesendet.

```
const handleSubmit = async (e) => {
  e.preventDefault();
  if (!file || !mapName.trim() || !time) {
    alert("Bitte wähle eine Datei, gib einen Kartennamen und eine Zeit ein.");
    return;
  }

  const formData = new FormData();
  formData.append("LiveMap", file);
  formData.append("mapname", mapName);
  formData.append("eventid", id);
  formData.append("maptime", time);

  try {
    const response = await fetch("http://localhost:3001/files/livemap/upload", {
      method: "POST",
      body: formData,
    });
    const messageText = await response.text();
    alert(messageText);
  } catch (error) {
    console.error("Fehler:", error);
    alert("Fehler beim Hochladen der Live-Map");
  }
};
```

Listing 21: Funktion zum Hochladen von Live-Maps

Die Funktion *saveIntermediateTimes* speichert die Zwischenzeiten, die die administrierende Person auf der Live-Map gesetzt hat. Sie wird aufgerufen, wenn die administrierende Person die Zwischenzeiten speichern möchte. Diese Funktion sendet die Markierungsdaten mit einer weiteren POST-Anfrage an das Backend.

```
const saveIntermediateTimes = async () => {
  if (!selectedGpx?.id || intermediateMarkers.length === 0) {
    console.error("Kein gültiger Marker zum Speichern!");
    alert("Kein gültiger Marker zum Speichern!");
    return;
  }

  try {
    for (let marker of intermediateMarkers) {
      const markerData = {
        idlivemap: selectedGpx.id,
        latitude: marker.latitude,
        longitude: marker.longitude
      };

      await axios.post("http://localhost:3001/files/intermediateTimes/add", markerData);
    }
    alert("Zwischenzeiten erfolgreich gespeichert!");
  } catch (error) {
    console.error("Fehler beim Speichern der Zwischenzeiten:", error);
    alert("Fehler beim Speichern der Zwischenzeiten");
  }
};
```

Listing 22: Funktion zum Speichern der Zwischenzeiten

5.3.1.2. AdminResults.jsx

In der AdminResults-Komponente kann man API-Link für die Rennergebnisse hochladen und diesen mit einer bestimmten Live-Map verknüpfen können.

Im useEffect-Hook wird eine API-Anfrage an das Backend gesendet, um die verfügbaren Live-Maps für das aktuelle Event abzurufen. Diese Daten werden im Zustand *liveMaps* gespeichert.

Die Funktion *handleSubmit* wird aufgerufen, wenn die administrierende Person das Formular absendet. Zuerst wird geprüft, ob alle Felder korrekt ausgefüllt sind. Falls eines der Felder leer ist, wird eine Warnung ausgegeben. Der API-Link wird mit einem regulären Ausdruck überprüft, um sicherzustellen, dass der Link im richtigen Format vorliegt. Ist der Link korrekt, werden die Daten über eine POST-Anfrage an das Backend gesendet.

```
const apiLinkPattern = /^https:\/\/api\.raceresult\.com\/\d+\/[A-Za-z0-9]+$/;

const handleSubmit = async (e) => {
  e.preventDefault();
  if (!listName || !listlink || !selectedLiveMap) {
    alert("Bitte alle Felder ausfüllen.");
    return;
  }
  if (!apiLinkPattern.test(listlink)) {
    alert("Ungültiger API-Link. Der Link muss das Format  
'https://api.raceresult.com/...' haben.");
    return;
  }
  try {
    const response = await fetch("http://localhost:3001/files/re-  
sults/upload", {
      method: "POST",
      headers: {"Content-Type": "application/json"},
      body: JSON.stringify({listname: listName, listlink, eventid:  
id, idlivemap: selectedLiveMap}),
    });
    const data = await response.text();
    alert(data);
  } catch (error) {
    console.error("Fehler beim Hochladen:", error);
    alert("Fehler beim Hochladen der Daten.");
  }
};
```

Listing 23. Funktion zum Verknüpfen von API-Links

5.3.1.3. AdminSponsors.jsx

In der AdminSponsors-Komponente hat die administrierende Person die Möglichkeit, Sponsor-Bilder hochzuladen, die Anzeige von Sponsoren zu steuern und die Änderungen zu speichern.

Der useEffect-Hook wird verwendet, um die Sponsorenliste aus der Datenbank abzurufen, sobald die Komponente geladen wird. Die fetchSponsors-Funktion führt eine GET-Anfrage an den Server aus und speichert die abgerufenen Daten in sponsorList.

Die *handleSubmit*-Funktion wird aufgerufen, wenn die administrierende Person probiert ein neues Sponsor-Bild hochzuladen. Es wird überprüft, ob alle erforderlichen Felder ausgefüllt sind. Anschließend wird die Datei zusammen mit dem Sponsornamen an den Server gesendet.

```
const handleSubmit = async (e) => {
  e.preventDefault();
  if (!file || !sponsorName.trim()) {
    alert("Bitte alle Felder ausfüllen.");
    return;
  }

  const formData = new FormData();
  formData.append("sponsorFile", file);
  formData.append("sponsorname", sponsorName);
  formData.append("eventid", id);

  try {
    const response = await fetch("http://localhost:3001/files/sponsors/upload",
    {
      method: "POST",
      body: formData,
    });
    const result = await response.text();
    alert(result);
    setFile(null);
    setSponsorName("");
    fetchSponsors();
  } catch {
    alert("Fehler beim Hochladen der Datei.");
  }
};
```

Listing 24: Funktion zum Hochladen von Sponsorbildern

Ein weiterer Teil der Komponente ist die *handleSponsorToggle*-Funktion. Die wird verwendet, um die Anzeige eines Sponsors zu aktivieren oder z

u deaktivieren. Die administrierende Person kann dies tun, wenn er das Kästchen neben einem Sponsor auswählt. Wenn ein Sponsor ausgewählt wird, wird sein *is_displayed*-Status im *sponsorList*-State geändert.

```
const handleSponsorToggle = (sponsorId) => {
  setSponsorList((prevList) =>
    prevList.map((sponsor) =>
      sponsor.idSponsors === sponsorId
        ? {...sponsor, is_displayed: sponsor.is_displayed === 1 ? 0
: 1}
        : sponsor
    )
  );
};
```

Listing 25: Funktion zur Auswahl des Anzeigestatus des Sponsorbilder

Nachdem die gewünschten Änderungen vorgenommen wurden, kann die administrierende Person die Änderungen speichern. Dies passiert in der *handleSaveDisplayedSponsors*-Funktion, die die IDs der Sponsoren, die angezeigt werden sollen, extrahiert und in einem POST-Request an den Server übermittelt.

```
const handleSaveDisplayedSponsors = async () => {
  try {
    const displayedSponsors = sponsorList
      .filter((sponsor) => sponsor.is_displayed === 1)
      .map((sponsor) => sponsor.idSponsors);

    const response = await fetch(`http://localhost:3001/files/spon-
sors/updateDisplay`, {
      method: "POST",
      headers: {"Content-Type": "application/json"},
      body: JSON.stringify({eventid: id, displayedSponsors}),
    });
    const result = await response.text();
    alert(result);
  } catch {
    alert("Fehler beim Speichern der Sponsoranzeige.");
  }
};
```

Listing 26; Funktion zur Bestätigung des Anzeigestatus der Sponsorbilder

5.3.1.4. EditText.jsx

In der EditText-Komponente können administrierende Personen den Text eines Events bearbeiten.

Im useEffect-Hook wird eine API-Anfrage an den Server gesendet, um die Event-Daten und den gespeicherten Text abzurufen. Die erste Anfrage ruft die Event-Daten ab und speichert sie im event-State, während die zweite Anfrage den Event-Text lädt und im eventText-State speichert.

Die *handleSubmit*-Funktion wird aufgerufen, wenn die administrierende Person den bearbeiteten Text speichern möchte. Zuerst wird sichergestellt, dass der überhaupt ein Text vorhanden ist, bevor die Daten an das Backend gesendet werden. Die Daten werden als JSON über eine POST-Anfrage gesendet.

```
const handleSubmit = async (e) => {
  e.preventDefault();
  try {
    const response = await axios.post("http://localhost:3001/files/text/upload", {
      idtext: id,
      text: eventText,
      eventid: id
    });
    alert(response.data);
  } catch (error) {
    console.error("Error uploading text:", error);
    alert("Error uploading text");
  }
};
```

Listing 27: Funktion zum Speichern des Event-Text

Der ReactQuill-Editor wird verwendet, um den Event-Text anzuzeigen und zu bearbeiten. Der Editor zeigt den aktuellen Text im *eventText*-State an, und jede Änderung wird in diesem State gespeichert. Mit der *modules*-Konfiguration wird der Funktionsumfang des Editors definiert, z. B. die Formatierung von Text, das Hinzufügen von Links oder Bildern und die Auswahl der Schriftgröße.

```
const modules = {
  toolbar: [
    [{ 'header': '1' }, { 'header': '2' }, { 'font': [] }],
    [{ size: [] }],
    ['bold', 'italic', 'underline', 'strike', 'blockquote'],
    [{ 'list': 'ordered' }, { 'list': 'bullet' }, { 'indent': '-1' }, { 'indent': '+1' }],
    ['link'],
    ['clean']
  ],
};
```

Listing 28: Konfiguration des ReactQuill-Editor in editText

5.3.1.5. AdminEdit.jsx

In der AdminEdit-Komponente ist die Bearbeitung von Event-Daten möglich. Das heißt die administrierende Person ist in der Lage, Event-Informationen wie Name, Datum, Ort, Adresse, Listenname und Listenlink zu aktualisieren sowie die zugehörigen Karten und Sponsoren zu bearbeiten.

Der useEffect-Hook wird verwendet, um beim Laden der Komponente eine API-Anfrage auszuführen, die die Event-Daten sowie zugehörige Informationen zu Karten, Listen und Sponsoren für das Event abrufen.

Die *handleUpdateClick*-Funktion öffnet ein Bearbeitungsfenster für das ausgewählte Feld, sodass die administrierende Person den Wert anpassen kann.

```
const handleUpdateClick = (field, value) => {
  setCurrentField(field);
  setCurrentValue(field === 'eventdate' ? new
Date(value).toISOString().split('T')[0] : value);
  setIsEditWindowOpen(true);
};
```

Listing 29: Funktion zur Initialisierung der Änderung von Event-Daten

Wenn die administrierende Person auf "Aktualisieren" klickt, öffnet sich ein Bearbeitungsfenster. Der neue Wert wird gespeichert, indem die *handleSave*-Funktion ausgeführt wird. Diese passiert, wenn die administrierende Person auf „Speichern“ klickt.

```
const handleSave = async () => {
  if (!currentValue) {
    alert("Das Feld darf nicht leer sein");
    return;
  }
  try {
    const formattedValue = currentField === 'eventdate' ? new Date(currentValue).toISOString().split('T')[0] : currentValue;
    const updateUrl = currentField.startsWith('map') ? 'updateMap' :
currentField.startsWith('list') ? 'updateList' : 'updateEvent';
    const updateId = currentField.startsWith('map') ? selected-
Map.idlivemap : currentField.startsWith('list') ? selectedList.id : id;
    const endpoint = currentField.startsWith('map') || current-
Field.startsWith('list') ? `http://localhost:3001/files/livemap/${up-
dateUrl}` : `http://localhost:3001/events/${updateUrl}`;
    await axios.put(endpoint, {id: updateId, field: currentField,
value: formattedValue});
    if (currentField.startsWith('map')) setSelectedMap(prev =>
({...prev, [currentField]: formattedValue}));
    else if (currentField.startsWith('list')) setSelectedList(prev =>
({...prev, [currentField]: formattedValue}));
    else setEvent(prev => ({...prev, [currentField]: formattedValue}));
    setIsEditWindowOpen(false);
  } catch (error) {
    setError("Fehler beim Aktualisieren der Daten");
  }
};
```

Listing 30: Funktion zum Speichern der Änderungen von Event-Daten

Der *handleDelete*-Handler fragt die administrierende Person zur Bestätigung, ob ein Element gelöscht werden soll. Wenn die Bestätigung erfolgt, wird eine DELETE-Anfrage an das Backend gesendet, um das Element zu löschen.

```
const handleDelete = async (type, id) => {
  const confirmDelete = window.confirm(`Möchten Sie diesen ${type} wirklich löschen?`);
  if (!confirmDelete) return;

  try {
    const endpoint = type === 'Map' ? 'http://localhost:3001/files/livemap/delete' : 'http://localhost:3001/files/sponsors/delete';
    await axios.delete(endpoint, {data: {id}});
    if (type === 'Map') {
      setMaps(prev => prev.filter(item => item.idlivemap !== id));
      setSelectedMap(null); // Deselect the map
    } else {
      setSponsors(prev => prev.filter(item => item.idSponsors !== id));
    }
  } catch (error) {
    setError(`Fehler beim Löschen der ${type}`);
  }
};
```

Listing 31: Funktion zum Löschen von einzelnen Event-Daten

Die Funktion *handleDeleteEvent* ermöglicht das Löschen eines gesamten Events. Hier wird auch eine Bestätigung angefordert, bevor das Event gelöscht wird. Nach erfolgreicher Löschung wird die administrierende Person zur *LandingPage.jsx* weitergeleitet.

```
const handleDeleteEvent = async () => {
  const confirmDelete = window.confirm("Möchten Sie dieses Event wirklich löschen?");
  if (!confirmDelete) return;

  try {
    await axios.delete(`http://localhost:3001/events/deleteEvent`, {data: {id}});
    navigate('/'); // Weiterleitung zu Landingpage.jsx
  } catch (error) {
    setError("Fehler beim Löschen des Events");
  }
};
```

Listing 32: Funktion zum Löschen von Events

5.3.2. Backend

Das Backend ist in Express.js implementiert und bietet die Routen zur Verwaltung von Events, Admins, API-Links, Live-Maps, Zwischenzeiten, Sponsoren, Text. Eine Route besteht immer aus einer HTTP-Methode (GET, POST, PUT, DELETE) und einer URL, die die Ressource angibt, die bearbeitet wird. Ebenso spielen die Konfiguration und Authentifizierung eine große Rolle.

Ein Beispiel für eine Route im Backend könnte so aussehen:

```
router.get('/livemap', (req, res) => {
  const eventId = req.query.eventid;
  if (!eventId) return errorHandler.handleValidationError('Event ID is
required.', res);

  connection.query('SELECT idlivemap, mapname, mappath, maptime FROM
LiveMap WHERE eventid = ?', [eventId], (err, results) => {
    if (err) return errorHandler.handleDatabaseError(err, res);
    res.status(200).json(results);
  });
});
```

Listing 33: Beispiel-Route im Backend

5.3.2.1. admin.js

Die *admin.js* Datei enthält Routen, die die Verwaltung und Authentifizierung von administrierenden Personen ermöglichen. Dazu gehört die Erstellung neuer administrierender Personen, das Login, die Überprüfung des Eventzugriffs sowie das Abrufen von Benutzerdaten.

- **GET /:** Diese Route gibt eine Liste aller administrierender Personen zurück, die in der Datenbank gespeichert sind. Sie führt eine SQL-Abfrage aus und gibt die administrierenden Personen als JSON zurück.
- **POST /createAdmin:** Mit dieser Route wird eine neuer administrierende Person erstellt. Dabei wird der Benutzername überprüft, das Passwort gehasht und die neuen Daten in die Datenbank eingefügt.
- **POST /login:** Administrierender Personen können sich über diese Route einloggen. Wenn der Benutzername und das Passwort korrekt sind, wird ein JWT-Token zur Authentifizierung generiert und zurückgegeben.
- **POST /checkEventAccess:** Diese Route überprüft, ob eine administrierende Person Zugriff auf ein bestimmtes Event hat. Sie vergleicht die Admin-ID und Event-ID mit den Daten in der Tabelle *AdminEventAccess*.
- **GET /me:** Diese Route liefert Informationen über die aktuell angemeldete administrierende Person. Das JWT-Token wird überprüft und die entsprechenden Benutzerdaten werden zurückgegeben.

5.3.2.2. events.js

Die *events.js*-Datei stellt Routen zur Verfügung, die für die Verwaltung von Events verantwortlich sind. Es gibt Routen zum Abrufen, Hinzufügen, Aktualisieren und Löschen von Events. Die Routen beinhalten auch SQL-Abfragen, um sicherzustellen, dass nur berechtigte administrierende Personen auf bestimmte Event-Daten zugreifen können.

- **GET /:** Diese Route gibt eine Liste aller Events zurück, die in der Datenbank gespeichert sind, einschließlich des Eventnamens und -datums.
- **POST /add:** Mit dieser Route können neue Events zur Datenbank hinzugefügt werden. Sie erwartet Event-Daten und eine Admin-ID zur Registrierung des Zugriffs auf das Event.
- **PUT /updateEvent:** Damit kann ein bestehendes Event aktualisiert werden. Ein bestimmtes Feld des Events (z. B. Eventname oder Datum) kann geändert werden.
- **DELETE /deleteEvent:** Diese Route löscht ein Event und alle zugehörigen Daten wie Event-Texte, Sponsoren und Zwischenzeiten aus der Datenbank.
- **GET /:id:** Diese Route ruft die Details eines bestimmten Events anhand seiner Event-ID ab.

5.3.2.3. files.js

Die *files.js*-Datei beinhaltet mehrere Routen, um verschiedene Dateien im Zusammenhang mit einem Event zu verwalten. Es gibt Routen zum Verwalten von Live-Maps, Sponsorenbilder, Zwischenzeiten, Event-Texte und API-Links

5.3.2.3.1. Live-Maps

Die Routen zur Verwaltung von Live-Maps umfassen das Abrufen von Karten, das Hochladen von GPX-Dateien, das Aktualisieren von Kartendaten, das Löschen von Karten und das Abrufen von Details einer Karte.

- **GET /livemap:** Diese Route ruft eine Liste von Karten für ein Event ab. Sie gibt die ID, den Namen, den Pfad und den Zeitpunkt der Karten zurück.
- **POST /livemap/upload:** Über diese Route können GPX-Dateien für Live-Maps hochgeladen werden. Sie erwartet die Event-ID, den Kartennamen und die Startzeit.
- **PUT /livemap/update:** Diese Route ermöglicht das Aktualisieren eines bestimmten Felds einer Live-Map. Die Route erwartet die ID der Karte, das zu aktualisierende Feld und den neuen Wert.
- **DELETE /livemap/delete:** Diese Route löscht eine Live-Map zusammen mit ihren zugehörigen Daten. Die zugehörigen Zwischenzeiten und API-Links werden ebenfalls entfernt.
- **GET /livemap/details/:eventid:** Diese Route gibt die Details einer Live-Map für ein Event zurück, einschließlich des Kartennamens und der Startzeit. Sie benötigt die Event-ID als Parameter.

5.3.2.3.2. API-Links

Die Routen in diesem Abschnitt ermöglichen das Verwalten von API-Links, die mit Live-Maps verbunden sind.

- **POST /results/upload:** Ermöglicht das Hochladen von API-Links und das Verknüpfen dieser mit einer Live-Map.
- **GET /results/fetch:** Holt das Rennergebnis für ein Event basierend auf der Event-ID. Sie gibt den Listennamen und den Link zum Ergebnis zurück.
- **GET /results/fetchFromMap:** Diese Route holt das Rennergebnis basierend auf der Live-Map-ID. Sie gibt den dazugehörigen API-Link zurück.

5.3.2.3.3. Sponsoren

Diese Routen ermöglichen das Hochladen von Sponsorenbildern, das Abrufen von Sponsoren und das Löschen von Sponsoren.

- **DELETE /sponsors/delete:** Löscht einen Sponsor aus der Datenbank.
- **POST /sponsors/upload:** Ermöglicht das Hochladen von Sponsorenbildern. Das Bild wird gespeichert, und die Daten werden in der Datenbank hinzugefügt.
- **GET /sponsors:** Holt alle Sponsoren für ein bestimmtes Event ab, basierend auf der Event-ID. Es werden Sponsor-Namen, Pfade der Bilder und deren Anzeigeeinstellungen zurückgegeben.
- **POST /sponsors/updateDisplay:** Diese Route ermöglicht das Aktualisieren der Anzeigeeinstellungen für Sponsoren eines Events. Hier können Sponsoren auf "anzeigen" oder "nicht anzeigen" gesetzt werden.

5.3.2.3.4. Zwischenzeiten

Die Routen für Zwischenzeiten ermöglichen das Hinzufügen, Abrufen und Löschen von Zwischenzeiten für Live-Maps.

- **POST /intermediateTimes/add:** Diese Route fügt Zwischenzeiten zu einer bestimmten Live-Map hinzu.
- **GET /intermediateTimes/getByMap:** Holt alle Zwischenzeiten für eine bestimmte Live-Map ab.
- **DELETE /intermediateTimes/delete:** Löscht eine Zwischenzeit anhand ihrer ID.

5.3.2.3.5. Text

Diese Routen ermöglichen das Abrufen und Hochladen von Texten, die mit einem Event verbunden sind.

- **GET /text/fetch:** Ruft einen Event-Text basierend auf der Text-ID ab.
- **POST /text/upload:** Diese Route wird verwendet, um Textdaten hochzuladen oder bestehende Event-Texte zu aktualisieren.

5.3.2.4. Backendkonfiguration und Authentifizierung

Dieses Kapitel beschreibt die Konfiguration des Backend. Es umfasst die Verbindung zur MySQL-Datenbank, die Einrichtung des Express-Servers zur Verwaltung von API-Routen, die Implementierung der Authentifizierung und die Fehlerbehandlung der Routen

5.3.2.4.1. db.js

In *db.js* wird eine Verbindung zur MySQL-Datenbank hergestellt. Das sorgt dafür, dass die Anwendung dazu fähig ist, Abfragen und Anfragen an die Datenbank zu senden

```
const connection = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: '',
  database: 'eventinsight'
});
```

Listing 34: Etablierung der Verbindungsdaten für die Datenbank

Die Verbindung zur MySQL-Datenbank wird hier hergestellt. Bei einem Fehler gibt der Code eine Fehlermeldung in der Konsole aus. Wenn die Verbindung erfolgreich ist, wird dies ebenfalls in der Konsole angezeigt.

```
connection.connect((err) => {
  if (err) {
    console.error('Fehler beim Verbinden zur Datenbank:', err.stack);
    return;
  }
  console.log('Erfolgreich verbunden mit der Datenbank');
});
```

Listing 35: Funktion zur Verbindungsherstellung zur Datenbank

5.3.2.4.2. server.js

server.js ist der Einstiegspunkt für die Node.js-Anwendung. In diesem Code wird der Express-Server eingerichtet, und die verschiedenen Routen werden konfiguriert, um Anfragen vom Client zu bearbeiten.

Die in der *routes.js* definierten Routen werden mit der *app.use()*-Methode eingebunden, so dass alle Anfragen, die mit einem Pfad übereinstimmen, an den entsprechenden Handler weitergeleitet werden. Der Server wird auf Port 3001 gestartet und gibt eine Bestätigung aus, sobald er läuft

```
const routes = require('./routes/routes');
app.use('/', routes);

app.listen(3001, () => {
  console.log('Server läuft auf http://localhost:3001');
});
```

Listing 36: Einbinden der Routen und Starten des Servers auf Port 3001

```
const eventsRoutes = require('./events');
const adminsRoutes = require('./admins');
const filesRoutes = require('./files');
const authRoutes = require('./authRoutes');

router.use('/events', eventsRoutes);
router.use('/admins', adminsRoutes);
router.use('/files', filesRoutes);
router.use('/auth', authRoutes);
```

Listing 37: Ausschnitt der Routendeklaration aus *routes.js*

5.3.2.4.3. errorHandler.js

errorHandler.js stellt Funktionen bereit, die Fehler im Backend behandeln. Wenn während einer Anfrage ein Fehler auftritt, sorgt dieser dafür, dass eine geeignete Fehlermeldung gesendet wird.

Diese Funktion wird aufgerufen, wenn ein Fehler bei einer Datenbankabfrage auftritt. Sie gibt den Fehler in der Konsole aus und sendet eine Fehlermeldung mit dem Statuscode 500.

```
handleDatabaseError = (err, res) => {
  console.error('Datenbankfehler:', err);
  res.status(500).json({error: 'Datenbankfehler aufgetreten'});
};
```

Listing 38: Funktion zur Behandlung von Datenbankfehler

Diese Funktion wird verwendet, wenn ein Fehler aufgrund ungültiger Eingabedaten auftritt. Sie gibt den Fehler in der Konsole aus und sendet eine Fehlermeldung mit dem Statuscode 400

```
handleValidationError = (message, res) => {
  console.warn('Validierungsfehler:', message);
  res.status(400).json({error: message});
};
```

Listing 39: Funktion zur Behandlung von ungültigen Eingabedaten

5.3.2.4.4. authenticateToken.js

authenticateToken.js enthält eine Middleware, die bei jeder Anfrage überprüft, ob der Benutzer authentifiziert ist, indem das JWT-Token im Authorization-Header überprüft wird. Diese Middleware wird auf geschützte Routen angewendet, um sicherzustellen, dass nur autorisierte Benutzer auf bestimmte Ressourcen zugreifen können. Das JWT-Token wird aus dem Authorization-Header extrahiert und mit der `jwt.verify()`-Methode überprüft. Falls das Token nicht vorhanden oder ungültig ist, wird der Zugriff verweigert und eine Fehlermeldung zurückgegeben. Ist das Token gültig, werden die Benutzerdaten extrahiert und der Zugriff auf die angeforderte Route gewährt.

```
const jwt = require('jsonwebtoken');
const secretKey = 'b8d1f7b9-7f1c-4b99-ae82-7a2d42e0d3d8';

const authenticateToken = (req, res, next) => {
  const token = req.header('Authorization')?.split(' ')[1];
  if (!token) return res.status(401).json({message: 'Kein Token gefunden'});

  jwt.verify(token, secretKey, (err, decoded) => {
    if (err) return res.status(403).json({message: 'Ungültiges Token'});
    req.user = decoded;
    next();
  });
};
```

Listing 40: Funktion zur Authentifizierung des JWT

5.3.2.4.5. authRoutes.js

authRoutes.js enthält eine Route zur Validierung von JWT-Token.

- **GET /validateToken**

Diese Route überprüft, ob ein gültiges JWT vorhanden ist und ob es korrekt ist. Es wird der Authorization-Header abgerufen. Falls der Header fehlt oder das Token nicht im richtigen Format vorhanden ist, wird eine Fehlermeldung zurückgegeben. Ist das Token jedoch vorhanden, wird es mit der `jwt.verify()`-Methode gegen den geheimen Schlüssel überprüft. Wenn das Token ungültig ist, gibt es eine Fehlermeldung zurück. Bei erfolgreicher Verifizierung des Tokens wird der darin enthaltene Benutzername der administrierenden Person extrahiert und als Antwort zurückgegeben.

```
router.get('/validateToken', (req, res) => {
  const authHeader = req.headers['authorization'];
  if (!authHeader) return errorHandler.handleValidationError('Authoriza-
tion header is missing', res);

  const token = authHeader.split(' ')[1];
  if (!token) return errorHandler.handleValidationError('Token is miss-
ing', res);

  jwt.verify(token, secretKey, (err, decoded) => {
    if (err) return errorHandler.handleValidationError('Invalid token',
res);
    res.status(200).json({username: decoded.username});
  });
});
```

Listing 41: Route zur Überprüfung der Gültigkeit eines JWT

5.3.2.4.6. AuthContext.js

AuthContext.js ist verantwortlich für das Verwalten des Authentifizierungsstatus. Er sorgt dafür, dass alle Komponenten im Frontend den Authentifizierungsstatus einsehen können und bei Bedarf darauf zugreifen können.

Der Code beginnt mit der Erstellung eines Authentifizierungs-Kontexts. `createContext()` wird verwendet, um den Authentifizierungszustand der Anwendung zu speichern und ihn allen Komponenten zugänglich zu machen, die diesen Status benötigen. Der `AuthContext` stellt sicher, dass alle untergeordneten Komponenten auf den aktuellen Anmeldestatus zugreifen können, ohne ihn manuell in jeder Komponente weiterzugeben.

```
import { createContext, useState } from 'react';  
const AuthContext = createContext();
```

In der `AuthProvider`-Komponente wird der Zustand für die Authentifizierung (`isAuthenticated`) und die Funktionen `login` und `logout` verwaltet. Der `useState`-Hook wird genutzt, um den Authentifizierungsstatus in der gesamten Anwendung zu steuern. Die `login`- und `logout`-Funktionen ermöglichen es, den Authentifizierungsstatus zu ändern, je nachdem, ob der Benutzer eingeloggt oder ausgeloggt ist.

- **login** setzt `isAuthenticated` auf `true`, was bedeutet, dass der Benutzer erfolgreich eingeloggt wurde.
- **logout** setzt `isAuthenticated` auf `false`, was darauf hinweist, dass der Benutzer ausgeloggt wurde.

```
export const AuthProvider = ({ children }) => {  
  const [isAuthenticated, setIsAuthenticated] = useState(false);  
  
  const login = () => {  
    setIsAuthenticated(true);  
  };  
  const logout = () => {  
    setIsAuthenticated(false);  
  };  
  return (  
    <AuthContext.Provider value={{ isAuthenticated, login, logout }}>  
      {children}  
    </AuthContext.Provider>  
  );  
};
```

Mit dieser `useAuth`-Funktion können andere Komponenten innerhalb der Anwendung auf den Authentifizierungsstatus zugreifen, ohne direkt auf den Kontext zugreifen zu müssen. Sie verwenden einfach den `useAuth`-Hook, um den aktuellen Anmeldestatus sowie die Funktionen zum Anmelden und Abmelden zu verwenden.

```
export const useAuth = () => {  
  return useContext(AuthContext);  
};
```

Listing 42: Funktion zum Prüfen des Anmeldestatus

5.3.2.4.7. PrivateRoute.js

PrivateRoute.js sorgt dafür, dass nur authentifizierte Benutzer auf bestimmte Routen zugreifen können. Wenn der Benutzer nicht eingeloggt ist oder kein gültiges Token besitzt, wird er automatisch zur Login-Seite weitergeleitet.

```
import React from 'react';
import { Navigate } from 'react-router-dom';

const PrivateRoute = ({ element }) => {
  const token = localStorage.getItem('token');
  const LOGIN_PATH = '/adminLogin';

  return token ? element : <Navigate to={LOGIN_PATH} />;
};

export default PrivateRoute;
```

Listing 43: Funktion zum Schützen von Routen

In diesem Code wird überprüft, ob ein Token im *localStorage* vorhanden ist, was darauf hindeutet, dass der Benutzer eingeloggt ist.

- **token:** Das aus dem *localStorage* wird abgerufen, das beim Login des Benutzers gespeichert wurde. Wenn ein Token vorhanden ist, gilt der Benutzer als authentifziert.
- **LOGIN_PATH:** Die Route, zu der nicht authentifizierte Benutzer weitergeleitet werden, wenn sie versuchen, auf eine private Route zuzugreifen (*/adminLogin*).

Die *PrivateRoute*-Komponente akzeptiert eine Prop namens *element*, die die zu schützende Route oder Komponente darstellt. Wenn ein Token vorhanden ist, wird das *element* (also die geschützte Route) angezeigt. Wenn kein Token vorhanden ist (der Benutzer ist nicht authentifziert), wird der Benutzer zur Login-Seite (*/adminLogin*) umgeleitet.

5.3.2.5. Datenbank

Die Datenbank von EventInsight dient als Speicherort für alle Daten der Anwendung. Die Datenbank ermöglicht die Speicherung von Informationen wie Events, administrierende Personen, Zwischenzeiten, Sponsoren und Event-Texte.

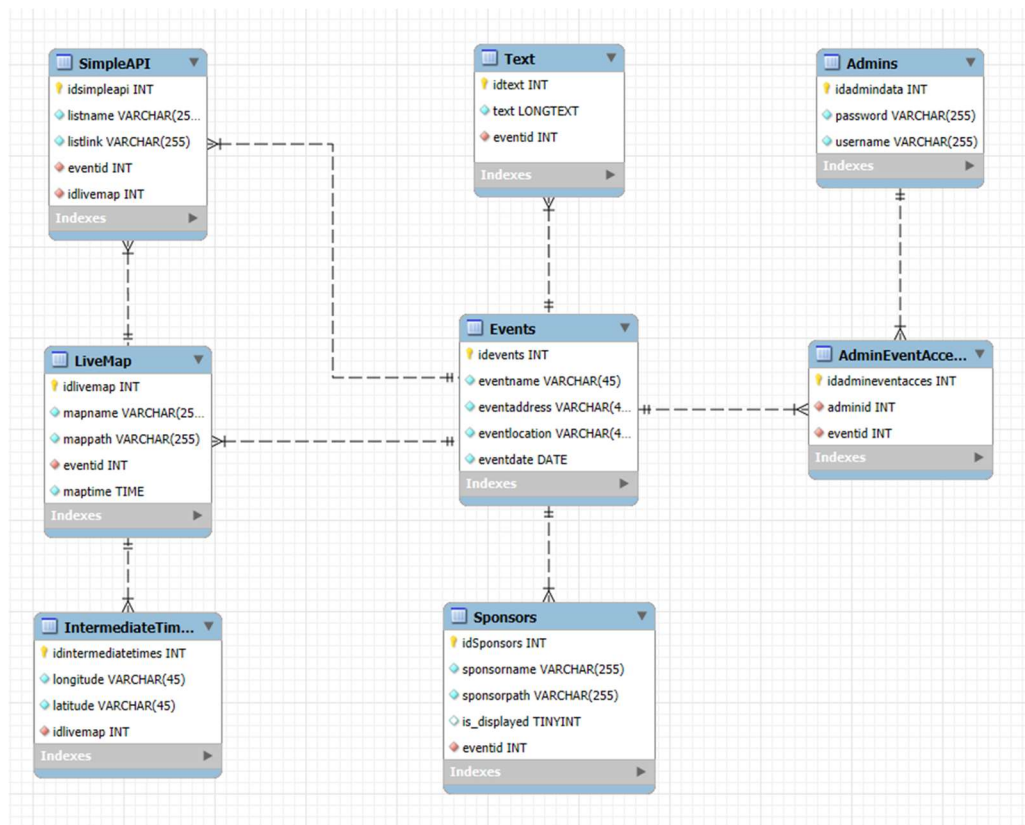


Abbildung 24: EE-Diagramm der Event Insight Datenbank

5.3.2.5.1. Admins-Tabelle

Die Admins-Tabelle speichert alle relevanten Informationen zu den administrierende Personen.

- **idadmindata:** ID der administrierenden Person (Primärschlüssel).
- **username:** Der Benutzername der administrierenden Person, der zur Anmeldung verwendet wird.
- **password:** Das Passwort der administrierendem Person, gespeichert in gehashter Form.

5.3.2.5.2. Events-Tabelle

Die Events-Tabelle enthält alle Informationen zu den verschiedenen Events, die verwaltet werden.

idevents: ID des Events (Primärschlüssel).

eventname: Der Name des Events.

eventdate: Das Datum und die Uhrzeit des Events.

eventlocation: Der Ort, an dem das Event stattfindet.

eventaddress: Die Adresse des Events.

5.3.2.5.3. LiveMap-Tabelle

Die LiveMap-Tabelle speichert alle Informationen zu den Live-Maps.

- **idlivemap:** ID der Live-Map (Primärschlüssel).
- **mapname:** Der Name der Live-Map.
- **mappath:** Der Dateipfad zur Live-Map-Datei.
- **eventid:** Die ID des Events zu dem die Karte gehört.
- **maptime:** Der Zeitpunkt, zu dem der Lauf stattfinden wird.

5.3.2.5.4. Sponsors-Tabelle

Die Sponsors-Tabelle enthält Informationen über die Sponsoren eines Events.

- **idsponsors:** ID des Sponsors (Primärschlüssel).
- **sponsorname:** Der Name des Sponsors.
- **sponsorpath:** Der Dateipfad zum Sponsor-Bild.
- **eventid:** Die ID des Events, auf das der Sponsor verweist.
- **is_displayed:** Ein Flag, das angibt, ob das Sponsor-Bild auf der Event-Seite angezeigt werden soll oder nicht.

5.3.2.5.5. IntermediateTimes-Tabelle

Die IntermediateTimes-Tabelle speichert alle Zwischenzeiten, die von administrierenden Personen während des Events auf der Live-Map eingezeichnet werden.

- **idintermediatetimes:** ID der Zwischenzeit (Primärschlüssel).
- **latitude:** Der Breitengrad der Zwischenzeit.
- **longitude:** Der Längengrad der Zwischenzeit.
- **idlivemap:** Die ID der Live-Map, auf der die Zwischenzeit markiert wurde.

5.3.2.5.6. SimpleAPI-Tabelle

Die SimpleAPI-Tabelle enthält API-Links, die mit Renndaten für bestimmte Events verknüpft sind.

- **id:** ID des API-Links (Primärschlüssel).
- **listname:** Der Name der Liste.
- **listlink:** Der API-Link, der auf die Renndaten verweist.
- **eventid:** Die ID des zugehörigen Events.
- **idlivemap:** Die ID der zugehörigen Live-Map, mit der die Liste verbunden ist.

5.3.2.5.7. AdminEventAccess-Tabelle

Die AdminEventAccess-Tabelle speichert die Verknüpfung zwischen administrierenden Personen und Events, um den Zugriff der administrierenden Personen auf bestimmte Events festzulegen.

- **adminid:** Die ID der administrierenden Person
- **eventid:** Die ID des Events, auf das die administrierende Person Zugriff hat

5.3.2.5.8. Text-Tabelle

Die Text-Tabelle speichert alle Texte, die zu einem bestimmten Event gehören.

- **idtext:** ID des Textes (Primärschlüssel).
- **text:** Der Text, der gespeichert werden soll.
- **eventid:** Die ID des zugehörigen Events, auf das der Text verweist.

6. Anhang

6.1. Projektmanagement

6.1.1. Aufgabenstellung des Gesamtprojekts

Maximilian Wolf Leopold Lindner beschäftigt sich mit der Evaluierung und Auswahl geeigneter Frameworks zur Entwicklung der Webapplikation, implementiert das Interface für die Administration, welches den Veranstaltenden ermöglicht, die Renndaten benutzerfreundlich zu erfassen, wichtige Informationen für Zusehende bereitzustellen und die Rennstrecke zu integrieren.

Lukas Peter Sumann implementiert das Interface für die Zusehenden und die Integration der Schnittstelle SimpleAPI von race result AG. Zusätzlich zur Abfrage der Daten werden Funktionen entwickelt, die für die dynamische Visualisierung der Zeiten und Informationen der Teilnehmenden in Echtzeit benötigt werden. Der aktuelle Rennverlauf soll auf Basis der Renndaten angezeigt und eine präzise Berechnung und Darstellung der Positionen der Teilnehmenden im Userinterface soll ermöglicht werden.

6.1.2. Scrum-Projektplan

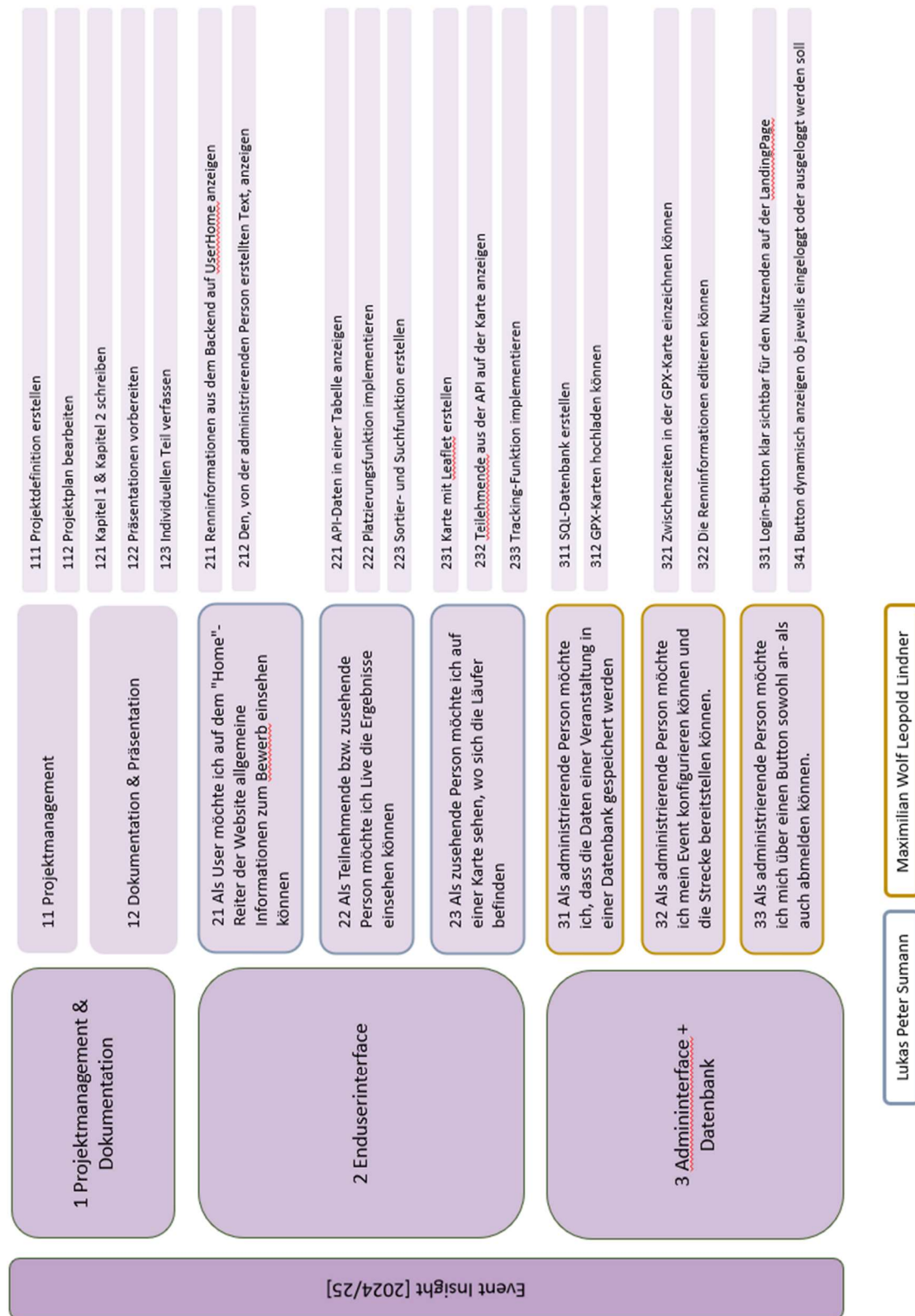


Abbildung 25: Scrum Projektplan mit Tasks

6.2. Inbetriebnahme

Zunächst muss die MySQL-Datenbank für Event Insight eingerichtet werden, indem zuallererst ein Docker Container gestartet wird. Die benötigte Struktur ist als MySQL-Dump-Datei im Git-Repository von Event Insight im *misc*-Ordner enthalten. Nach dem Importieren der Datei innerhalb der *MySQL Workbench*, wird eine neue Datenbank namens *eventinsight* erstellt und mit den Daten aus der MySQL-Dump-Datei aus dem Repository gefüllt.

Nach der erfolgreichen Einrichtung der MySQL-Datenbank wird das Event Insight-Projekt vom Git-Repository heruntergeladen. Anschließend werden die Abhängigkeiten mit *npm install* installiert, um alle in der *package.json*-Datei angegebenen Bibliotheken herunterzuladen. Hierbei ist zu beachten, dass es für Frontend und Backend separate *package.json*-Dateien gibt. Daher muss man bei beiden die Abhängigkeiten installieren und vor dem Eingeben des „*npm install*“-Befehls in das richtige Verzeichnis navigieren.

Das Backend und Frontend müssen separat gestartet werden. Zunächst wird das Backend mit „*npm run startBackend*“ gestartet, danach das Frontend mit „*npm run startWeb*“.

6.3. Besprechungsprotokolle



am Tag, Datum
Dienstag, 8.10.2024

Ort/Raum
HTL Mössingerstrasse/W101

Teilnehmer und Unterrichtete		ERGEBNISPROTOKOLL ZUR BESPRECHUNG	Eingangs- vermerke
		Veranstalter HTBLA Mössingerstrasse	
		Protokollführer Lukas Peter Sumann	
		Projektbetreuer Dipl.-Ing. Alexander Rodiga	
		Erstellungsdatum: 2024-10-08	

Thema:
Projektstartgespräch / Projektstatusgespräch

Projektname: Event Insight

Teilnehmer (alphabetisch)		Unterrichtete (alphabetisch)	
Name	Klasse, Standort	Name	Klasse, Standort
Hr. Lukas Sumann	W101	Hr. Alexander Rodiga	W101
Hr. Maximilian Lindner	W101	Hr. Herwig Guggi	W101
		Hr. Harald Grünanger	W101

Event Insight:

Lukas Sumann:
Hauptaufgabe: Userinterface und Live-Map

Ergebnisse:

- Recherche der race result Simple API-Typen und Implementierungen in React
- Erfassen der Renndaten

Offene Punkte

- Designen der Userinterface Struktur
- Verarbeitung der Daten (Ergebnistabellen)

Maximilian Lindner:
Hauptaufgabe: Admininterface und Loginportal

Ergebnisse:

- Technologierecherche und Einigung auf React als Framework
- Einbinden der Karte in React

Offene Punkte

- Designen der Admininterface Struktur
- Loginportal

Besprochene Änderungen:

- Race result nicht mehr als Wirtschaftspartner zu benennen.
- Überarbeiten des Blockschaltbildes
- Besprechung, dass für das Verwalten von Events und Loginportal eine Datenbank benötigt wird

**Erledigung
von, Termin**

8.10.2024

gez. Name

HTL Mössingerstrasse
Projektbetreuerprotokoll_Milestone_1.docx

Seite 1 von 1
V01

Abbildung 26: Besprechungsprotokoll am 8.10.2024

AN	ERGEBNISPROTOKOLL ZUR BESPRECHUNG		Eingangs- vermerke
Teilnehmer und Unterrichtete	Veranstalter	HTBLA Mössingerstraße	Erstellungsdatum: 2024-11-26
	Protokollführer	Lukas Peter Sumann	
	Projektbetreuer	Dipl.-Ing. Alexander Rodiga	
am Tag, Datum		Ort/Raum	
Dienstag, 26.11.2024		HTL Mössingerstraße/W101	
Thema: Projektstartgespräch / Projektstatusgespräch			
Projektname: Event Insight			
Teilnehmer (alphabetisch)		Unterrichtete (alphabetisch)	
Name	Klasse, Standort	Name	Klasse, Standort
Hr. Lukas Sumann	W101	Hr. Alexander Rodiga	W101
Hr. Maximilian Lindner	W101	Hr. Herwig Guggi	W101
		Hr. Harald Grünanger	W101
Event Insight: Lukas Sumann: Hauptaufgabe: Userinterface und Racemap Ergebnisse: - Grund-Design des Userinterfaces - Import von Race Result API-Daten in Form einer Ergebnistabelle - Anzeigen von GPX-Strecke mittels Leaflet Offene Punkte - Anzeigen der Teilnehmer der API auf der Live-Map - Fertigstellung der Live-Map Maximilian Lindner: Hauptaufgabe: Admininterface und Loginportal Ergebnisse: - Datenbank erstellt um Daten, wie die GPX Karte und weiteres zu speichern - Backend mittels Express.js - Upload von GPX Daten Offene Punkte - Login Portal für den Administrator			Erledigung von, Termin
26.11.2024 gez. Name HTL Mössingerstraße <i>Projektbesprechungsprotokoll_Milestone_2.docx</i>			Seite 1 von 1 V01

Abbildung 27: Besprechungsprotokoll vom 26.11.2024

AN	ERGEBNISPROTOKOLL ZUR BESPRECHUNG		Eingangs- vermerke
Teilnehmer und Unterrichtete	Veranstalter	HTBLA Mössingerstraße	Erstellungsdatum: 2025-1-28
	Protokollführer	Maximilian Wolf Leopold Lindner	
	Projektbetreuer	Dipl.-Ing. Alexander Rodiga	
am Tag, Datum		Ort/Raum	
Dienstag, 28.01.2025		HTL Mössingerstraße/W101	
Thema: Projektstartgespräch / Projektstatusgespräch			
Projektname: EventInsight			
Teilnehmer (alphabetisch)		Unterrichtete (alphabetisch)	
Name	Klasse, Standort	Name	Klasse, Standort
Hr. Lukas Sumann	W101	Hr. Alexander Rodiga	W101
Hr. Maximilian Lindner	W101	Hr. Herwig Guggi	W101
		Hr. Harald Grünanger	W101
Lukas Sumann: Hauptaufgabe: User Interface und Live-Karte <u>Ergebnisse:</u> - Überarbeitetes Design der Ergebnistabelle für verbesserte Optik und Usability. - Ergebnisse jetzt kategorisiert nach Geschlecht (männlich und weiblich). - Implementierung von Such- und Sortierfunktionen in der Tabelle. - Möglichkeit, TeilnehmerInnen für die Live-Karte auszuwählen, inkl. Suchleiste. - Ausgewählte TeilnehmerInnen werden anschließend auf der Live-Karte angezeigt. <u>Offene Punkte:</u> - Implementierung der Tracking-Funktion: Nutzung von API-Daten zur Berechnung und Anzeige der Teilnehmer*Innenpositionen in Echtzeit. - Echtzeit-Updates der Teilnehmer*Innenpositionen. - Automatische Entfernung der TeilnehmerInnen von der Live-Karte nach beenden des Rennens. - Anzeige des Höhenprofils der Strecke unterhalb der Live-Karte.			Erledigung von, Termin
Maximilian Lindner: Hauptaufgabe: Admin Interface und Backend <u>Ergebnisse:</u> - Admin-Login-Portal mit persistentem Login (Token-basiert, Speicherung von Benutzerdaten in Local Storage). - Prototyp für ein Sponsorship-Tool: Hochladen von Bildern und Anzeige der Inhalte auf der Event-Startseite. - Suchfunktion für Events auf der Landingpage, die BenutzerInnen und Admins differenziert anzeigt (Admins können nur zugewiesene Events verwalten). - Sicherheitsoptimierte Routen und verbessertes Design. <u>Offene Punkte:</u> - Verbesserung der Routen-Sicherheit - Tool zur manuellen und automatischen Eingabe von Event-Informationen für Admins. - Optimierung des Sponsorship-Tools: Einheitliche Sponsor*Innenbilder, bessere Platzierung und Sichtbarkeit auf allen Eventseiten.			
28.01.2025		Maximilian Lindner	
HTL Mössingerstraße			
Projektbesprechungsprotokoll_Milestone_3.docx			

Abbildung 28: Besprechungsprotokoll vom 28.1.2025

3AN	ERGEBNISPROTOKOLL ZUR BESPRECHUNG		Eingangs- vermerke
Teilnehmer und Unterrichtete	Veranstalter	HTBLA Mössingerstraße	Erstellungsdatum: 2024-11-26
	Protokollführer	Maximilian Wolf Leopold Lindner	
	Projektbetreuer	Dipl.-Ing. Alexander Rodiga	
am Tag, Datum		Ort/Raum	
Dienstag, 11.03.2024		HTL Mössingerstraße/W101	
Thema: Projektstartgespräch / Projektstatusgespräch			
Projektname: Event Insight			
Teilnehmer (alphabetisch)		Unterrichtete (alphabetisch)	
Name	Klasse, Standort	Name	Klasse, Standort
Hr. Lukas Sumann	5AHEL, Klagenfurt	Hr. Alexander Rodiga	W101
Hr. Maximilian Lindner	5AHEL, Klagenfurt	Hr. Herwig Guggi	W101
		Hr. Harald Grünanger	W101
Lukas Sumann: Hauptaufgabe: User Interface und Live-Karte <u>Ergebnisse:</u> - Implementierung der Tracking-Funktion: Nutzung von API-Daten zur Berechnung und Anzeiger der Teilnehmer*innenpositionen in Echtzeit - Echtzeit-Updates der Teilnehmer*innenpositionen <u>Offene Punkte:</u> - Richtige Sortierung und Platzierungsvorgabe bei UserResults (Zwischenzeiten) - Wenn man einen Läufer in der ParticipantsCheckbox auswählt soll er oben angepinnt werden - Verknüpfung zwischen Livemap und UserResults - Bei Refresh Auswahl beibehalten - Popup von Läufer bearbeiten (Aktuelle Zeit und Erwartete Zielzeit) - Anzeige des Höhenprofils der Strecke unterhalb der Karte Maximilian Lindner: Hauptaufgabe: Admin Interface und Backend <u>Ergebnisse:</u> - Verbesserung der Routen-Sicherheit - Tool zur manuellen und automatischen Eingabe von Event-Informationen für Admins. - Optimierung des Sponsorship-Tools: Einheitliche Sponsor*Innenbilder, bessere Platzierung und Sichtbarkeit auf allen Eventseiten. <u>Offene Punkte:</u> - Tool zur Bearbeitung von Events und zugehörigen Daten - Funktion zur Bearbeitung von Text 11.03.2025 Maximilian Lindner			Erledigung von, Termin

Abbildung 29: Besprechungsprotokoll vom 11.3.2025

6.4. Arbeitszeitznachweis

Datum	Anfang	Ende	Beschreibung	Dauer in h
18.07.2024	09:30	11:30	Besprechen des Arbeitsplanes über den Sommer	2,00
15.09.2024	17:30	19:30	Strukturierung von Git	2,00
22.09.2024	16:30	18:45	Diplomarbeit erstes Kapitel verfassen	2,25
28.09.2024	13:30	17:00	Recherche zur Race Result SimpleAPI und Dokumentation	3,50
04.10.2024	15:00	18:00	Implementieren der SimpleAPI und Dokumentation	3,00
06.10.2024	18:30	21:30	Vorbereitung für die erste Präsentation	3,00
13.10.2024	16:30	19:00	Dokumentation API Einbindung	2,50
14.10.2024	17:45	20:30	API Daten in einer Tabelle anzeigen: Erste Versuche	2,75
15.10.2024	17:00	19:15	Initiales Design der Website integrieren	2,25
17.10.2024	16:00	20:00	Grundelemente der Webpages initialisieren	4,00
18.10.2024	16:10	19:15	Optimieren der Dropdown Box	3,08
20.10.2024	14:00	16:15	"initial_structure" wurde fertiggestellt und mit "main" gemerged	2,25
24.10.2024	17:30	20:30	Implementierung einer Komponente für die Ergebnistabellen	3,00
25.10.2024	15:15	19:15	Optimierung der Ergebnistabelle	4,00
03.11.2024	17:20	20:00	Nun kann auf der Website der API Link genutzt werden, um Ergebnisse zu erzeugen	2,67
07.11.2024	18:00	21:00	Optische Verbesserung der Webpages	3,00
09.11.2024	16:00	20:45	Einfügen der Live-Map mit Leaflet	4,75
13.11.2024	21:00	23:30	Erstellen eines GPX Parsers	2,50
16.11.2024	16:00	19:15	Finalisierung der GPXMap Komponente	3,25
17.11.2024	17:15	19:30	Mergen von User_ Interface und Backend in main	2,25
19.11.2024	16:00	18:15	Search Bar für LiveMap und Results bearbeiten	2,25
19.11.2024	17:15	20:15	Bearbeiten der HTL-Diplomarbets-Homepage	3,00
19.11.2024	20:00	23:00	React Crashcourse Video anschauen	3,00
25.11.2024	18:30	21:30	Vorberreitung zweite Iteration	3,00
26.11.2024	17:00	20:00	LiveMap wurde verbessert, Marker für Start/Ziel erstellt.	3,00
29.11.2024	17:15	20:15	ParticipantMarker programmiert und Pop-Up mit Daten des Teilnehmers.	3,00
01.12.2024	16:30	19:30	Marker für Zwischenzeiten hinzugefügt	3,00
05.12.2024	17:15	20:45	Marker der Teilnehmer werden aus der API entnommen	3,50
08.12.2024	18:00	21:30	LiveMap und Marker-Optimierung für bessere Übersicht.	3,50
12.12.2024	17:00	20:30	ResultTable hat nun Gruppierungsfunktion	3,50

17.12.2024	16:30	20:30	ResultTable hat nun eine Suchfunktion	4,00
24.01.2025	18:30	21:45	Programmierung einer Teilnehmersauswahl auf der Live-Map	3,25
25.01.2025	07:30	11:30	Dokumentation	4,00
25.01.2025	18:45	22:00	Verfeinern der Webpage-Styles	3,25
26.01.2025	13:15	18:00	unerfolgreiche Entwicklung einer Checkbox Komponente für die Teilnehmersauswahl	4,75
26.01.2025	19:30	22:00	Debugging, Vorbereitung auf Iteration 3 und merging	2,50
27.01.2025	18:30	22:15	Vorbereitung 3te Präsentation	3,75
01.02.2025	16:00	20:15	Redesign von Markern und Trackingbeginn	4,25
05.02.2025	17:15	19:45	Participant Marker Color Bug fixes	2,50
08.02.2025	09:15	12:45	Am Tracking weiterarbeiten	3,50
15.02.2025	15:45	20:00	Prototyp fürs Tracking fertiggestellt	4,25
17.02.2025	18:30	20:15	RaceResult Testevent erstellen	1,75
18.02.2025	15:15	16:45	Redesign der Start und Ziel Marker	1,50
20.02.2025	19:15	22:30	Versuchtes Implementieren eines neuen StartGoal-Marker Icons	3,25
22.02.2025	18:30	19:30	Bugfixing und fertiges Implementieren der Marker	1,00
23.02.2025	17:15	19:15	SearchableSelect Bugfixing	2,00
26.02.2025	18:15	21:00	Tracking: Bugfixes	2,75
27.02.2025	18:45	21:30	Optimierung der ResultTable Gruppierung (MW und AK)	2,75
28.02.2025	15:00	15:45	Organisieren des Stylesheets zur Übersichtlichkeit	0,75
01.03.2025	20:00	21:45	Sortierung in der ResultTable wurde optimiert	1,75
06.03.2025	18:00	20:00	Tracking: Implementierung der Zwischenzeiten	2,00
07.03.2025	16:00	19:00	Fertigstellung des Trackings	3,00
08.03.2025	13:15	14:00	Dokumentation	0,75
09.03.2025	11:00	15:00	Vorbereitung auf die 4te Iteration	4,00
10.03.2025	19:30	22:00	Kleine Bugfixes und UI Changes	2,50
11.03.2025	20:00	21:30	Sortierung in der Checkbox + Popup Verbesserung	1,50
15.03.2025	14:00	15:00	Dokumentation	1,00
19.03.2025	09:30	11:30	Finalisierung der ResultTable und UserResult page	2,00

20.03.2025	16:15	18:00	Reset Button für die LiveMap	1,75
24.03.2025	21:00	22:30	Bugfixes in der elevation-chart	1,50
26.03.2025	19:00	20:30	Dokumentation	1,50
27.03.2025	18:00	21:00	Dokumentation	3,00
30.03.2025	17:00	19:30	Dokumentation	2,50
			Gesamtarbeitszeit (Ist)	173,25
			Soll	160,00
			Offen (Ist-Soll)	-13,25

Tabelle 6.1: Arbeitszeitnachweis von Lukas Peter Sumann

Datum	Anfang	Ende	Beschreibung	Dauer in h
15/09/2024	09:30	11:30	Besprechen des Arbeitsplanes über den Sommer	2.00
19/09/2024	17:30	19:30	Erste Planung und Auswahl der Technologien.	2.00
23/09/2024	16:30	17:45	Grundstruktur für das Projekt machen	1.25
25/09/2024	13:30	17:00	Recherche über Möglichkeiten der Projektrealisierung	3.50
29/09/2024	15:00	17:45	Git-Struktur Änderungen	2.75
02/10/2024	18:45	20:15	YouTube Videos zu React	1.50
05/10/2024	16:45	19:00	Vorbereitung erste Präsentation	2.25
09/10/2024	17:45	20:30	Erstellung der Grundstruktur der Admin-Komponenten	2.75
12/10/2024	17:00	19:15	Header Komponente kreiert	2.25
15/10/2024	16:00	20:00	Projektstruktur in Main verbessert	4.00
17/10/2024	16:00	17:15	Kleinere Sachen und readme Branch zu main gemerged	1.25
19/10/2024	14:30	16:15	Optimierung der Styles	1.75
21/10/2024	17:30	20:30	Fehlerbehebung und Optimierungen	3.00
23/10/2024	15:15	19:00	Strukturverbesserungen im Admininterface	3.75
25/10/2024	17:30	20:00	Blockschaltbild und Strukturplan erstellt	2.50
27/10/2024	18:00	21:00	Logo in Photoshop kreieren und in Code einbinden	3.00
29/10/2024	16:00	20:45	Diplomarbeit weitergeschrieben	4.75
02/11/2024	21:00	23:30	Recherche zu Backend, Express.js und MySQL	2.50
04/11/2024	16:00	19:15	MySQL Datenbank Grundgerüst bauen	3.25
07/11/2024	18:15	19:30	Backend-Projekt hinzugefügt	1.25
09/11/2024	16:00	17:15	Funktionsfähiges Backend das mit der Datenbank verbunden ist	1.25
13/11/2024	17:30	18:15	Upload von GPX-Dateien möglich, Testfunktionen dafür erstellt	0.75
14/11/2024	21:30	23:00	Funktionalität zum Hochladen von GPX-Dateien (noch in der Entwicklung)	1.50
16/11/2024	20:00	21:00	Kleine Änderungen an der Gesamtstruktur vorgenommen	1.00
19/11/2024	21:30	23:30	Allgemeine Farbänderung vorgenommen, package.json geändert	2.00
21/11/2024	19:00	22:00	Main in admin_interface gemerged und alte Seiten aktualisiert	3.00
25/11/2024	18:30	22:00	Vorbereitung für die zweite Iteration	3.50
28/11/2024	21:00	22:00	Upload-Route geändert, speichert jetzt als GPX und Bugfixes	1.00
05/12/2024	18:00	20:00	Event-spezifische User-Pages hinzugefügt (noch in der Entwicklung)	2.00
06/12/2024	20:30	22:00	Diplomarbeit weitergeschrieben	1.50
08/12/2024	19:00	23:00	Dependencies geupdated und AdminResults.jsx kann man Listen hochladen	4.00
12/12/2024	20:30	23:30	Separate Routing-Dateien und Fehlerbehandlung hinzugefügt + AdminLogin-Test durchgeführt	3.00
13/12/2024	20:30	23:30	Login-Portal mit Authentifizierung hinzugefügt, Routen-Schutz implementiert	3.00
17/12/2024	18:30	22:30	Struktur des Logins geändert, Header und Footer angepasst	4.00
22/12/2024	18:30	20:30	Diplomarbeit weitergeschrieben	2.00
25/12/2024	20:00	23:00	Weiter probiert eventspezifische Seiten zum Funktionieren zu bringen	3.00

26/12/2024	21:00	23:30	Code-Optimierungen und funktionierender Login-Logik (Admin sieht nur eigene Events)	2.50
05/01/2025	20:00	22:00	Code hinzugefügt, um doppelte GPX-Dateien in UserLiveMap.jsx zu verhindern	2.00
06/01/2025	19:30	23:30	Logik zum Hinzufügen von Sponsoren hinzugefügt (Fehler vorhanden)	4.00
13/01/2025	18:30	20:30	Eventspezifische Seiten funktionieren, jedes Event hat eine einzigartige Seite	2.00
15/01/2025	18:00	21:00	Minimale Änderungen zu Styles und Code, Aufbau von Admin und User Seiten geändert	3.00
19/01/2025	21:30	23:30	Diplomarbeit weitergeschrieben	2.00
21/01/2025	18:00	22:00	Probleme beim Seiten wechseln behoben	4.00
24/01/2025	19:30	22:15	Fehlerbehebungen, wenn man die Seite neu lädt bleibt man eingeloggt	2.75
26/01/2025	20:00	23:00	Dritte Iteration Vorbereitung	3.00
01/02/2025	19:30	22:30	Bugfixes bei Routen, bessere Routensicherheit	3.00
04/02/2025	20:00	22:00	Style Änderungen für Sponsors, GPXMap und UserLiveMap	2.00
07/02/2025	18:30	22:00	Suchfunktion für Events auf der Landingpage	3.50
10/02/2025	19:00	23:00	Kleine Quality of Life Changes gemacht	4.00
12/02/2025	18:30	22:30	Diplomarbeit weitergeschrieben	4.00
15/02/2025	21:00	23:00	Fehlerbehandlung in Routen und UserLiveMap aktualisiert	2.00
18/02/2025	20:30	23:30	UserLiveMap-Komponente refrakturiert für bessere Lesbarkeit und Performance	3.00
21/02/2025	19:30	23:00	Komponenten refrakturiert für verbesserte Lesbarkeit und Performance; API-Aufrufe optimiert	3.50
24/02/2025	20:00	22:30	Diplomarbeit weitergeschrieben	2.50
27/02/2025	19:00	22:00	UserLiveMap optimiert und unnötige Zeilen im Stylesheet gelöscht	3.00
02/03/2025	18:30	22:30	Stile und Komponenten für verbesserte Lesbarkeit refaktoriert; Checkbox- und Button-Stile verbessert	4.00
05/03/2025	20:30	23:00	AdminHome und UserHome-Komponenten refaktoriert; Hauptklassenstruktur vereinfacht und verbessert	2.50
08/03/2025	18:30	23:00	Vorbereitung vierte Iteration	4.50
10/03/2025	19:30	23:00	Fix für das Marker-Upload-Problem in AdminLiveMap.jsx gemacht	3.50
11/03/2025	19:30	22:30	Event-Darstellung in AdminHome und UserHome refaktoriert, Platzhaltertext aktualisiert	3.00
13/03/2025	21:00	23:30	Admin UI verbessert: Button-Stile und Header-Klasse für bessere Sichtbarkeit aktualisiert	2.50
15/03/2025	20:00	23:00	AddEvent-Komponente hinzugefügt: Formular zur Event-Erstellung mit Eingabefeldern für Eventdaten implementiert	3.00
18/03/2025	18:45	20:15	Event-Erstellungsfunktionalität hinzugefügt: Formular zum Hinzufügen von Events mit Eingabefeldern implementiert	1.50
24/03/2025	17:30	18:15	EditText-Komponente hinzugefügt	0.75

25/03/2025	20:00	22:00	Zeit-Eingabe zu AdminLiveMap hinzugefügt und EditText-Komponente für Text-Upload-Funktionalität refaktoriert	2.00
26/03/2025	18:00	20:45	maptime zu LiveMap-Abfragen hinzugefügt und UserLiveMap berücksichtigt jetzt Startzeiten	2.75
27/03/2025	16:00	17:30	Abhängigkeit chartjs-plugin-crosshair hinzugefügt und Hover-Funktionalität in User-LiveMap	1.50
31/03/2025	23:00	06:00	Diplomarbeit weitergeschrieben	7.00
02/04/2025	22:30	03:00	Letzte Feinschliffe bei Diplomarbeit	4.50
03/04/2025	19:30	22:30	Diplomarbeit fertiggeschrieben	3.00
			Gesamtarbeitszeit (Ist)	189,50
			Soll	160,00
			Offen (Ist-Soll)	-29,50

Tabelle 61.2: Arbeitszeitznachweis von Maximilian Wolf Leopold Lindner

6.5. Wettbewerbe

Event Insight hat beim Jugend-Innovativ Wettbewerb teilgenommen, ist jedoch im Pre-Check ausgeschieden.

Ergebnis des PRE-CHECKS Jugend Innovativ im Schuljahr 2024/25

Von MAKRISEVIC Milena <M.Makrisevic@aws.at>

Datum Fr, 20.12.2024 17:33

An Maximilian Wolf Lindner (5AHEL) <maximilian.lindner@edu.htl-klu.at>

Lieber Maximilian Lindner!

Vielen herzlichen Dank für die Projektanmeldung zum **38. Jugend Innovativ Wettbewerb** mit dem Projekttitel

**Event Insight,
Projekt-Nr. P2461569**

in der Kategorie „Design“ / Spezifikation „**Interdisziplinärer Einsatz von Multimedia, Fotografie und audiovisueller Medien, sowie Mediendesign (Film, Animation, Game-, App- und Webdesign)**“.

Leider müssen wir euch mitteilen, dass der Pre-Check gemäß den Teilnahmebedingungen für das oben genannte Projekt nicht erfolgreich war. Das Projekt ist somit leider für die weitere Teilnahme nicht qualifiziert.

Aufgrund der hohen Anzahl an Einreichungen ist eine detaillierte Begründung der Absage leider nicht möglich. Wir bitten dahingehend um Verständnis.

Für Rückfragen steht das JI-Team jedoch gerne unter 01/501 75 - 514, Fr. Breyer oder DW 562, Fr. Makrisevic zur Verfügung bzw. für Email-Anfragen an: j.breyer@aws.at oder m.makrisevic@aws.at.

Diese Mitteilung soll jedoch nicht abhalten, mit dem Projekt weiter zu machen! Wir wünschen allen Projektbeteiligten weiterhin gutes Gelingen bei der Projektarbeit!

Mit besten Grüßen
Euer JI-Team



Milena Projektmanagerin Entrepreneurship
MAKRISSEVIC Entrepreneurship Jugend | Innovative
Innovativ Youth

Austria Wirtschaftsservice Gesellschaft mbH
Rechte Wienzeile 225, 1120 Wien
T +43 (1) 501 75 - 562
E m.makrisevic@aws.at
www.aws.at | www.jugendinnovativ.at

Abbildung 30: Email Jugendinnovativ

Literaturverzeichnis

- [1] „Race Result,“ race result AG, 28 September 2024. [Online]. Available: <https://www.raceresult.com/>. [Zugriff am 28 September 2024].
- [2] „React,“ Meta Platforms, Inc, 19 September 2024. [Online]. Available: <https://react.dev/>. [Zugriff am 19 September 2024].
- [3] „nodejs,“ [Online]. Available: <https://nodejs.org/>. [Zugriff am 2025 3 30].
- [4] „Express.js,“ [Online]. Available: <https://expressjs.com/en/guide/routing.html>. [Zugriff am 21 10 2024].
- [5] „ChatGPT: "Erkläre warum React als Framework passend zu Event Insight ist",“ [Online]. Available: chatgpt.com. [Zugriff am 1 April 2025].
- [6] V. Agafonkin, „Leafletjs,“ 15 Februar 2025. [Online]. Available: <https://leafletjs.com/>. [Zugriff am 15 Februar 2025].
- [7] „ChatGPT: "Erkläre warum Leaflet eine gute Wahl für die Live-Map ist",“ [Online]. Available: chatgpt.com. [Zugriff am 1 4 2025].
- [8] „Chart.js,“ 1 März 2025. [Online]. Available: <https://www.chartjs.org/>. [Zugriff am 1 März 2025].
- [9] „ChatGPT: "Erkläre was Chart.js ist und warum es eine gute Wahl für Event Insight ist",“ [Online]. Available: chatgpt.com. [Zugriff am 1 4 2025].
- [10] „ChatGPT: "Erkläre kurz was nodejs ist und warum es eine passende Wahl für Event Insight ist",“ [Online]. Available: chatgpt.com. [Zugriff am 1 4 2025].
- [11] „ChatGPT: "Erkläre kurz was Express.js ist und warum es gut zu Event Insight passt",“ [Online]. Available: chatgpt.com. [Zugriff am 1 4 2025].
- [12] „mysql,“ [Online]. Available: <https://www.mysql.com/de/>. [Zugriff am 1 4 2025].
- [13] „ChatGPT: "Erkläre was Mysql ist und warum es zu Event Insight passt",“ [Online]. Available: chatgpt.com. [Zugriff am 1 4 2025].
- [14] „ChatGPT: "Schreibe mir eine Zusammenfassung zur Systemarchitektur von Event Insight",“ [Online]. Available: chatgpt.com. [Zugriff am 1 4 2025].

- [15 „ChatGPT: "Schreibe eine grobe Zusammenfassung von der UserResult-page",“ [Online].
] Available: chatgpt.com. [Zugriff am 1 4 2025].
- [16 „ChatGPT: "Fasse mir Informationen zu Leaflet zusammen",“ [Online]. Available:
] chatgpt.com. [Zugriff am 1 4 2025].
- [17 „topografix“,“ 1 2 2025. [Online]. Available: <https://www.topografix.com/gpx.asp>. [Zugriff
] am 1 2 2025].
- [18 „ChatGPT: "Erkläre mir diese Code-Abschnitte: ..., im Kontext von Event Insight",“
] [Online]. Available: chatgpt.com. [Zugriff am 1 4 2025].
- [19 „ChatGPT: "Erkläre die Features des ParticipantMarkers",“ [Online]. Available:
] chatgpt.com. [Zugriff am 1 4 2025].
- [20 D. Braun, „Daniel Braun“,“ 15 Dezember 2024. [Online]. Available: <https://www.daniel-braun.com/technik/distanz-zwischen-zwei-gps-koordinaten-in-java-berchenen/>. [Zugriff
] am 15 Dezember 2024].
- [21 „ChatGPT: "Erkläre die parseGPXtoElevationData Methode",“ [Online]. Available:
] chatgpt.com. [Zugriff am 1 4 2025].
- [22 „ChatGPT: "Erkläre diesen Code Abschnitt: ...“,“ [Online]. Available: chatgpt.com. [Zugriff
] am 1 4 2025].
- [23 „ChatGPT: "Erkläre die Gruppierungsfunktion der ResultTable",“ [Online]. Available:
] chatgpt.com. [Zugriff am 1 4 2025].
- [24 „ChatGPT: "Erkläre die Sortierfunktion in der ResultTable",“ [Online]. Available:
] chatgpt.com. [Zugriff am 1 4 2025].
- [25 „ChatGPT: "Erkläre die Suchfunktion der ResultTable",“ [Online]. Available: chatgpt.com.
] [Zugriff am 1 4 2025].
- [26 „ChatGPT: "Erkläre die Berechnung der Platzierung in der ResultTable",“ [Online].
] Available: chatgpt.com. [Zugriff am 1 4 2025].
- [27 „Axios Docs“,“ [Online]. Available: <https://axios-http.com/docs/intro>. [Zugriff am 17 10
] 2024].
- [28 „React Router“,“ [Online]. Available: <https://reactrouter.com/home>. [Zugriff am 16 11
] 2024].

- [29 „JSON Web Tokens (JWT): Safe Online and Mobile Authentication and Authorisation,”
] [Online]. Available: <https://medium.com/@kimtai.developer/json-web-tokens-jwt-safe-online-and-mobile-authentication-and-authorisation-f3deb7ec2d0e>. [Zugriff am 30.3.2025].
- [30 „Bcrypt.js,” [Online]. Available: <https://github.com/bcrypt-ruby/bcrypt-ruby>. [Zugriff am 6.11.2024].
- [31 „Why Use MySQL,” [Online]. Available: <https://www.ropstam.com/why-use-mysql-advantages/#:~:text=Reliability%20and%20durability,the%20face%20of%20system%20failures..> [Zugriff am 2.4.2025].

Abbildungsverzeichnis

Abbildung 1.1: Systemstruktur.....	9
Abbildung 2: Produktstruktur	12
Abbildung 3: Scrum Projektplan.....	13
Abbildung 4: Live-Map	14
Abbildung 5: Landing Page	16
Abbildung 6: UserHome	16
Abbildung 7: UserLiveMap	18
Abbildung 8: UserResults	19
Abbildung 9: RACE RESULT 12 Abrechnungsmodus.....	19
Abbildung 10: SimpleAPI Eingabefenster in RACE RESULT 12	23
Abbildung 11: Beispiel für Leaflet Marker (Leafletjs.com)	24
Abbildung 12: StartGoalMarker	28
Abbildung 13: Marker + Popup der Teilnehmenden	29
Abbildung 14: ElevationChart	31
Abbildung 15: ElevationChart-Popup.....	34
Abbildung 16: Nach Jahrgang sortierte Tabelle	37
Abbildung 17: Gefilterte Ergebnisliste	38
Abbildung 18: Startseite des Administratorinterfaces.....	42
Abbildung 19: Webpage des Administratorinterfaces zum Verwalten von Live-Maps.....	43
Abbildung 20: Webpage des Administratorinterfaces zum Verwalten von Sponsoren	44
Abbildung 21: Webpage des Administratorinterfaces zum Verknüpfen von API-Links	45
Abbildung 22: Webpage des Administratorinterfaces zum Bearbeiten von Infotext	46
Abbildung 23: Webpage der AdminEdit Seite.....	47
Abbildung 24: EE-Diagramm der Event Insight Datenbank	73
Abbildung 25: Scrum Projektplan mit Tasks.....	77
Abbildung 26: Besprechungsprotokoll am 8.10.2024.....	79
Abbildung 27: Besprechungsprotokoll vom 26.11.2024.....	80
Abbildung 28: Besprechungsprotokoll vom 28.1.2025.....	81
Abbildung 29: Besprechungsprotokoll vom 11.3.2025.....	82
Abbildung 30: Email Jugendinnovativ	89

Tabellenverzeichnis

Tabelle 6.1: Arbeitszeitnachweis von Lukas Peter Sumann.....	85
Tabelle 6.2: Arbeitszeitnachweis von Maximilian Wolf Leopold Lindner	88

Listings

Listing 1: Beispiel für eine GPX-Datei	25
Listing 2: GPX-Data fetching	26
Listing 3: Parsing der GPXDatei	26
Listing 4: calculateCenter-Methode	27
Listing 5: Mapcontainer	27
Listing 6: StartGoalMarker	28
Listing 7: Berechnung des Fortschritts	30
Listing 8: parseGPXtoElevationData	32
Listing 9: chartData	33
Listing 10: ResultTable Kategorienwechsel	35
Listing 11: Sortierfunktion	36
Listing 12: Suchfunktion	37
Listing 13: Participant Marker Results-Link	38
Listing 14: Berechnung der Platzierung	39
Listing 15: Beispiel einer Fehlerbehandlung beim Abrufen von Daten	48
Listing 16: Beispiel einer Fehlerbehandlung beim Hochladen von Daten	49
Listing 17: Beispielcode für useEffect-Hook	52
Listing 18: AdminHome Anzeige der Event-Daten	53
Listing 19: AdminHome Button zum Bearbeiten des Event-Text	53
Listing 20: Funktion zum Wählen einer LiveMap in AdminLiveMap	54
Listing 21: Funktion zum Hochladen von Live-Maps	55
Listing 22: Funktion zum Speichern der Zwischenzeiten	56
Listing 23: Funktion zum Verknüpfen von API-Links	57
Listing 24: Funktion zum Hochladen von Sponsorbildern	58
Listing 25: Funktion zur Auswahl des Anzeigestatus des Sponsorbilder	59
Listing 26; Funktion zur Bestätigung des Anzeigestatus der Sponsorbilder	59
Listing 27: Funktion zum Speichern des Event-Text	60
Listing 28: Konfiguration des ReactQuill-Editor in editText	60
Listing 29: Funktion zur Initialisierung der Änderung von Event-Daten	61
Listing 30: Funktion zum Speichern der Änderungen von Event-Daten	61
Listing 31: Funktion zum Löschen von einzelnen Event-Daten	62
Listing 32: Funktion zum Löschen von Events	62
Listing 33: Beispiel-Route im Backend	63
Listing 34: Etablierung der Verbindungsdaten für die Datenbank	67
Listing 35: Funktion zur Verbindungsherstellung zur Datenbank	67
Listing 36: Einbinden der Routen und Starten des Servers auf Port 3001	67
Listing 37: Ausschnitt der Routendeklaration aus routes.js	68
Listing 38: Funktion zur Behandlung von Datenbankfehler	68
Listing 39: Funktion zur Behandlung von ungültigen Eingabedaten	68
Listing 40: Funktion zur Authentifizierung des JWT	69

Listing 41: Route zur Überprüfung der Gültigkeit eines JWT.....	70
Listing 42: Funktion zum Prüfen des Anmeldestatus.....	71
Listing 43: Funktion zum Schützen von Routen	72